

**CAPSTONE PROJECT PROPOSAL**

**SecureFlow: An Automated End-to-End Onboarding System with Real-Time  
Document**

In partial fulfillment for the requirements in Capstone 1 and  
Research Subject

**Khent Vincent Amor - BSIT 3**

**John Paul Jeruta - BSIT3**

**Vince Gayem - BSIT3**

**Damon Julian Zaldivar - BSIT3**

**Ela Suasin - BSIT3**

**Jay Vincent Rico - BSIT3**

*Computer Security Track*

Second Semester

2025-26

## Table of Contents

|                                               |           |
|-----------------------------------------------|-----------|
| Table of Contents.....                        | 2         |
| CHAPTER 1.....                                | 5         |
| INTRODUCTION.....                             | 5         |
| 1.1 Background of the Study.....              | 5         |
| 1.2 Statement of the Problem.....             | 7         |
| 1.3 Objectives of the Study.....              | 8         |
| 1.3.1 General Objective.....                  | 8         |
| 1.3.2 Specific Objectives.....                | 8         |
| 1.4 Scope and Delimitation.....               | 9         |
| 1.5 Significance of the Study.....            | 10        |
| 1.6 Definition of Terms.....                  | 11        |
| CHAPTER 2.....                                | 13        |
| REVIEW OF RELATED LITERATURE AND STUDIES..... | 13        |
| 2.1 Foreign Studies.....                      | 13        |
| <b>2.2 Local Studies.....</b>                 | <b>43</b> |
| 2.3 Related Literature.....                   | 60        |
| 2.4 Synthesis of the Reviewed Literature..... | 106       |
| CHAPTER 3.....                                | 110       |
| METHODOLOGY.....                              | 110       |
| 3.1 Research Design.....                      | 110       |
| 3.2 System Development Methodology.....       | 111       |
| 1. Plan.....                                  | 112       |
| 2. Design.....                                | 112       |
| 3. Code (Development).....                    | 128       |
| 4. Testing (Integration).....                 | 129       |
| 5. Deployment.....                            | 130       |
| 6. Operations.....                            | 130       |
| 7. Monitoring.....                            | 131       |
| 3.3 System Architecture.....                  | 131       |
| 3.3.1 Client-server setup.....                | 132       |
| 1. Client Layers.....                         | 134       |
| 2. Service Layer.....                         | 135       |
| 3. Data Layer.....                            | 135       |
| 3.3.2 User Interaction Flow.....              | 136       |
| 3.4 Tools and Technologies Used.....          | 137       |
| 3.4.1 Frontend Technologies.....              | 137       |
| 3.4.2 Backend and Database Technologies.....  | 138       |
| 3.4.3 Deployment and Development Tools.....   | 139       |

|                                                                                          |     |
|------------------------------------------------------------------------------------------|-----|
| 3.3.3 Security architecture.....                                                         | 140 |
| 3.5 Data Gathering Techniques.....                                                       | 144 |
| 3.6 Testing Procedures.....                                                              | 147 |
| 1. Unit Testing.....                                                                     | 147 |
| 2. System Testing.....                                                                   | 147 |
| 3. User Acceptance Testing (UAT).....                                                    | 148 |
| 3.7 Evaluation Criteria.....                                                             | 149 |
| 1. Functionality.....                                                                    | 149 |
| 2. Usability.....                                                                        | 149 |
| 3. Reliability.....                                                                      | 150 |
| 4. Security.....                                                                         | 150 |
| 5. Efficiency.....                                                                       | 151 |
| SUPPORTING DIAGRAMS (Included within Chapter 3).....                                     | 152 |
| Data Flow Diagram (DFD – Level 0).....                                                   | 152 |
| Document Upload Flow.....                                                                | 153 |
| Approval WorkFlow.....                                                                   | 154 |
| Entity Relationship Diagram (ERD).....                                                   | 155 |
| Database Scheme.....                                                                     | 156 |
| Row Level Security(RLS) Policies.....                                                    | 161 |
| Use Case Diagram.....                                                                    | 163 |
| CHAPTER 4.....                                                                           | 165 |
| SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS,<br>AND DISCUSSION..... | 165 |
| 4.2 Overview of the Developed System.....                                                | 165 |
| 4.3 System Development Methodology.....                                                  | 168 |
| 1. Plan.....                                                                             | 169 |
| 2. Design.....                                                                           | 170 |
| 3. Code.....                                                                             | 170 |
| 4. Testing.....                                                                          | 171 |
| 5. Deployment.....                                                                       | 171 |
| 6. Operations.....                                                                       | 171 |
| 7. Monitoring.....                                                                       | 172 |
| 4.4 System Architecture.....                                                             | 173 |
| 1. Presentation Layer (Frontend Client).....                                             | 173 |
| 2. Application Layer (Backend Business Logic).....                                       | 174 |
| 3. Data Layer (Persistence & Object Storage).....                                        | 174 |
| 4.5 Hardware and Software Specifications.....                                            | 175 |
| 4.5.1 Hardware Requirements.....                                                         | 175 |
| 4.5.2 Software Requirements.....                                                         | 176 |
| 4.6 Database Design.....                                                                 | 178 |
| 1. User Table.....                                                                       | 180 |
| 2. User Profiles Table (profiles).....                                                   | 180 |

|                                            |     |
|--------------------------------------------|-----|
| 3. Applicant Table (documents).....        | 181 |
| 4. Document Audit Logs Table (object)..... | 182 |
| CHAPTER 5.....                             | 185 |
| CONCLUSION AND RECOMMENDATIONS.....        | 185 |
| 5.1 Introduction.....                      | 185 |
| 5.2 Summary of Findings.....               | 185 |
| 5.3 Conclusion.....                        | 186 |
| 5.4 Recommendations.....                   | 187 |
| System Improvements.....                   | 187 |
| Future Research Directions.....            | 188 |
| Implementation Strategies.....             | 189 |
| 5.5 Impact of the Study.....               | 189 |
| 5.6 Summary.....                           | 190 |

## **CHAPTER 1**

### **INTRODUCTION**

#### **1.1 Background of the Study**

The modern corporate environment requires rapid and secure onboarding of new talent. Currently, many HR departments, including those in the local community, rely on manual processes: using physical paperwork tracking PDF's via email, using different spreadsheets for candidate status, and manually verifying credentials. That limit was caused by the decline in employee productivity, admin cost height and higher risk of human or human error.

Globally, the rapid shift toward remote hybrid, and distributed workforces transformed employee onboarding into a critical digital platform for modern companies. Organizations worldwide were replacing traditional, paper reliant hiring workflows with automated, cloud-based systems to accelerate candidate integration and reduce time to productivity. According to global human resource industry reports, manual preboarding methods lead to higher candidate drop off rates and increased operational expenses due to administrative overhead. Furthermore international data protection standards mandate that sensitive candidate credentials such as government IDs and financial details, had to be transmitted and stored under strict encryption standards. Consequently, global corporate technology trends heavily emphasize centralized platforms that combine automated workflow tracking with robust database security to safeguard pre-employment data across international borders.

Management had been difficult using traditional onboarding techniques in the local industry due to increasing hybrid and remote job models. Organizations in the Philippines face growing pressure to digitize their operations to reach the standards needed to remain competitive and attract tech-savvy talent in a globalized market.

Without change, local businesses were struggling to handle the high volume of sensitive documentation required for legal compliance and recruitment.

In the Philippines, most local businesses and HR departments still struggled to shift away from fragmented, paper-heavy onboarding practices. Organizations frequently relied on a combination of physical document submissions, unencrypted email attachments, and manual spreadsheet tracking to manage candidate progress. This reliance on disconnected channels creates severed visibility gaps, leaving HR personnel overwhelmed by manual data entry while candidates remain uninformed about their approval status. Additionally, handling physical records and unsecured PDF attachments exposed Philippine companies to significant data privacy compliance risks under the Data Privacy Act of 2012 (RA 10173). As local industries face increasing pressure to modernize their operations and compete for tech-savvy talent, establishing a secure, centralized digital onboarding platform had become an urgent necessity.

Enabling an automatic end-to-end onboarding system with real time document handling offered robust solutions, harnessing modern web technologies. The system removed the delays and security deficiencies found in the traditional manual handling of documents by using automation in the entire workflow from initial recruitment stage to final approval. The document verification was performed by Administrator, while the platform supports real-time updates, allowing submitted documents and status changes to be instantly visible to all authorized users.

By centralizing the entire onboarding lifecycle into a Single Page Application (SPA), the system eliminated the friction caused by scattered communication channels and physical paper handling. Modern cloud native database architectures enforced strict Row Level Security (RLS) policies, guaranteeing that sensitive candidate files remained isolated and accessible only to verified evaluators (HR Personnel/Admin). Furthermore, the integration of serverless

background triggers automated routine tasks such as updating document status (pending, approval, rejected) and sending instant email notifications to candidates without required manual administrative intervention. This seamless combination of human oversight for document validation and automated system logic ensured a highly efficient, secure, and auditable onboarding pipeline for growing organizations.

This capstone project proposed the development of SecureFlow, an Automated End-to-End Onboarding System with Real-Time Document designed for secure and efficient recruitment. The system included a React frontend with a Supabase backend to provide real-time document updates and intense encryption or data security. The project applied the principles of software development and database management for faster processing and storage of documents and serves as a practical application of knowledge obtained by BSIT students in developing secure systems for real industry problems.

## **1.2 Statement of the Problem**

Traditional paper-based onboarding and informal digital methods such as sending files back and forth over basic email were slow, inefficient, and created real security risks for organizations. Managing pre-employment paperwork manually leads to misplaced documents, delayed evaluations, and exposed personal candidate data.

To address these challenges, this study specifically aims to solve the following problems:

- Traditional onboarding relied heavily on manual work, creating process inefficiencies and delaying how fast new hires could complete their preboarding requirements.
- Submitted files were vulnerable to unauthorized changes or untracked resubmissions during the review phase.

- Applicants often send sensitive credentials through non-encrypted platforms and standard email, exposing personal data to potential privacy breaches.
- Manual processes cause slow communication, leaving applicants, HR personnel and Administrator without real-time updates on document status and approval progress.

### **1.3 Objectives of the Study**

#### **1.3.1 General Objective**

The general objective of this study was to design and develop an end to end onboarding system that automates the recruitment and workflow of employee onboarding while enhancing data security and real time document handling.

#### **1.3.2 Specific Objectives**

Specifically, the study aims to:

1. Analyzed user requirements to identify system needs.
2. Designed a role-based Single Page application architecture.
3. Implemented server-side logic via Supabase Edge Functions and secure sensitive authentication and administrative operations.
4. Developed an automated document tracking and real-time status updates.
5. Ensured data auditability by adding a dedicated email tracking column in user profiles.
6. Stored and manage data through a secured database
7. Tested and evaluated the system's accuracy, usability and



performance.

#### **1.4 Scope and Delimitation**

This study focused on building **SecureFlow**, a web based onboarding system designed to replace manual hiring paperwork with a secure, digital process. The app was built as a Single Page Application (SPA) using React and Supabase, giving applicants, HR personnel, and admins their own dedicated portals based on their account roles. Applicants could register, track their onboarding progress, and upload their pre-employment documents. On the other hand, HR and Admins could view uploaded files directly in their dashboard, leave specific feedback on documents, update applicant status from Pending to Approved or Rejected, and manage user accounts. The system also used automated background functions to send real-time email updates whenever an applicant's status changes.

To keep the project manageable and focused on core document management, clear constraints were implemented on what the system did not do. First, all document checks were done manually by HR or Admins. The system did not use AI or machine learning to automatically verify if a document was authentic. Second, SecureFlow strictly handles onboarding document collection, so it did not cover payroll, salary computations, tax processing, or bank integrations. Third, the system relied on standard email and password logins rather than biometric systems like fingerprint or face scanners. Lastly, all uploaded files were required to be in PDF format to keep document previews consistent and secure across the platform.

### **1.5 Significance of the Study**

The development of the system provided practical and technical value to various stakeholders involved in the recruitment and onboarding ecosystem. Specifically, this study benefits the following:

- Applicants - Provided a centralized, secure platform where candidates could easily upload their pre-employment PDF documents, track their approval status in real time, and receive instant feedback without the risk of data loss or privacy exposure associated with unsecured email channels.
- Administrators/HR Personnel - It simplified and sped up the candidate evaluation process by providing an organized dashboard to manage user profiles, review submitted documents, record per-document feedback, and update onboarding statuses without relying on manual spreadsheet tracking.
- Organizations - It enhanced operational efficiency and data security during employee onboarding by implementing role-based access control (RBAC), Row Level Security (RLS), and secure authentication protocols to protect sensitive candidate information and maintain regulatory compliance.
- BSIT students - It served as a practical learning reference and demonstration of integrating modern frontend frameworks with secure serverless backend foundations, database access policies, and automated workflow pipelines.

- Future researchers - It provided a valuable technical benchmark and literature reference for future studies focusing on automated onboarding workflows, secure document handling, real-time data management practices, and cloud-native software architecture.

## 1.6 Definition of Terms

- **Onboarding System:** A structured, web-based platform designed to help organizations digitize, streamline, and integrate new employees into the company by managing credential files and approval workflows.
- **Automation:** The use of software technologies such as database triggers and serverless Edge Functions to execute administrative tasks, trigger notifications, and process status changes with human intervention.
- **Real-time Document Handling:** The capability of the system to automatically update, track, and manage submitted candidate's files, allowing HR evaluators and applicants to see status updates and feedback immediately without manual page reloads or delays.
- **End-to-End Process:** A complete, self-contained workflow that covers all stages of preboarding, from initial candidate account registration and PDF upload to administrative evaluation, status updating (pending, approved, rejected), and automated email notification delivery.
- **Administrator:** A system user with elevated privileges responsible for managing user accounts, assigning role permissions, configuring security policies, providing feedback, updating applicant approval status and overseeing overall platform functionality.
- **Supabase:** An open source, cloud-based Backend-as-a-Service (BaaS) platform that provides SecureFlow with PostgreSQL

database management, Supabase Auth session handling, secure object file storage, and real-time data synchronization.

- **Applicant:** A newly hired candidate who uses the system to submit required pre-employment documents, track onboarding status, and view administrative feedback.
- **HR Personnel:** Authorized organizational staff responsible for reviewing submitted candidate documents.
- **Single Page Application (SPA):** A modern web application architecture built with React that dynamically rewrites the current page rather than loading entire new pages from a server, providing a fast and fluid user experience.
- **Role-Based Access Control (RBAC):** A security mechanism that restricts system access and feature availability based on the assigned user role (Applicant, HR Personnel, and Administrator).
- **Row Level Security (RLS):** A database security feature in PostgreSQL that restricts which rows of data a user could read, insert, update, or delete based on their authenticated identity and permissions.
- **Edge Functions:** Serverless TypeScript functions hosted on Supabase that execute server-side backend logic—such as sending automated SMTP email notifications—in response to system events.
- **Dropzone:** An interactive user interface component that allows applicants to drag and drop or select PDF files for uploading to the system.
- **Upsert:** A database operation that either inserts a new record or updates an existing record if it already exists, preventing duplicate document entries in storage.

## **CHAPTER 2**

### **REVIEW OF RELATED LITERATURE AND STUDIES**

#### **2.1 Foreign Studies**

Petrilli et al.(2022) investigated the impact of digital onboarding processes on new hire experiences in the study Digital Onboarding: Facilitators and Barriers.The primary objective of the study was to evaluate how digital systems help employees integrate into a workplace while maintaining consistent communication and engagement with human resource personnel. Using a qualitative exploratory research design, data were collected through participants' data where they stated their experiences with communication and accessibility.The findings revealed that while digital platforms improved consistency,a lack of interpersonal interaction and slow feedback loops from HR could create significant barriers to successful integration. The researcher concluded that digital systems were more effective when they prioritize continuous, active communication to bridge the gap between automation and human engagement.

Similar findings were reported also by Sani et al.(2022), who examined the relationship between digital onboarding practices and employee experiences in the UK. The study found that digital onboarding could improve employee adjustment in the environment and workplace experiences , however lack of social interaction of the employees and reduced opportunities for relationship building might had had a bad effect on newcomer integration. The researchers emphasized that organizations should combine digital onboarding tools with meaningful communication to strengthen employee connections.

Aligning with Yarbrough (2023) explored virtual onboarding and employee socialization experiences and found out that through communication, collaboration, and opportunities for social interaction could significantly influence the success of virtual onboarding programs. The study revealed that employees experienced difficulties establishing organizational relationships when digital onboarding lacked personal engagement and social support. Similarly, Myburgh (2024) investigated the effects of digital onboarding on newcomer proactivity and socialization outcomes, revealing that digital systems alone might not be enough to encourage information and feedback seeking, and workplace adjustment without organizational support.

Furthermore, Frögéli et al.(2023) conducted a systematic review on the effectiveness of formal onboarding practices in facilitating organizational socialization. The findings stated that structured onboarding programs, continuous guidance with organizational support improves newcomer adjustment and engagement. Likewise, Perez and Poot (2023) found that effective onboarding requires clear communication, employee support, and interaction between newly hired employees and the organization to strengthen engagement ,familiarity and commitment.

The findings from this research were directly applicable to the proposed system, which was relevant to address the core challenge of feedback delays in digital workflows . While the study identified slow feedback as a barrier, this project solved the issue through its real-time capability, ensuring that any changes or action taken by an applicant, employee or HR reflects immediately for all selected parties. This instant update fostered the active communication recommended by the researcher, allowing HR to provide immediate feedback on submitted contracts or documents. Therefore,the proposed system transformed a potential barrier into a facilitator by

streamlining real-time document handling ,allowing users to maintain high levels of engagement throughout the onboarding lifecycle.

Based on the relationship between mental safety, employee engagement, and retention during onboarding, Hartley (2024) concluded that the new employee was more likely to took part actively in an organization and be engaged if he feels safe asking any questions, giving opinions, and expressing any concerns. The findings of the study showed that organizations should create onboarding environments based on the principles of mutual trust to made it easier to engage employees and increase their retention rate.

Effective onboarding strategies in Polish enterprises from the perspective of human resource specialists (Krugielka et al. 2024) include not only administrative orientation but also training, communication, and organizational assistance to made adaptation of new employees much easier. The study concluded that a well-planned onboarding program led to quicker adaptation, better work performance, and organizational commitment.

Likewise, Rani et al. (2024) examined onboarding as one of the methods for sustaining human resources in organizations. It was found out that structured onboarding strategies increase employee self-confidence, improve job satisfaction, and decrease employee turnover due to the fact that new employees improved understand organizational policies, responsibilities, and organizational culture. The researchers stated that organizations should implement onboarding strategies to improve workforce stability and increase retention rate.

Furthermore, Jeske and Olson (2022) analyzed onboarding in the context of virtual work environments and stressed the importance of

communication, availability of resources, and social interaction. With the implementation of structured onboarding systems, it was possible to maintain connection, guidance, and assistance between employees, whereas poor onboarding processes could lead to the reduction of employee confidence and their belongingness.

Onboarding of newly hired employees in startup organizations always faces onboarding challenges because of limited resources, rapidly changing work environment, and a rather casual organizational structure (Pratiwi 2023). Nevertheless, the implementation of structured onboarding, communication, and continuous guidance facilitated adjustment, role understanding, and productivity of employees. Moreover, the researcher stated that systematic onboarding practices were important for successful adaptation of new employees in startup organizations.

The findings of these studies support the proposal of the system since they prove the need for a structured and centralized onboarding management system. The proposed system offers a digital tool that made it possible to implement structured onboarding by organizing document submission, approval, and employee requirements into a clear and automated workflow. With the help of real-time status update and synchronization of information exchange, the system provides immediate information regarding onboarding processes for applicants, employees, and human resource personnel.

According to Bessen (2019), automation technologies had a positive impact on employment because of their potential to increase productivity and create opportunities for new jobs when implemented appropriately. In the study, it was found out that automation increases efficiency of organizational operations by eliminating



repetitive manual activities that allow workers to deal with more complex activities. In other words, the results of this study suggest that rather than replacing humans, automation could be used to complement human capabilities by increasing efficiency and creating jobs.

Moreover, the findings of Syed et al. (2020) were consistent with those of Bessen (2019). In this paper, the authors analyzed the use of RPA in different organizational processes and stated that automation increases efficiency by reducing repetitive activities, improving process consistency and reducing errors. At the same time, the researchers stressed that automation was a complex process that requires proper process design and continuous human engagement to ensure the alignment of automated activities with the needs of an organization.

Hofmann et al. (2020) also studied the benefits of RPA implementation for organizational processes. It was found out that automation allows for faster processing of information, improvement of process accuracy and decrease of administrative load. At the same time, the researchers stated that an organization should pay attention to the identification of activities that could be automated without the loss of human expertise because some of the activities require interpretation, evaluation, and decision-making.

Moreover, according to van der Aalst et al. (2018), automation was beneficial only for structured, repetitive, and rule-based activities because the latter was capable of improving process efficiency but cannot interpret and evaluate data and made decisions based on it. Thus, activities that involve interpretation, evaluation, and decision-making should remain in the hands of humans.

In addition, Folgado et al. (2022) conducted a review of the development of Industry 4.0 in the field of automation and supervisory systems. In the study, the researchers reviewed modern automation architectures, communication systems and intelligent

monitoring systems used in different industrial sectors. As a result, it was found out that automation technologies improve process efficiency and accuracy and provide real-time decision-making through integration of different digital systems. At the same time, the researchers stressed that automation with intelligent supervisory systems could improve process efficiency.

Thus, the results of the selected studies prove the need for automation to improve the onboarding process. Despite the fact that the proposed system did not use Artificial Intelligence or Robotic Process Automation, it applies the principles of workflow automation by creating a unified platform for submission of documents, their approval, and tracking of status. Through automation of the document transmission, the system reduces the delays in the manual coordination of processes while allowing HR administrators to maintain their control over document approvals.

**Groenewald (2004)** conducted a case study entitled *Electronic Document Management: A Human Resource Management Case Study*. This was to examine the implementation of digital document management systems within an HR environment. The study followed a multi-phase assessment and evaluation approach to observe user interaction with digital document handling procedures and examine how the system supported information accessibility and knowledge sharing. Organizational challenges and complex user interfaces identified as major factors that might contribute to the discontinuation of digital systems. The results revealed that electronic document management systems improve document organization and accessibility ,however, their long-term success depends greatly on user acceptance, system usability, and effective implementation.

By supporting these findings, **Davis (1989)** introduced the Technology Acceptance Model, which stated that perceived usefulness and ease of use were major factors influencing users' willingness to adopt

information systems, However it also demonstrated that even technically capable systems might fail when users perceive them as difficult to understand or inconvenient to operate. **Venkatesh et al. (2003)** also studied and found that technology adoption was influenced by performance expectations, effort expectations, and user perceptions regarding the value of a system.

Furthermore, according to **Alavi and Leidner (2001)**, digital information systems improve knowledge sharing and organizational efficiency when they were aligned with user needs and supported by proper implementation strategies. Likewise, the electronic content management systems improved document accessibility and workflow efficiency when integrated with existing organizational processes (**von Brocke et al. 2014**)

The findings from these studies supported the proposed system by emphasizing the importance of usability, accessibility, and workflow alignment in digital document management. That addressed these requirements by providing a centralized and user-friendly onboarding portal designed for HR personnel, applicants, and administrators. The system reduces complexity in document handling Through real-time document tracking, automated status updates, and organized approval workflows while improving transparency and accessibility. By prioritizing simplicity and ease of use, the proposed system addresses the implementation challenges identified in previous research and promotes sustainable adoption throughout the onboarding document lifecycle.

According to Nyame and Qin (2020), RBAC was an effective approach for managing information security because clearly defined roles simplify permission management and improve access control. The study focused on investigated the design and implementation of Role-Based Access Control (RBAC) and its influence on information security within the

study *Precursors of Role-Based Access Control Design in Knowledge Management Systems*. The researchers examined Through conceptual analysis and an illustrative case study of how defining organizational roles affects the protection of sensitive information and prevents unauthorized access. At the same time, the researchers highlight that successful implementation requires careful analysis of user responsibilities to ensure that permissions align with organizational functions.

Supporting these findings, Sandhu et al. (1996) stated that role definitions allow organizations to manage permissions more effectively compared with assigning access individually to each user introduced. The study was about the RBAC framework and demonstrating the assigned permissions according to organizational roles provides a flexible and efficient method for controlling access to information systems. The study emphasized that. Similarly, Ferraiolo et al. (2003) explained that RBAC improves security administration by connecting user access rights with job responsibilities, reducing complexity in authorization management.

Role-Based Access Control (RBAC) relies heavily on maintaining security while adapting to changing responsibilities stated by Zhang et al. (2003). In addition, Hu et al. (2015) also emphasized that effective access control requires authorization policies based on user characteristics, responsibilities, and organizational requirements. Together, these insights demonstrate that secure information systems depend on well-designed access mechanisms that ensure users only access resources necessary for their roles.

The findings from these studies were supported by validating the importance of implementing role-based access control for secure onboarding document management. That applies these principles by assigning distinct roles for HR personnel, applicants, and administrators, ensuring that sensitive documents such as contracts and personal information were accessible only to authorized users. Through structured authorization and permission management, the

system protects data integrity while maintaining efficient document workflows. Therefore, the proposed system demonstrates that secure, real-time document handling could be achieved through properly designed access control mechanisms without requiring complex AI-based security solutions.

Pimentel and Nickerson (2012) compared the WebSocket protocol for communicating and displaying real-time data and with traditional communication methods such as HTTP polling. Then the researchers developed a real-time application that transmitted sensor measurements under different network conditions and evaluated performance based on latency and network overhead. The results showed that WebSocket provides lower latency and improved efficiency by maintaining a persistent bidirectional connection, allowing data to be delivered immediately without repeating client requests. At the same time, WebSocket was suitable for applications requiring continuous synchronization and real-time communication among multiple users.

According to Fette and Melnikov (2011), WebSocket reduces unnecessary network requests and improves the efficiency of real-time web applications compared with traditional request-response communication methods. The researcher introduced the WebSocket protocol specification and highlighted its ability to establish persistent, full-duplex communication between clients and servers. Similarly, Wang et al. (2013) examined WebSocket-based communication and found that maintaining continuous connections improves data transmission efficiency and reduces delays in interactive applications.

Furthermore, Loreto et al. (2011) explored real-time communication technologies and pointed out that WebSocket provided a more efficient approach for web-based applications requiring immediate information exchange. The researchers identified that persistent connections improve responsiveness by allowing servers to directly push updates

to connected users. Likewise, Friendly et al. (2023) designed and implemented an Internet of Things (IoT) communication system using WebSocket and PHP to enable real-time interaction between devices and web applications. The study found that WebSocket provides efficient bidirectional communication, allowing IoT devices to exchange data with minimal delay. The researchers also observed that increasing data size and the number of simultaneous connections affected transmission time, although WebSocket maintained reliable real-time communication suitable for IoT-based applications.

The findings from these studies support the proposed system by validating the use of real-time communication mechanisms for improving document workflow synchronization. By applying WebSocket-based communication principles, the proposed system enables immediate reflection of onboarding activities across user interfaces. When applicants submit documents or HR personnel approve requirements, updates could be delivered instantly to authorized users without requiring manual page refreshes. This real-time synchronization reduces communication delays, improves workflow transparency, and enhances the efficiency of the onboarding document management process.

Škvorc, Horvat, and Srbljić (2014) analyzed the performance of the WebSocket protocol in terms of its full-duplex communication capacity via analytical and empirical research performed at the 37th International Convention on MIPRO. Specifically, the authors determined the time needed for establishing the communication session and compared network traffic in the case of WebSocket usage against plain TCP communication. It was found that while WebSocket adds small overhead when performing the handshake, it provides an efficient two-way communication after that and the overhead becomes insignificant during large-scale data transmission. Thus, it was

concluded that WebSocket allows implementing asynchronous full-duplex communication and was compatible with web technologies.

Lombardi (2015) described the architecture and implementation of WebSocket technology in the field of web development. According to the author, WebSocket establishes a long-lived connection between client and server that allows constant bidirectional communication without need of repeatedly opening new HTTP connections. The study highlighted the importance of such communication model in providing improved application responsiveness, reduced network traffic, and implementation of necessary real-time capabilities.

Deshpande et al. (2024) designed a real-time chatting system based on the implementation of WebSocket technology to increase the efficiency of user communication. Specifically, the researchers implemented WebSocket to enable constant bidirectional communication between client and server and allow sending messages without repeated sending of HTTP request each time. The results of the experiment showed that the system managed to provide low-latency message transfer, constant connectivity, and effective real-time communication.

Therefore, the findings of these studies support the proposed system by proving the effectiveness of the use of WebSocket-based communication for the implementation of the real-time onboarding document management. In particular, by applying full-duplex communication, the proposed system was able to provide instant updating of communication between applicants, employees, and HR administrators every time when documents were be sent, viewed, accepted, or changed.

Muller (2014) examined the HTML5 WebSocket protocol and its application in distributed computing environments to evaluate its effectiveness in supporting real-time communication. The study analyzed different WebSocket server implementations and conducted performance benchmarking using scalable frameworks capable of

supporting multiple simultaneous client connections. The findings demonstrated that WebSocket enables efficient low-latency communication by maintaining persistent bidirectional connections between web browsers and servers. The study concluded that WebSocket reduces communication delays compared with traditional HTTP-based methods, making it suitable for applications requiring continuous synchronization among distributed users.

In their research, Ogundeyi and Yinka-Banjo (2018) focused on the implementation of WebSocket technology for the creation of real-time applications. The researchers tested the capacity of the protocol to provide constant client-server connection and discovered that WebSocket decreases the response time when compared to traditional HTTP communication. Thus, the findings indicate that the technology might be successfully applied for the implementation of applications that require updates and synchronization.

Erkkilä (2018) explored the issue of WebSocket security and identified some typical weaknesses and vulnerabilities that threaten the persistent network connection and require special protection for WebSocket communication. In particular, the researcher stressed out the necessity to implement the system's authentication and communication encryption as well as the validation of requests from the server side.

In their study, Fernando et al. (2024) conducted the comparative performance benchmarking of WebSocket libraries that were implemented in Node.js and Golang. The performance was estimated using such parameters as response time, throughput, latency, and other resource utilization. The findings showed that both systems allowed for establishing and maintaining persistent bidirectional connections, but there were certain differences in the performance of the systems depending on the implementation of WebSocket libraries.

The research findings provide the basis for the implementation of the proposed system as they allow applying the principles of WebSocket



technology to the process of real-time communication within the onboarding document management portal. As the result, using the persistent server connection, the system could immediately notify the user whenever documents were submitted, requirements were checked by HR, or application was accepted by an administrator.

Chawla et al. (2017) studied various data transfer approaches applied in web applications in real-time, such as AJAX polling, Long Polling, Server-Sent Events (SSE), and WebSocket. The authors performed architectural analysis and experimental benchmarking of each technology based on latency, bandwidth usage, and scalability under various loads. According to the results, the best performance for two-way communication could be provided by the WebSocket protocol due to its persistence connection with the client side, while traditional approaches generated extra overhead because of additional HTTP requests. The study concluded that the technologies for communication should be chosen taking into account the particular needs of a real-time application and its update frequency.

Yinka-Banjo and Ogundeyi (2019) designed a financial stock monitoring application that uses the WebSocket protocol in order to send real-time information from the market. It was shown that the protocol allows for immediate transmission of stock price changes without additional client requests. As a result, the study concluded that the WebSocket protocol was efficient for applications with real-time requirements of instant information delivery.

Kularatna et al. (2023) created an agent-based centralized management tool for Security-Enhanced Linux (SELinux) policies' analysis and management with the use of the WebSocket protocol. The authors incorporated the WebSocket technology in order to achieve persistent communication between the centralized management tool and the agents and perform real-time monitoring of the system and policies' changes.

It was found out that the WebSocket protocol increases the responsiveness of the security management system since it enables administrators to get immediate system information and changes. The conclusion made by the researchers was that WebSocket was an appropriate protocol for centralized security management systems.

Muller (2014) analyzed the usage of HTML5 WebSocket protocol in distributed computing environments and considered its capability to manage multiple clients simultaneously. The author came to the conclusion that persistent bidirectional communication improves responsiveness and reduces delays comparing to conventional HTTP-based communication methods.

The findings from these studies support the proposed system by validating the use of WebSocket-based real-time communication for onboarding document management. Since the proposed system requires immediate synchronization between applicants, employees, and HR administrators during document submission, review, and approval processes, an efficient communication mechanism was essential. By implementing a persistent bidirectional communication approach, the system could provide instant status updates and notifications without requiring repeated page refreshes. This reduces communication overhead, improves workflow responsiveness, and ensures that all authorized users receive accurate and timely information throughout the onboarding process.

Cömert, Hekim, and Adem (2016) conducted a comparative study of the WebSocket protocol against traditional web communication methods, including Polling and Long Polling, to evaluate their effectiveness in modern interactive applications. The researchers analyzed each communication method based on server workload, scalability, and latency. The findings revealed that WebSocket provides superior performance by maintaining an open connection after the initial

handshake, reducing the overhead caused by repeated HTTP requests. The study concluded that while traditional communication methods might be sufficient for applications requiring occasional updates, WebSocket was more suitable for highly interactive systems requiring continuous and efficient data exchange.

**Safriadi et al. (2024)** implemented WebSocket technology in an Internet of Things (IoT)-based smart home door security system using the ESP32-CAM with facial recognition capabilities. The researchers developed a system that transmitted security events instantly between IoT devices and users through persistent WebSocket communication. The findings showed that WebSocket enabled real-time notifications, efficient device synchronization, and immediate access control updates, improving the overall responsiveness and reliability of the smart home security system.

**Soewito et al. (2022)** designed a WebSocket-based communication framework for smart home applications within an Internet of Things (IoT) environment. The researchers demonstrated that WebSocket allows IoT devices and web applications to exchange information instantly through persistent bidirectional communication. The findings showed that the protocol reduced communication delays, improved device synchronization, and enhanced overall system responsiveness, making it suitable for real-time IoT applications.

**Saudagar et al. (2024)** proposed a lightweight threshold-based approach for detecting Distributed Denial-of-Service (DDoS) attacks while utilizing WebSocket alerts for real-time notification. The study evaluated the effectiveness of combining threshold-based detection techniques with WebSocket communication to provide immediate security alerts whenever abnormal network traffic was identified. The findings indicated that WebSocket allowed rapid dissemination of security notifications, enabling administrators to respond promptly to potential cyber threats. The researchers concluded that integrating WebSocket with network monitoring systems

enhances real-time threat detection and improves incident response capabilities.

The findings from these studies support the proposed system by establishing the technical advantages of WebSocket-based communication for real-time document management workflows. Since the proposed onboarding portal requires continuous interaction between applicants, employees, and HR administrators during document submission, review, and approval stages, an efficient communication mechanism was necessary. By implementing persistent bidirectional communication, the system could provide immediate updates regarding document status changes while reducing unnecessary server requests. This improves system responsiveness, minimizes communication delays, and ensures a more reliable and interactive onboarding experience.

Gavrilă, Băjenaru, and Dobre (2019) examined the architecture of modern Single Page Applications (SPAs) and their influence on system performance and user experience through a case study approach. The researchers analyzed key architectural components, including client-side rendering and API communication, to determine practices that improve interaction between frontend and backend services. The findings demonstrated that SPA architectures enhance responsiveness, reduce unnecessary page reloads, and provide a more dynamic user experience while maintaining scalability and modularity.

**Huda et al. (2024)** developed a Single Page Application (SPA) using ReactJS for an online printing business to improve the efficiency of customer transactions and business operations. The researchers found that the SPA architecture reduced page reloads and enabled dynamic content updates, resulting in faster response times and a smoother user experience. The study concluded that ReactJS-based SPAs enhance application performance and usability by providing responsive navigation and efficient client-side rendering.

Similarly, Mikowski and Powell (2013) discussed the development of Single Page Applications and emphasized that SPA architectures improve user experience by moving application logic to the client side and enabling faster interactions. The authors explained that SPAs provide a more application-like experience by reducing navigation delays and allowing users to interact with dynamic interfaces more efficiently.

Furthermore, Fink and Flatow (2014) explored modern web application development approaches and demonstrated that client-side frameworks improve maintainability, responsiveness, and separation between frontend interfaces and backend services. The study highlighted that structured frontend architectures allow developers to create scalable applications capable of handling complex user interactions.

The findings from these studies support the proposed system by validating the use of SPA architecture for developing a responsive onboarding document management portal. By applying client-side rendering and asynchronous communication principles, the proposed system allows HR administrators, applicants, and employees to interact with onboarding features without experiencing delays caused by traditional page refreshes. The SPA approach improves interface responsiveness, supports real-time document updates, and provides a smoother user experience throughout the onboarding workflow. Therefore, adopting SPA architecture strengthens the system's ability to deliver an efficient, scalable, and interactive platform for managing onboarding processes.

Nowacki and Plechawska-Wójcik (2016) conducted a comparative analysis of frontend frameworks, including AngularJS, ReactJS, and Ember.js, to evaluate their effectiveness in developing Single Page Applications. The researchers created test applications to assess rendering performance, maintainability, and development complexity across different frameworks. The findings showed that ReactJS

provides strong flexibility and performance advantages due to its virtual DOM architecture, which enables efficient rendering and more responsive user interfaces compared with traditional web development approaches.

Supporting these findings, Gavrilă, Băjenaru, and Dobre (2019) examined modern Single Page Application architectures and analyzed how client-side rendering and API communication contribute to improved web application performance. The study demonstrated that SPA architectures enhance responsiveness by reducing unnecessary page reloads and enabling smoother interaction between users and web systems. The researchers emphasized that proper frontend architecture was essential for developing scalable and efficient applications.

**Rossi (2024)** proposed a scalable Single Page Application framework designed to support next-generation artificial intelligence ecosystems in pediatric healthcare and intelligent financial operations. The study emphasized that SPA architecture improves scalability, maintainability, and responsiveness by allowing seamless interaction between users and complex backend services. The findings demonstrated that combining SPA technologies with modern web architectures enhances system performance while supporting real-time data processing and intelligent decision-making.

Furthermore, **Hossain (2023)** compared Server-Side Rendering (SSR), Single Page Applications (SPA), and Static Site Generation (SSG) as approaches to modern web development. The study evaluated each architecture based on performance, scalability, search engine optimization, and user experience. The findings indicated that SPAs provide superior responsiveness and interactive user experiences by updating application content dynamically without requiring full page reloads, making them suitable for web applications requiring continuous user interaction.

The findings from these studies support the proposed system by providing technical justification for the use of ReactJS and Single Page Application architecture in developing an onboarding document management portal. By utilizing a component-based frontend structure and efficient rendering approach, the proposed system could handle complex workflows such as applicant registration, document submission, and HR approval processes more effectively. The ReactJS architecture enables faster interface updates and supports real-time document status synchronization, ensuring that users receive immediate feedback during the onboarding process. Therefore, adopting ReactJS and SPA principles improves system responsiveness, maintainability, and overall user experience.

Timanovskyi and Plechawska-Wójcik (2020) evaluated the performance of various Single Page Application (SPA) technologies to determine their effectiveness in delivering high-quality user experiences. The researchers used benchmarking indicators, including resource utilization, browser efficiency, and response times under different workloads, to assess the performance of modern frontend frameworks. The findings revealed that optimized SPA frameworks improve application responsiveness, reduce loading times, and support efficient handling of dynamic data. The study concluded that selecting an appropriate frontend technology stack was essential for maintaining performance and scalability in modern web applications.

Supporting these findings, **Phan and Khoi (2023)** conducted a comprehensive study on Single Page Applications, examining their advantages, limitations, and practical implementation guidelines. The researchers found that SPAs improve application responsiveness, reduce unnecessary server requests, and enhance user satisfaction through seamless navigation. However, the study also noted challenges related to search engine optimization, initial loading performance,

and browser memory consumption. The authors recommended adopting appropriate optimization techniques to maximize the benefits of SPA development.

Similarly, **Jadhav et al. (2018)** implemented a Single Page Application using AngularJS to simplify web application development and improve user interaction. The researchers demonstrated that SPA architecture enables efficient client-side rendering, allowing users to navigate the application without frequent page refreshes. The findings indicated that AngularJS-based SPAs improve responsiveness, reduce communication delays, and provide a more interactive browsing experience.

Furthermore, **Nygaard and Klaus (2016)** explored single-page architecture as the foundation for modern web applications. The study discussed how SPA design separates the user interface from backend services, enabling asynchronous communication and dynamic content updates. The researchers concluded that SPA architecture improves application flexibility, responsiveness, and maintainability, making it an effective solution for interactive web-based information systems. .

The findings from these studies support the proposed system by validating the need for an optimized frontend architecture capable of handling large amounts of onboarding data and document transactions. By utilizing a modern SPA framework, the proposed system could provide a responsive interface for applicants, employees, and HR administrators while efficiently managing document submissions, reviews, and status updates. The use of optimized frontend technologies ensures that system performance remains stable as the number of users and stored documents increases. Therefore, implementing a high-performance SPA architecture contributes to a reliable, scalable, and efficient onboarding platform.



Bushong et al. (2021) examined the evolution of microservice architectures and their impact on software scalability and maintainability through a systematic mapping study. The researchers reviewed existing literature to identify common patterns in service decomposition, orchestration, and communication within cloud-native systems. The findings revealed that dividing applications into independent services improves deployment flexibility, scalability, and maintainability. However, the study also emphasized that effective communication between services was essential to manage architectural complexity and preserve system reliability.

Supporting these findings, Newman (2021) discussed the principles of microservice architecture and highlighted how independent services allow organizations to develop, deploy, and scale system components more efficiently. The study emphasized that service boundaries, reliable communication, and proper system design were critical factors in ensuring the long-term maintainability of distributed applications.

Similarly, Dragoni et al. (2017) analyzed the characteristics and challenges of microservice-based systems and found that decomposing applications into smaller, independent services improves flexibility and enables faster development cycles. The researchers emphasized that successful microservice implementation requires careful consideration of service communication, data management, and architectural complexity.

Furthermore, Jamshidi et al. (2018) conducted a systematic review of microservice migration and identified scalability, maintainability, and independent deployment as major advantages of microservice architectures. The study highlighted that modular system structures allow applications to adapt more easily to changing requirements while reducing the impact of modifications on other system components.

The findings from these studies support the proposed system by validating the use of modular and scalable architecture for the HR document management portal. By separating major system functions, such as authentication, document management, and database services, the proposed system could improve maintainability and support future expansion. The use of independent services allows additional features or user groups to be integrated without significantly affecting existing workflows. Therefore, applying microservice principles helps ensure that the onboarding platform remains reliable, flexible, and capable of supporting increasing numbers of applicants, employees, and document transactions.

Taibi, Lenarduzzi, and Pahl (2019) investigated how microservices and DevOps practices support the development and maintenance of cloud-based applications through a systematic mapping study. The researchers analyzed existing approaches related to continuous delivery, deployment strategies, and software maintenance practices to identify methods that improve reliability in cloud environments. The findings revealed that combining modular architectures with automated development and deployment processes enables faster update cycles, improved scalability, and increased system stability. The study concluded that cloud-based infrastructure and DevOps practices were important foundations for maintaining reliable and adaptable modern web applications.

Supporting these findings, **Raj et al.** discussed cloud-native computing with emphasis on designing, developing, deploying, and securing microservices-based applications. The authors explained that cloud-native architectures allow applications to be divided into smaller, independently deployable services, improving scalability, flexibility, and maintainability. The study also emphasized the importance of incorporating security throughout the development and

deployment lifecycle to protect distributed services and data. These principles were relevant to systems that require scalable web services and secure communication between different application components.

Similarly, **Bellundagi** examined the design and implementation of a scalable microservices architecture for digital payment systems. The study demonstrated how separating application functionality into independent services could improve scalability, reliability, and maintainability in systems that handle multiple transactions and services. The findings support the use of microservices for applications that require flexible service management and the ability to scale individual components according to workload.

Furthermore, **Pinheiro et al.** examined performance modeling for microservices-based cloud infrastructures, focusing on how the behavior and resource requirements of distributed services could be evaluated under different workloads. The study emphasized the importance of performance modeling in predicting system behavior and identifying potential bottlenecks in cloud environments. The findings demonstrate that performance analysis could help developers make informed decisions about resource allocation, scalability, and service configuration, which were important considerations when developing cloud-based applications.

The findings from these studies support the proposed system by validating the use of cloud-based and modular architectures for developing a scalable HR document management portal. By utilizing cloud technologies and service-oriented design principles, the proposed system could maintain availability, support future expansion, and handle increasing numbers of applicants, employees, and document transactions. The integration of modular services with reliable deployment practices ensures that real-time document handling features remain accessible and stable while allowing system improvements to be implemented efficiently. Therefore, adopting

cloud-based microservice principles provides a strong foundation for a secure, maintainable, and scalable onboarding platform.

Alshammari et al. (2021) investigated the enhancement of traditional Role-Based Access Control (RBAC) by incorporating trust evaluation mechanisms into cloud service environments. The researchers proposed a Task Role-Based Access Control (T-RBAC) model that evaluates users based on assigned roles, task requirements, and trust levels before granting access to cloud resources. The findings demonstrated that integrating trust-based evaluation with RBAC improves cloud security by reducing unauthorized access while maintaining efficient system performance. The study concluded that adaptive access control models were effective for organizations handling sensitive information in cloud environments.

Supporting these findings, **Abusini (2024)** investigated the application of Risk-Based Access Control (RBAC) in securing smart home environments against evolving cyber threats. The study proposed an adaptive access control model that incorporates contextual risk assessment to determine authorization decisions dynamically rather than relying solely on predefined user roles. The findings demonstrated that integrating risk evaluation with RBAC improves the protection of interconnected devices by limiting unauthorized access while maintaining system usability. The researcher concluded that adaptive access control mechanisms strengthen security in distributed systems by considering both user roles and contextual risk factors during access decisions.

Similarly, **Santos-Pereira et al. (2013)** proposed a secure Role-Based Access Control (RBAC) model for healthcare institutions using mobile agents to facilitate secure information exchange across heterogeneous healthcare systems. The researchers integrated RBAC with a public key infrastructure (PKI) to strengthen authentication and authorization

while enabling mobile agents to access distributed medical information securely. The study found that assigning permissions based on predefined organizational roles improved access control, protected sensitive data, and supported secure interoperability between healthcare institutions. The authors concluded that RBAC provides an effective framework for managing user permissions in environments where confidentiality, integrity, and controlled access were critical.

Furthermore, **Islam et al. (2026)** proposed an identity-centric security model for enterprise web systems that integrates identity authentication, identity profiling, behavioral monitoring, risk evaluation, and policy-based access control within a unified framework. Unlike traditional Role-Based Access Control (RBAC), which primarily relies on predefined user roles and static authentication, the proposed model continuously evaluates user behavior throughout an active session to identify suspicious activities and adjust access decisions accordingly. The findings demonstrated that continuous identity monitoring and adaptive policy enforcement improved anomaly detection, increased the accuracy of access control decisions, and strengthened enterprise security against unauthorized access. The researchers concluded that identity-centric security provides a more adaptive and context-aware approach for protecting modern enterprise web systems.

The findings from these studies support the proposed system by validating the implementation of role-based security controls within a cloud-based onboarding document management platform. Similar to the T-RBAC model proposed by Alshammari et al. (2021), the proposed system assigns specific permissions to HR administrators, applicants, and employees according to their responsibilities. This ensures that confidential documents, employment contracts, and personal information were accessible only to authorized users. By applying structured access control principles, the system improves data

protection, maintains document integrity, and provides secure management of sensitive onboarding information.

Mell and Grance (2011), representing the National Institute of Standards and Technology (NIST), established a standardized definition and framework for cloud computing by identifying five essential characteristics: on-demand self-service, broad network access, resource pooling, rapid elasticity, and measured service. The publication also defined three primary cloud service models: Infrastructure as a Service (IaaS), Platform as a Service (PaaS), and Software as a Service (SaaS). The framework emphasized that cloud computing provides significant benefits, including scalability, accessibility, and cost efficiency; however, it also requires strong security mechanisms to maintain data confidentiality, integrity, and availability.

Supporting this framework, Armbrust et al. (2010) examined the opportunities and challenges of cloud computing and identified scalability, accessibility, and reduced infrastructure costs as major advantages of cloud-based systems. The researchers also highlighted security, privacy, and reliability as important concerns that had to be addressed when storing and processing organizational data in cloud environments.

Similarly, Buyya et al. (2009) explored cloud computing as an emerging model for delivering computing resources as services. The study emphasized that cloud platforms enable flexible resource allocation and allow applications to scale according to user demand. However, the researchers noted that effective management and security controls were necessary to ensure reliable cloud service delivery.

Furthermore, Zhang, Cheng, and Boutaba (2010) analyzed cloud computing architecture and management approaches, emphasizing the

importance of virtualization, resource management, and security mechanisms in maintaining reliable cloud-based applications. The study highlighted that cloud systems had to balance accessibility and scalability with proper protection of sensitive information.

The findings from these studies support the proposed system by validating the use of cloud-based infrastructure for developing a scalable onboarding document management platform. By utilizing cloud storage and services, the proposed system allows applicants, employees, and HR administrators to access onboarding documents from different locations while maintaining centralized data management. Following the security considerations emphasized by Mell and Grance (2011), the system incorporates authentication mechanisms, controlled access permissions, and secure document storage to protect sensitive employment information. Therefore, cloud computing provides the necessary foundation for achieving accessibility, scalability, and reliability throughout the onboarding process.

Rose et al. (2020), through the National Institute of Standards and Technology (NIST), developed the Zero Trust Architecture (ZTA) framework to address security challenges in modern information systems. The publication introduced the principle of "never trust, always verify," which requires continuous authentication and authorization of users and devices before granting access to protected resources. The findings emphasized that eliminating implicit trust, applying least-privilege access, and continuously validating requests could reduce unauthorized access and limit potential security threats within cloud environments.

Supporting these findings, Mell and Grance (2011) established the NIST cloud computing framework and highlighted that while cloud technologies provide scalability, accessibility, and cost efficiency, organizations had to implement strong security controls to protect

data confidentiality, integrity, and availability. The framework emphasized that secure authentication and access management were essential components when deploying applications on cloud infrastructure.

Similarly, **Asundi (2025)** examined the enhancement of Role-Based Access Control in Microsoft Dataverse for enterprise applications, focusing on advanced security models and governance within Power Apps and Dynamics 365 CRM environments. The study emphasized the importance of properly defining user roles, permissions, and governance policies to prevent unauthorized access to enterprise data. The findings demonstrate how RBAC could provide structured access management while supporting security and governance requirements in complex business applications.

Furthermore, **Shin et al. (2024)** proposed an architecture for enhancing communication security in Internet of Things (IoT)-based microgrids using Role-Based Access Control (RBAC). The researchers integrated RBAC with Data Distribution Service (DDS) communication to regulate user and device permissions through predefined roles and authorization policies. The findings demonstrated that the proposed architecture effectively reduced unauthorized access and strengthened communication security while maintaining efficient real-time data exchange among interconnected devices. The study concluded that implementing RBAC improves access control, enhances cyber resilience, and supports secure communication in distributed systems.

The findings from these studies support the proposed system by validating the implementation of strong authentication and controlled access mechanisms within a cloud-based onboarding document management platform. Following the Zero Trust principles established by Rose et al. (2020), the proposed system ensures that users were verified before accessing sensitive onboarding documents and system functions. By combining role-based permissions with secure authentication practices, HR administrators, applicants, and employees could access only the information required for their specific responsibilities.



This approach strengthens document confidentiality, prevents unauthorized access, and ensures secure management of sensitive employment records throughout the onboarding process.

The systematic review of literature on Zero Trust Architecture (ZTA) was conducted by Gambo and Almulhem (2026). Gambo and Almulhem (2026) described that the Zero Trust model was based on the principle of "never trust, always verify," meaning that the users and their devices should constantly be verified, trust was being evaluated on-the-fly, access was provided depending on certain circumstances, and the principle of least privilege was applied. In addition, the authors highlighted that this security architecture was successfully deployed in cloud computing, IoT, healthcare, enterprise applications, and industry. However, there were some difficulties associated with its implementation: they include deployment complexity, scalability issues, and preparation for implementation among others. Thus, the literature reviewed by Gambo and Almulhem (2026) proves that Zero Trust Architecture increases the effectiveness of access management and minimizes unauthorized access risks.

NIST Cybersecurity Framework (2018) offers a risk-based methodology that could be utilized by organizations to assess and mitigate cybersecurity risks. The framework includes the five functions of cybersecurity, namely Identify, Protect, Detect, Respond and Recover. It was necessary to implement security controls that provided protection against cybersecurity risks. Asset management, access management, protective technologies and recovery planning were among controls that were needed to ensure protection of sensitive digital information.

To prove the need to conduct cybersecurity risk assessment, Tounsi and Rais (2018) analyzed cybersecurity risk assessment methodologies

and revealed that identification of vulnerabilities and threats was a significant stage in developing security strategies. According to researchers, it was necessary for organizations to apply risk management process to protect information assets and increase cybersecurity preparedness.

Similarly, Craigen, Diakun, and Purse (2014) analyzed cybersecurity frameworks and their importance for improvement of organizational security policies and practices.

In addition, Von Solms and Van Niekerk (2013) highlighted the significance of cybersecurity awareness and governance in ensuring the safety of digital information systems. It was pointed out that cybersecurity involves not just technical aspects, but also the right policy, processes, and organization practice.

The results obtained from these studies confirm the necessity to implement structured security practices to ensure the safety of onboarding documents and user data. As it was suggested by the NIST Cybersecurity Framework and its risk management approach, the proposed system was able to recognize possible security risks, control them in a preventive way, and ensure the reliability of the documents being protected.

The suggested solution fits into the NIST (2018) framework because of the use of proactive protection of data belonging to applicants. Through the application of role-based authorization and secure document management, the project took care of the "Protect" and "Identify" capabilities of the framework. In doing so, the

cybersecurity threat was minimized while managing electronic contracts and identity documents.

Kamara & Lauter (2010) introduced a cloud storage framework that was meant to ensure the security of private data through cryptographic approaches. The researchers suggested that a combination of encryption, key management, and data possession proofs could be used in order to enable cloud service providers to store data while having no access to its content. It was established that the use of cryptography enhances security of data.

The study carried out by Kamara & Lauter (2010) emphasizes the role of the security methods used in the proposed system. Considering the fact that the project involves handling employment documents in a cloud setting, it was vital to focus on confidentiality and integrity. The research ensures the use of secure access controls and authentication encryption in the project so that the applicants' files were not viewed or altered illegally.

## **2.2 Local Studies**

A web-based solution called "Development of Electronic Business Permit and Licensing System (eBPLS)" was introduced by Oñate, Omorog, and Babol (2018) in the Philippines to simplify the permit process and renewals. Following the Modified Princeton Project Management Methodology, the authors managed to develop a platform which allowed the users to submit the requirements and monitor the processing of the permit electronically in order to prevent delays. It had been

found that the developed system proved to be highly efficient; the processing took just one day and the acceptability of the eBPLS was rated high by the government officials and IT specialists.

According to Jaca (2022), the Electronic Document Management and Tracking System (EDMTS) was created in order to enhance the process of document management and tracking within the PhilHealth Regional Office of the National Capital Region. This research identified issues with missing documents, poor distribution, poor tracking, and difficulties in document retrieval, which were solved by designing an online platform for storage, distribution, sharing, and tracking of electronic documents. The design involved different modules including user management module, document classification module, action management module, and document tracking module. Through these modules, users could easily manage document transactions via the online platform.

This research justifies the need for the proposed onboarding system, since both systems were interested in making the process of documentation easier by the use of centralized web applications. Just like EDMTS, the proposed system offers an online platform where applicants could upload their documents and human resource officers could track the status of the document. The addition of real time synchronization made it easy to monitor the document status.

According to Amo and Labuguen (2023), the researchers had designed an automated HR system to enhance processes such as recruitment, recording of personnel details, and onboarding for TNVS operations. With the help of the Agile framework, the research team was able to incorporate several HR modules within the digital platform in order to minimize paper works and facilitate better record management. From the findings, it was observed that automation helped in improving the efficiency of onboarding and monitoring through creation of a centralized database of all data. On the contrary, validation and

approval of the process still needed the involvement of HR staff directly.

This research was in favor of the design of the proposed system, which combines both digital automation and human control. The emphasis of this project to give HR a real time environment to analyze and approve the documents of the applicants was in accordance with the need for human involvement in the TNVS study. With the development of a real time reflection of all the changes made to the documents, the proposed system ensures a more efficient interaction of HR, applicants and employees using the same platform.

The application of a Human Resource Information System (HRIS) in the context of Yngen Datacom Corporation was investigated in the study conducted by Caparas et al. (2023). In the study, the effectiveness of the use of HRIS for managing employee information and the ways in which HRIS made HR processes more efficient by integrating human resource functions with information technology were analyzed. Some of the criteria for analyzing an HRIS included the efficiency, usability, and satisfaction of users with HR information systems, which increase the accessibility and organization of information about employees.

The research was relevant to the proposed onboarding system as both of them seek to enhance HR functions using technology. Just like in the HRIS adopted by Caparas et al., the proposed onboarding system integrates all the information about employees and job applicants but did not involve any paperwork. It took this idea further and focuses on onboarding processes, enabling HR staff to check documents and monitor their progress.

A study carried out by Magtoto and Garcia (2019) aimed at coming up with a workflow management system for automation of permits. They incorporated various aspects like role-based access and approval

processes in order to enhance visibility and cut down on delay times in the local government agencies. It was noted that the implementation of the process improved efficiency in addition to giving administrators control over the documents being processed and made it easy for users to track the progress of the process. It was concluded that workflow automation was necessary for enhancing visibility during the whole process.

This workflow-oriented design was a key element of the system under consideration, which applies the very same routing scheme since the submitted documents of applicants appear in real-time for HR consideration. This guarantees that the same degree of transparency and visibility was provided as in the case of the local government system, thus enabling applicants and employees to follow up with their onboarding in real-time. Thus, the focus was made on automatic status monitoring and routing, which helps to increase efficiency and speed up the process. These results were fully incorporated in this project, which provides full control over the approval process in real-time.

Castro et al. (2022) designed a system called DocTrack: A Dynamic Document Tracking System for the University of Northern Philippines which aims to enhance document tracking and process workflow. The system was intended to aid paperless transactions in an organization since the system had features that track the documents starting from its creation until approval. The researchers highlighted the significance of document tracking, workflow automation, and centralized management. The researchers had found out that digital document tracking systems increase transparency and document tracking capability.

The present research was pertinent to the proposed system since both systems share a common goal of streamlining document workflow through tracking and monitoring capabilities. Like the DocTrack system, the

proposed onboarding system tracks documents at various stages of approvals, giving the HR administrator and the applicant an opportunity to monitor their progress. Through the introduction of real-time updating features, the proposed system made improvements to the conventional tracking techniques.

The "Electronic Document Archive Management System (EDAMS)" was created by Caluza (2017) as an alternative to paper-based file storage in the registrar's office of a Philippine university. The methodology of the study consisted of digital indexing, archiving, and searching of documents to make the process more effective and secure. According to the results of the research, the implementation of the system had increased the speed of document retrieving and decreased the chances of losing documents. The conclusion of the research was that digital archiving was more effective than paper-based one.

This research study was very useful for the proposed system because it emphasizes the security of storing and managing the HR documents. Just like the EDAMS system, the proposed system acts as a central electronic storage for the contract and personal files of both applicants and employees, thus eliminating the manual and time-consuming process of document management. Making sure that the archived documents were synchronized in real time when the HR department took an action, made sure that we got the same benefits in terms of security and monitoring that Caluza discovered in his research.

Jaca (2022) also supports digital document archiving by demonstrating how centralized electronic repositories improve document organization and retrieval. The Electronic Document Management and Tracking System provided structured storage and monitoring of organizational documents through a web-based platform. The study found that digital

document management improves accessibility, reduces document-related issues, and allows organizations to manage records more efficiently compared to manual filing methods.

This study supports the proposed system because the onboarding portal also requires secure storage and retrieval of important documents such as applicant requirements, contracts, and employee records. Similar to EDMTS, the proposed system replaces manual document handling with a centralized repository where authorized users could access updated records. The integration of real-time updates strengthens document accuracy by ensuring that changes made during the approval process were immediately synchronized.

In this study by Argana, Sunico, and Francisco (2020), the authors had designed an automatic Human Resource Information System (HRIS) with digital archiving capabilities to ensure efficient records management and leave management in a local college. The RAD model was used to design this system and included security verification and control mechanisms to make sure that the employees' personal data was protected. The findings, which were analyzed according to ISO 9126 criteria, revealed that the system had been able to get very high scores on usability and efficiency, eliminating manual paperwork. The findings of this research prove that integrated digital archiving greatly improves the organization of documents and access to them.

As it could be seen, the successful outcome of the local HRIS study supports the use of the proposed system because both studies focus on the transition from manual paperwork to digital records management with the protection of sensitive information. The use of digital archiving technologies and controlled access to records was one of the main elements of both studies. Moreover, in the proposed project, the



portalwassuggested to update the data in real time for HR, applicant, and employee sides.

According to Caparas et al. (2023), the authors explored the use and effectiveness of a Human Resource Information System in the implementation of HR processes. In particular, the authors underlined the fact that HRIS systems could offer enhanced management of data about the employees through organizing the information electronically. The importance of system quality, user acceptance, and efficiency in supporting HR processes was pointed out by the authors.

The research underlines the importance of the designed onboarding system since the two systems had common goals related to offering an organized and safe digital platform for management of HR information. Just like HRIS systems, the designed onboarding systemwasaimed at providing the controlled access to confidential information about the applicants and employees. Authentication and permissions allow only HR administrators, applicants, and employees to had access to the information related to their needs.

Gonzales et al. (2016) created a web-based document management system called "Web-Based Document Management and Information Dissemination Through Email and SMS Notification With Members Attendance Monitoring System for LCFEA." Its aim was the optimization of the organization's document handling and timely dissemination of information through automated notifications. A program capable of centralizing documents and notification modules was created and evaluated in terms of its functionality and usability. Results proved that it helped to organize the documents better and to disseminate information faster

than it would be possible manually; therefore, it increased organizational efficiency.

It confirms the proposed system as both projects concentrate on the handling of digital documents and timely notifications. Like the study's results, the portal in question shows the usefulness of using a combination of web-based management and automated communication. Using such a centralized approach for the handling of documents, the proposed portal could make sure that the applicants and administrators were always aware of the status of their documents.

Libadia et al. (2025) created a web-based Electronic Records Management System (ERMS) that centralized all record keeping and facilitated information gathering and processing. Libadia et al. conducted interviews and performed feasibility analysis to determine the system requirements. They used various technologies such as Laravel and MySQL to develop various modules in order to allow the system to be capable of real time data monitoring and generating reports. It was shown that the system enhanced the efficiency of record management by providing an easy-to-use and secure interface for retrieving information. The assessment, according to the ISO 25010 model, showed that the ERMS was capable of solving problems related to manual record keeping through optimized digital workflow.

The study was highly relevant due to the fact that the proposed system uses similar technologies to manage and monitor documents in real time. In addition, both studies recognize the necessity to provide for secure storage and effective workflow management in order to increase administrative efficiency. Thus, the ability of the researchers to implement real time data monitoring for organizational records gives additional evidence that a proposed model could work successfully in practice.

Dela Rosa (2022) created the "Web-Based Management Information System of Cases Filed with the National Labor Relations Commission" aimed at enhancing case filing and tracking processes. Using the Agile Software Development methodology, the researcher managed to create a system that had elements of case management, reporting and database security mechanisms along with compliance with the data privacy policies. The results showed that the designed system was evaluated positively by experts and end users in terms of its efficiency and effectiveness of the workflow optimization and status tracking. It was concluded that digital management was needed in order to improve the accuracy of case records.

It might be stated that the goals of the current study coincide with the ones of the described research because both systems presuppose submission of confidential records for review by the administrator through the secure platform. Moreover, the creation of a digital platform for status tracking allows to gain similar efficiency and transparency that were found during the Dela Rosa study.

Salvador et al. (2024) had created a web-based database management system to assist in the data management of project data and real-time monitoring of activities at various institutions. The authors had implemented a centralized database that consists of two modules, namely data collection module and real-time monitoring module. The results showed that the system helped greatly to improve information sharing, collaboration, and data retrieval by participants. It was concluded by the authors that the centralized access to the updated information improves decision making and allows organizations to track their participation more efficiently.

The relevance of this study to the proposed system lies in the fact that both projects were based on centralized web technologies that help to keep information updated in real time. Just like the consortium database, the proposed portal allows the administrator to monitor applications and applicant status via the web interface. Thus, the results provided by the authors regarding the benefits of the centralized access to information and data retrieval confirm the objectives of the proposed portal in terms of creation of a central point for HR and applicants.

The researchers Abrajano et al. (2024) came up with a cost-effective IoT-based water quality monitoring system that could transmit real-time environmental data to a cloud database as well as provide users with notifications about the data. The research entailed designing a monitoring system that included sensors which transmitted the collected data to a web application for real-time access. The study demonstrated that the system could improve accessibility to information and help users get access to up-to-date data because of integration with the cloud. The researchers stated that timely transmission of data and updates were necessary for improving system responsiveness.

Although the setting of this study was related to environmental monitoring, the technical principles which could be used to justify the use of real-time communication in the design of the monitoring system could also be applied to the proposed system. The research results confirm that immediate updates of the information were important to improve the responsiveness of the system, which was one of the primary objectives of the proposed onboarding portal. Thus, it proves the necessity of instant status reflections and notifications for HR and applicants in the project.

Libadia et al. (2025) developed a web-based Electronic Records Management System (ERMS) designed to improve record storage and accessibility within Philippine government offices. The researchers implemented a centralized platform that combined data storage with document processing to replace traditional, slower manual methods. The findings showed that the system significantly enhanced management efficiency and reduced the time required for information retrieval. The study concluded that digital records management was a far more effective method for organizing and monitoring organizational data than manual record-keeping.

This study was relevant to the proposed system because the onboarding document management portal also utilizes a centralized web architecture for storing and managing sensitive applicant documents. Both systems aim to improve administrative efficiency by replacing manual filing with digital workflows. The researchers' success in reducing retrieval times through a web portal provides a validated model for the proposed project's document handling features. This reinforces that a centralized, digital approach was the most effective way to manage the documents required for an end-to-end onboarding process in a local setting.

Argana, Sunico, and Francisco (2020) conducted research aimed at developing a Human Resource Information System with a digital archiving component that would enhance the organization of employee documentation and its accessibility. The authors used the Rapid Application Development (RAD) framework to create a platform that would centralize all administrative procedures and protect sensitive information using a MySQL database. Evaluation of the project based on ISO 9126 criteria showed that the system enhanced usability and functionality when compared with the manual processing of documentation.

The current study supports the proposed project as both projects deal with digital archiving of documentation within the context of human resource management. The research results show the importance of having a secure and centralized system of storage of documentation of applicants and employees to ensure the effectiveness of the response of the company to any changes in the situation. By implementing the features of digital handling of documentation like those provided by the archiving system by Argana et al., the proposed project ensures the availability of documentation for HR administrators.

Grepon (2021) developed and deployed a web-based e-justice system to keep track of the cases docketed and stored in the Regional Trial Courts of Northern Mindanao. The researcher employed the System Development Life Cycle (SDLC) model to design a solution to convert court records into digital form and facilitate workflow processes. It was found out that the implementation of the said system had greatly helped in making the process of document tracking and information retrieval easier and faster by minimizing the problems related to manual document management. The study highlights the fact that digital document management was important to ensure reliable and efficient transaction processes in a legal environment.

The goals of the current research and the proposed system were similar as both were meant to handle sensitive documents via web-based application. The proposed system would help in achieving the goals of the research in terms of providing a web interface for status tracking and workflow management. The research had proved that an effective web application was capable of facilitating complex administrative process and document tracking.

A web-based management information system of cases filed with the National Labor Relations Commission" was created by Dela Rosa (2022) to help with the monitoring and management of labor records. The system was designed with modern technology and used for case monitoring and status tracking. The research revealed that the creation of the web-based system helped to manage records effectively and improved the process of tracking cases. The study concludes that digital status tracking was vital to improve the management of labor-related documents.

This research backs the proposed project because both of them were related to document management and status tracking using a centralized web-based tool. As in case management, the proposed project helps HR administrators to follow the progress of applications' submission and made necessary changes on time. The results of this research prove that the digital status tracking of documents was the most effective way to improve the transparency of the workflow. This strengthens the architecture of the proposed system since applicants and HR got information about the process of onboarding in real time using the live portal.

In this context, De Jesus et al. (2022) designed the "Institutionally Farmed Research Online Repository and Management System (InFORMs)" to archive and organize academic documents within the institution. The authors used a web-based repository, with digital indexing and retrieval abilities, to make documents more easily accessible. As a result, the research found that the use of such a system led to significant improvements in the efficiency of retrieving and organizing the information. In general, the authors' conclusion was that an online repository was a much better tool for document management compared to traditional paper-based filing systems.

This research was very important from the perspective of the proposed system, because both projects use centralized document repositories to manage digital documents. The findings prove the effectiveness of implementing cloud-based storage and retrieval capabilities in the proposed HR portal for managing applicant contracts and employees' documents. In general, both the repository and the proposed project aim at providing users with easy access to sensitive documents. It proves that a centralized digital repository was a much better solution for organizing documents through the entire onboarding process.

Republic Act No. 10173, also referred to as the Data Privacy Act of 2012, was the main Philippine legislation addressing personal information protection. The act creates the National Privacy Commission and requires organizations to provide organizational, physical, and technical security measures for data subjects. This legislation raises privacy standards in the Philippines as it sets strict criteria of compliance with such aspects of personal data processing as transparency, legitimacy, and proportionality.

Republic Act No. 10173 (2012) was the legal framework for the development of the suggested system and determines how personal information of applicants and employees should be handled. The use of cloud storage with restricted access permissions and encryption of authentication in the system implementation was aimed at compliance with technical security measures required by the act.

A study conducted by Tosper et al. (2025) resulted in the creation of WebYu: A Barangay Yuson Web-based Information Management System Initiatives due to problems posed by manual systems and lack of proper management of information in Barangay Yuson, Guimba, Nueva Ecija. This research was conducted through developing a web-based



information management system for Barangay Yuson via the Scrum methodology in order to facilitate a process of record centralization. The research employed ISO 25010 and Technology Acceptance Model (TAM) for evaluation and revealed that there was a positive effect of the system on information management, efficiency of work with information and its usability.

The study conducted was in support of an idea of developing an onboarding document portal as both systems aim at replacing fragmented manual systems with centralized systems based on web-based technologies. Like in the case of WebYu, the onboarding document portal was a system, which unifies the space where people might submit, access and monitor the information.

Eder and Doroja (2013) had designed a software security model aimed at safeguarding confidential transactions in Philippine academic organizations. This process entailed installing a software gateway through which user authentication, authorization, and encryption could be used to ensure the safety of web-based applications. It had been found that the above-stated multiple layers of security were able to preserve the confidentiality and integrity of the documents transmitted remotely.

The concepts that had been formulated by Eder and Doroja (2013) were applicable in the analysis of the document processing feature of the proposed system. Similarly to the academic gateway, the proposed system was going to implement a secure portal to transmit and approve digital contracts. Ensuring that the only individuals who had access to the document were the authorized HR personnel and the applicant ensures the security first attitude.

In the research carried out by Gamba et al., (2014), the attention was given to creation of a records archiving and repository of electronic documents for multi-campus organizations with the use of a centralized system. The encryption and user authentication process were used in this research for securing files when stored and exchanged. Results from the above study show that digital repository made a big impact on improving accessibility and collaboration by reducing the associated risks posed by manual filing of documents.

This project addresses the objectives set forth in the research by Gamba et al. (2014). This was due to the fact that in this project, there was centralization of HR documents in a secure digital repository. This project eliminates the need for manual handling of documents and introduces digital interface. From the above study, we find out that authenticated access should be used to gain access to sensitive files.

Angala et al. (2023) developed a web-based Document Management System (DMS) for the Ilocos Sur Polytechnic State College in order to update their documentation process. Using WAMMI (Website Analysis and Measurement Inventory), the authors assessed user satisfaction and efficiency. They found out that users were very satisfied with how the system allowed the organization to manage digital documents since there was a noticeable decrease in the use of paper-based management within the institution.

Implementation of the suggested system was consistent with digital transformation discussed by Angala et al. (2023). The project was aimed at reducing delays related to the contract approval process by offering a single platform for contract submission and approval. The

article clearly underlines the importance of using a user-friendly document digitalization system for HR productivity improvement.

This study was conducted by Bumalod and Velasco (2024) and was centered around a combined information security design that integrates the concept of Role-Based Access Control (RBAC) along with the use of sophisticated encryption techniques and document classification. It was important to note that the methodology used in this paper was centered around the design of a secure intranet document management system. The findings of this research show that such an integrated method enhances confidentiality and prevents any form of unauthorized access.

The security model designed by Bumalod and Velasco (2024) corroborates the technical implementation of the proposed system. As the proposed system relies on the role-based permissioning and cloud-based storage of the data, it would be able to classify the applicant's data depending on the role. Therefore, this research proves that an integrated security system was required to protect sensitive HR records.

As evident from the local literature reviewed herein, there was a definite tendency towards the digitalization of administrative, human resource, and documentation procedures in the Philippines in order to cope with the inherent drawbacks associated with paper-based practices. Namely, studies conducted by Oñate, Omorog, and Babol (2018); Magtoto and Garcia (2019); Dela Rosa (2022); and Libadia et al. (2025) prove that the implementation of web portals made the organization's procedures much more advanced, allowing for automation

of applications, permit licensing, and case management. It was proven by all these scholars that the shift to online platforms results in a drastic decrease of the processing time, fewer errors in manual filing and increased transparency of the procedure because of automated status tracking. In turn, research conducted by Amo and Labuguen (2023) and Grepon (2021) proves that the creation of digital repositories designed specifically for HR procedures made the system more responsive and ensures that communication between the administrators and applicants took place in a seamless manner. Thus, the evidence presented could be considered as a powerful proof of the need for the proposed system and its objectives.

Apart from operational efficiency, the studies conducted locally made it clear that the successful introduction of digital documents was tightly connected with a solid security system and compliance with the laws of the Philippines. As a core element of this theme, there was a Republic Act No. 10173 (2012), also called the Data Privacy Act, which prescribes the obligatory technical and organizational measures required for any system working with the information about individuals' private life. Proceeding from the legal base provided by the mentioned act, the need for multi-layered security measures, including such solutions as encryption, secure gateways, and digital archives, was discussed in such works as Eder and Doroja (2013), Bumalod and Velasco (2024), and Caluza (2017). At the same time, the results of the researches of Argana, Sunico, and Francisco (2020) and Nyame et al. (2020) confirm that RBAC plays an important part in making sure that the confidential information cannot be viewed by unauthorized persons. It means that the findings of the abovementioned authors confirm the architecture of the proposed system with secure cloud storage and role-based portals for employment contracts and official documents.

Lastly, the literature emphasizes the significance of reliability and user-centered design when it came to promoting the adoption of the system and achieving data precision. Gamba et al. (2014) and Angala et al. (2023), for instance, had proved that the use of centralized electronic document storage systems helps not only to speed up the process of information search but also increases the level of user satisfaction due to the creation of a more user-friendly interface. In addition, the findings of De Jesus et al. (2022), Gonzales et al. (2016), and Abrajano et al. (2024) had shown that the use of real-time tracking mechanisms, automatic notifications, and cloud-based backend services provide users with an instant experience while maintaining high data integrity levels. It could be concluded that the most efficient solutions combine the security of the information at the high level with the efficient information management and the immediate visibility. Therefore, the proposed "Automated End-to-End Onboarding System with Real-Time Document Handling" meets the local standards of excellence.

## **2.3 Related Literature**

### **1. Single Page Application (SPA) Frameworks**

SPA research literature indicates that client-side rendering techniques modify one page without loading a new page from a server hence reducing network congestion and increasing interactivity (Mesbah & van Deursen, 2007). In contemporary web development practices, Mikowski and Powell (2013) noted that placing routing and view rendering capabilities in the browser instead of backend servers transforms the web environment into interactive desktop software applications. Moreover, according to software engineering foundations, restricting full page reloads ensures that the application state was maintained during any interaction thereby significantly reducing system latency or making the system faster and optimizing server resource consumption (Pressman, 2019).

Client-side state management literature points out that maintaining persistent runtime memory in a browser engine needs good memory management to avoid issues such as memory leaks (Sommerville, 2016). According to Mesbah and van Deursen's (2007) architecture analysis, decoupling of data extraction from layout generation via asynchronous communication made it possible for applications to exchange lightweight JSON messages rather than heavyweight HTML files. Recent studies indicate that the shift minimizes usage and improves start-up time. It benefits data processing activities (Fielding, 2000).

From scientific studies of user experience and system performance within enterprise-level administration systems, there was evidence that user interface design directly impacts system efficiency and effectiveness (Davenport, 1993). Current research on frontend applications' performance shows that SPA systems counterbalance user perception latency by using techniques such as optimistic UI updates and local caching before validation from the server ends (Hunt, 2014). Therefore, the academic literature constantly refers to the single-page application framework as a key technical standard for efficient web platforms that prioritize user engagement and continuous operations (Mikowski & Powell, 2013).

Comparative studies on CSR vs. SSR demonstrate that CSR approaches increase localized interaction speeds due to moving layout generation responsibility onto user machines (Iskandar et al., 2020). It was stated that although SSR frameworks produce full HTML pages at each initial request, CSR frameworks send HTML shell pages together with modular JavaScript that hydrated within the browser (Mesbah & van Deursen, 2007). Research on empirical benchmarks shows that after the first JavaScript parsing process, subsequent page transitions no longer cause latency issues in visual state updating (Iskandar et al., 2020).

Systematic research on the issue of single-page navigation shows that handling navigational states in the browser memory leads to avoiding redundant DNS lookups and TLS handshakes (Fielding, 2000). The use of browser-built-in History API allows for the manipulation of local uniform resource identifiers (URIs), which happens without additional round trips to the server (Mikowski & Powell, 2013). It should be noted that according to software design theory, the implementation of this type of localized routing allows for preserving execution context in order to keep complex data entered by users intact (Pressman, 2019).

Academic reviews of frontend assets deployment processes reveal that optimization methods like code-splitting and chunk loading play a key role in increasing page hydration speed (Bundschuh et al., 2019). Researchers indicate that non-optimized applications had the potential of experiencing increased First Contentful Paint (FCP) values because of extremely large amounts of JavaScript at the beginning of work (St. Amant & McClain, 2018). In order to solve this problem, modern software development practices include splitting of code into module-based chunks (Sommerville, 2016).

Memory management literature associated with long-lived browser applications warns that poor memory management might lead to drastic performance impact (Sommerville, 2016). Since SPA works permanently within one document space without any typical page deletion procedures, heaps of memory had to be regulated under strict control of the lifecycle of components (Mesbah & van Deursen, 2007). It was a computer science practice to state that removal of event listeners, cancelation of asynchronous requests, and removal of references from the global state during the unmount phase were mandatory to avoid browser performance slowdowns (Pierce, 2002).

Concerning enterprise systems architecture, software engineering methodologies stress the importance of concern segregation by separating client-side UI from server-side database operations

(Pressman, 2019). In order to achieve this separation of concerns, single-page application design allows client-side frontend application components to connect to backend services only via RESTful or GraphQL APIs (Fielding, 2000). According to academic research findings, this segregation allows easier scaling of the platform, since microservices on the backend side could evolve independently of the rendering process on the client side (Sommerville, 2016).

Empirical tests concerning Network Transport Layer throughput reveal the substantial drop of total volume of data transmitted via asynchronous JavaScript transfers (Nurseitov et al., 2009). Using JSON objects exchange in lieu of full HTML document transfers minimizes network bandwidth usage in frequent administrative operations (Tusa & Clayman, 2015). Therefore, JSON-based asynchronous communication was considered to be the optimal solution for keeping performance level high in case of restricted network bandwidth conditions in the context of information systems' literature (Davenport, 1993).

Research in the field of Human-Computer Interaction (HCI) reveals that responsiveness of UI was crucial for maintaining cognitive flow and reducing user tasks' duration (Shneiderman & Plaisant, 2010). The studies show that delays more than 100 milliseconds destroy the illusion of direct manipulation by users and introduce friction in the administrative data entry pipeline (Casteleyn et al., 2014). Single-page application design pattern was in accordance with basic HCI principles and provides a friction-free environment for administrative users (Hunt, 2014).

Technical assessments of client-side caching had shown that the storage of operational data structures in local storage engines of clients significantly enhances fault tolerance of the systems



(Tanenbaum & Van Steen, 2016). Studies had shown that local caching of static resources and user session information decreases the load on the back-end servers during peak hours (Buyya et al., 2018). In addition, it was noted in academic literature that local persistent state management allows SPAs to support offline functionality and enables users to compose documents or operate on their caches during brief periods of disconnected mode (Mikowski & Powell, 2013).

Security research papers regarding client-side rendered web applications had highlighted the fact that moving of business logic to user browsers changes threat models of such platforms (Saltzer & Schroeder, 1975). As client-side code and state could be fully inspected by users, researchers emphasize that SPA platforms should consider client environments to be untrusted environments (Pressman, 2019). Modern cybersecurity literature requires that all the authorization policies, input validation techniques, and state validations should be enforced at the level of the database or API gateway rather than the front-end views (Saltzer & Schroeder, 1975).

Research on modern frontend application compilation reveals that Ahead-of-Time (AOT) compilation and tree-shaking were necessary components in order to decrease JavaScript execution costs (Aho et al., 2006). In computer science literature, it was explained that tree-shaking algorithms use static code analysis in order to remove dead, non-executed code from the final build bundle before deployment (Sommerville, 2016). Evidence supports the claim that both AOT compilation and tree-shaking automatically made applications faster by decreasing CPU work load on code parsing, thereby increasing initialization speed of applications (Bierman et al., 2014).

Lastly, academic research of digital operational transformations reveals that using modern SPA architecture enables enterprise platforms to have great adaptability and durability (Davenport & Ronanki, 2018). In software maintenance literature, it was revealed that SPAs' modular component structure made future refactoring

cheaper because developers were able to modify isolated interface components without affecting the whole system negatively (Sommerville, 2016). Therefore, there was clear evidence that information systems literature highly recommends single-page client-side architectures for developing complex administrative workflows (Mesbah & van Deursen, 2007).

Research into the Web Worker architecture for background processing underscores the importance of offloading processing operations from the browser's main UI thread (Hickson, 2012). In single page applications, complex computations in JavaScript, such as document parsing, cryptographic signature verification, or client-side image compression, could lock the single-threaded event loop, causing user interface lags and delays (Mikowski & Powell, 2013). Studies on web performance show that offloading background tasks to worker threads allows for preserving the computing power of the UI thread to maintain smooth frames per second and responsive user interface (Gudla et al., 2016).

Studies that investigate how modern PWAs could be implemented in single-page architectures point out the importance of using Service Worker scripts for network resilience and offline support (Biørn-Hansen et al., 2017). In software engineering studies, it was clarified that the Service Workers act as proxy clients who could intercept HTTP requests, deliver pre-cached content of the application, and postpone network failures to local queues (Tanenbaum & Van Steen, 2016). Research shows that Service Worker caching approaches along with dynamic single-page hydration could provide operational continuity for applications despite severe network failures (Biørn-Hansen et al., 2017).

Thirdly, literature analyzing micro-frontend architecture patterns exemplifies how current web-based enterprise systems enable

single-page application scalability by breaking down monolithic front-end codebases into modules that represent independent and deployable feature domains (Rappl & Schöttner, 2021). Literature in the field of software engineering reveals that with the increasing size of single-page applications, using a single monolithic front-end codebase leads to serious build time delays and developer inter-dependency (Sommerville, 2016). Through the use of the concepts of microservices to the client-side runtime environment through dynamic module federation, businesses could create independent units of business domains and enhance maintainability metrics and engineering speed (Simões et al., 2023).

## **2. End-to-End Business Process Integration**

Business process integration literature explains that end-to-end framework paradigms seek to eliminate structural friction, manual document handoffs, and data fragmentation across complex enterprise workflows (Davenport, 1993). In modern organizational engineering, Hammer and Champy (2001) stressed that radical Business Process Reengineering (BPR) requires replacing isolated, departmental silos with unified digital pipelines to achieve dramatic leaps in operational speed and processing quality. Furthermore, foundational information systems research emphasizes that automating end-to-end administrative pipelines significantly reduces human data-handling interaction, mitigating operational error rates and lowering aggregate transaction costs (Pressman, 2019).

Analysis of research on the topic of workflow automation suggests that the theories of loose coupling and component decoupling were crucial for building resilient enterprise architecture (Tanenbaum & Van Steen, 2016). In their software engineering frameworks, Sommerville (2016) proved that isolation of workflow processing stages was vital to avoid propagation of isolated failures in

interconnected systems. Modern studies prove that end-to-end decoupled pipelines provide a consistent throughput and predictable flow of data which was a secure basis for processing of sensitive administrative documents (Davenport & Ronanki, 2018).

Literature reviews of the effectiveness of organizations working in document-driven environments show that manual data input and several stages of approvals create major bottlenecks and delays (Hammer & Champy, 2001). The recent research supports an idea that switching from paper-based or partially electronic processes to fully automated and integrated digital workflows reduces the time required to complete each process (Davenport, 1993). Hence, scientific literature considers automation of end-to-end process integration as an important technical solution to get rid of administrative backlogs and to ensure continuity of workflow (Pressman, 2019).

Systemic studies of ERP and document orchestration models demonstrate that centralized data validation gates guarantee structural integrity before the database ingestion (Silberschatz et al., 2020). The scholars note that the classical workflow structure frequently features asynchronous data mismatches caused by manual validation and handover of documents between different departments (Davenport, 1993). Modern process architectures leverage schema-matching mechanisms embedded directly into the integration pipeline to ensure that the administrative engines operate only with clean data (Sommerville, 2016).

Human-System Interaction theories explored in academic studies of automated administrative systems highlight that fewer points of manual interference lead to lower cognitive load and errors (Shneiderman & Plaisant, 2010). The scholarly research reveals that when employees were released from monotonous data transcriptions, their work was redirected toward high-level management and

decision-making (Davenport & Ronanki, 2018). In information systems studies, intuitive and state-aware process pipelines prove to be productive for workers and guarantee regulatory compliance of document flows (Hammer & Champy, 2001).

Literature evaluating Business Process Model and Notation (BPMN) standards notes that formal workflow abstractions enable software engineers to map, execute, and monitor complex administrative sequences systematically (Mendling et al., 2017). By translating abstract operational policies into executable software workflows, system designers could explicitly enforce procedural rules, access control boundaries, and automated state transitions (Pressman, 2019). Empirical software engineering studies demonstrate that formal process modeling reduces system ambiguity and accelerates software deployment in heavily regulated domain environments (Sommerville, 2016).

Research focusing on real-time process monitoring highlights that event-driven integration frameworks allow supervisory figures to maintain complete, transparent oversight over pending application pipelines (Tanenbaum & Van Steen, 2016). By logging granular timestamps and state updates at every processing node, modern workflow engines establish immutable audit trails for administrative review (Saltzer & Schroeder, 1975). Computer science scholars note that comprehensive operational observability deters unauthorized document tampering while simplifying regulatory compliance verification across administrative divisions (Pressman, 2019).

In the context of enterprise architecture design, systems integration research indicates that using standardized Web APIs could help different legacy systems communicate with contemporary processing environments easily (Fielding, 2000). When legacy databases or third-party software applications were encapsulated within API

boundaries, software engineers create integrated end-to-end workflow systems without making major changes to the infrastructure (Sommerville, 2016). Research shows that process orchestration based on API helps organizations to become more flexible as they could expand systems quickly as operational needs change (Davenport, 1993).

The empirical testing of network overhead for distributed workflows shows that event-driven message protocols reduce idle time in servers compared to the traditional synchronous polling technique (Tanenbaum & Van Steen, 2016). Since modern integration systems publish event triggers only when the application state changes, they save network overhead and minimize server CPU usage (Buyya et al., 2018). Information Systems research proves that event-driven process integration was the best practice to use when designing high-concurrency document processing systems (Davenport & Ronanki, 2018).

Workflow automation literature on information security highlights that the integration boundaries should strictly adhere to the Principle of Least Privilege at all processing stages (Saltzer & Schroeder, 1975). Considering that sensitive administrative documents flow through multiple system components, security experts advocate including authentication of identity within the layers responsible for orchestrating the workflow (Pressman, 2019). Research indicates that implementing granular access control in combination with automated document routing ensures that unauthorized users did not gain access to restricted documents while in transit (Saltzer & Schroeder, 1975).

Studies of fault tolerant workflow engines underscore the significance of automated retry operations and transaction rollbacks for ensuring the integrity of data when processing was disrupted (Silberschatz et al., 2020). In case of a complicated multi-step process, network interruptions or failures of APIs could result in interruption of in-flight document submissions (Tanenbaum & Van

Steen, 2016). The research of software reliability emphasizes that executing processes within an integrated system under ACID (Atomicity, Consistency, Isolation, Durability) constraints enables failed transactions to be rolled back without damaging the database (Codd, 1970).

The technical literature that discusses cloud native process execution highlights the fact that containerization and serverless function processes help enterprise integration layers dynamically scale up during peak load situations (Armbrust et al., 2010). Instead of deploying a static infrastructure with underused servers, cloud integration technologies deploy processing micro-runtimes on demand to cope with varying volumes of documents being submitted (Buyya et al., 2018). The literature on computer systems shows that process execution in a serverless architecture reduces infrastructure overheads while ensuring predictable performance SLAs for critical administrative tasks (Satyanarayanan, 2017).

The comparative analysis of organizational transformation reveals that the companies that use process integration in an automated end-to-end fashion had higher SLA compliance compared to those that apply a combination of manual and digital procedures (Hammer & Champy, 2001). Process integration helps reduce the overall time needed for document lifecycles through format standardization, validation check automation, and elimination of manual routing (Davenport, 1993). Therefore, the management and information systems literature considers process reengineering the key strategic instrument of improving efficiency (Davenport & Ronanki, 2018).

Academic studies of data governance through automated systems highlight that continuous data validation avoids the spreading of corrupt or malicious payloads through enterprise systems (Pressman, 2019). The use of parsing filters during the intake of data isolates

invalid files or missing data fields prior to initiating downstream processing activities (Sommerville, 2016). The field of cybersecurity and databases highlights that front-end and pipeline validation protects enterprise data repositories against injection attacks and schema drifts (Silberschatz et al., 2020).

Ultimately, academic assessments of digital administrative transformation indicate that process integration was the fundamental backbone needed in modern transparent governance systems (Davenport & Ronanki, 2018). Scholars within software engineering state that a modular and documented integration framework facilitates technological integration in the future without disturbing existing core processes (Sommerville, 2016). Therefore, information systems literature highly recommends end-to-end process integration as a critical foundation for high-throughput administrative systems (Davenport, 1993).

### **3. DevOps and Agile Methodologies**

According to agile and DevOps software engineering literature, combining iterative development methodologies with automated operational delivery pipelines increases software release velocity, software reliability, and the entire software product quality (Fitzgerald & Stol, 2017). Modern software engineering literature emphasizes that closing the gap between historically separated software development and operations ensures continuous feedback loops needed to deliver fast and high-quality software iterations (Kim et al., 2016). In addition, methodology research proves that abandoning inflexible linear waterfall development methods in favor of continuous Agile development models reduces software project risks and guarantees that software functionality corresponds to changing customer needs (Pressman, 2019).

Analysis of the 'Continuous \*' (Continuous Star) methodology framework confirms that unifying Continuous Planning, Continuous Integration (CI), and Continuous Delivery (CD) results in an



adaptive software engineering environment (Fitzgerald & Stol, 2017). In their analysis of modern software deployment, Humble and Farley (2010) showed that treating deployment as automated quality control gates helps to eliminate risky manual builds. Modern empirical studies confirm that continuous integration delivery pipeline accelerates feature delivery and guarantees high reliability of production software builds (Kim et al., 2016).

Modern empirical studies that examine mechanics of Agile Scrum and sprint management frameworks prove that time-boxed iterative development increases team adaptability and provides higher project transparency (Schwaber & Sutherland, 2020). With structured development processes in the form of time-boxed iterative sprints, software development teams could re-prioritize task backlog dynamically in accordance with new feedback and technical constraints (Pressman, 2019). Studies in empirical software engineering show that time-boxed iterative sprint planning prevents scope creep and improves software project scheduling predictability (Sommerville, 2016).

In systematic investigations of automated CI infrastructure, scholars identify that early defect detection in the development process (shift left quality assurance) helps to minimize development costs (Fitzgerald & Stol, 2017). Researchers observe that fixing syntax errors, dependency problems, or broken logic in the process of local feature development was much cheaper than solving these issues in the production environment (Pressman, 2019). Triggers for automatic static analysis linters, type checkers, and unit tests on each code commit guarantee high codebase quality standards in modern CI pipelines (Humble & Farley, 2010).

Research in the field of collaborative repository management identifies that using effective branching strategies such as GitFlow

or trunk-based development guarantees high codebase stability in case of multiple parallel changes made by several developers (Chacon & Straub, 2014). Scholarly studies identify that applying strict branch isolation rules helps to prevent any unverified code in primary production branches (Sommerville, 2016). Literature in information systems field proves that branch protection rules combined with mandatory peer code review help to enforce organizational quality control standards (Kim et al., 2016).

However, scientific literature examining CD processes indicates that automated deployment procedures significantly diminish operational down-time associated with releasing new software versions (Humble & Farley, 2010). Using automated deployment procedures, such as blue-green or canary deployment, engineering teams were able to release software updates in production safely (Kim et al., 2016). Software engineering studies demonstrate that automated deployment procedures eliminate operational errors performed by engineers, thus allowing for performing numerous low-risk software releases (Fitzgerald & Stol, 2017).

Research focusing on automated testing frameworks highlights that comprehensive test-driven development (TDD) strategies ensure long-term system stability and prevent regression bugs during rapid code iteration (Beck, 2003). By writing automated unit, integration, and end-to-end tests before or alongside feature implementation, developers establish a continuous safety net for future refactoring efforts (Pressman, 2019). Computer science scholars note that high automated test coverage builds developer confidence, enabling teams to modify complex system components without inadvertently breaking existing functionality (Sommerville, 2016).

Software project management literature suggests that visual

control systems based on Kanban optimize operational throughput and help with managing WIP limitations within engineering teams (Anderson, 2010). Using Kanban boards, engineering teams visualize the progression of their tasks throughout the pipeline stages - from ingesting them from the product backlog to deploying them in production (Pressman, 2019). Scientific literature shows that combining Kanban visual management with Scrum sprint mechanics helps to increase process visibility and streamline team throughput (Schwaber & Sutherland, 2020).

Evaluations of team velocity and deployment parameters of high-performing DevOps organizations indicate that such organizations achieve much higher release rates and much lower MTTR after production incidents (Forsgren et al., 2018). With the use of automated telemetry, automated monitoring and centralized logging, engineering teams were able to quickly detect and troubleshoot the problems in the production (Kim et al., 2016). Research dedicated to software development highlights feedback loops based on telemetry as an essential requirement for achieving high system availability (Fitzgerald & Stol, 2017).

Scientific literature dedicated to information security, while examining the problem of integrating DevOps practices, which was usually referred to as DevSecOps, highlights that security controls should be built right into the CI/CD pipelines and cannot be considered as a separate step performed manually (Kim et al., 2016). The fact was that continuous deployment at fast speed made impossible the implementation of manual security audit, and security researchers suggest that dependency vulnerability scanners and SAST tools should be integrated with the initial stage of builds (Pressman, 2019). It was proven that automated security checks allow for identifying security issues prior to release of code into production (Saltzer & Schroeder, 1975).

Investigations into Infrastructure as Code (IaC) paradigms emphasize that managing server environments through version-controlled configuration files ensures operational consistency across development, staging, and production tiers (Humble & Farley, 2010). When working in complex cloud-native systems, the manual procedure of setting up infrastructure was a source of configuration drifts and inconsistencies (Armbrust et al., 2010). Systems engineering literature demonstrates that the IaC tooling enables engineering teams to programmatically provision, update, and destroy server environments (Kim et al., 2016).

Literature on technical evaluations of DevEx and developer productivity points out that automation of development processes and streamlining of the engineering pipeline improves efficiency and code quality (Forsgren et al., 2018). Eliminating manual build processes, unnecessary local setup procedures, and switching from one task to another lets developers focus on problem-solving activities (Shneiderman & Plaisant, 2010). Engineering research demonstrates that investments in efficient DevOps toolchains pay off through increased productivity and quality of software products (Fitzgerald & Stol, 2017).

Comparative analysis of software delivery performance shows that organizations with unified Agile DevOps frameworks were superior to companies with old-fashioned approaches to software development on reliability indicators (Forsgren et al., 2018). Reducing time spent on building the software product, automating code validation processes, and fostering collaboration between developers and operators allows such enterprises to quickly adapt to changing operational needs (Kim et al., 2016). As a result, software engineering literature argues that unified Agile DevOps frameworks should be considered as the standard for modern enterprise software engineering (Fitzgerald & Stol, 2017).

Research focused on software maintenance and technical debt management shows that continuous refactoring through automated CI/CD pipelines avoids decay of codebase in the long run (Sommerville, 2016). Addressing technical debt as a backlog item during the course of every iteration ensures that code stays readable and modular (Pressman, 2019). Computer science research notes that proactive technical debt management ensures software platforms remain adaptable to future functional requirements (Fitzgerald & Stol, 2017).

Finally, academic evaluation of modern software engineering methods shows that unified Agile DevOps frameworks were the basis for creating robust, flexible and scalable software products (Kim et al., 2016). Software engineering scholars note that blending human-oriented approach to planning of the project and automated pipeline allows achieving flexibility and stability at the same time (Sommerville, 2016). Consequently, information systems literature strongly recommends unified Agile DevOps frameworks as an important element of modern high-throughput software development (Fitzgerald & Stol, 2017).

#### 4. React.js Component Architecture

Component-Based Software Engineering (CBSE) literature states that modern UI design was based on creation of encapsulated software components containing visual elements and logic (Sommerville, 2016). In client-side web application development, Hunt (2014) noted that declarative rendering was used instead of error-prone DOM manipulations. In addition, fundamental research in software engineering points out that modular component architecture minimizes duplications, increases maintainability metrics and simplifies development of web applications (Pressman, 2019).

A study analyzing client-side rendering engines emphasizes React's

Virtual DOM as a vital design pattern facilitating efficient updating of document tree structure (Hunt, 2014). In the case of conventional dynamic web applications, direct manipulation of the browser's DOM tree elements causes expensive recalculation of the style, layout re-flow and repaints in the browser engine (Mikowski & Powell, 2013). Empirical research of current web performance problems confirms that maintaining an in-memory copy of the real DOM tree allows React to make batch DOM changes effectively, thus minimizing the costs of client-side rendering (Iskandar et al., 2020).

Mathematical evaluations of React's reconciliation engine demonstrate that heuristic  $O(n)$  tree-diffing algorithms made real-time component re-rendering computationally feasible (Hunt, 2014). Whereas standard tree comparison approaches work in  $O(n^3)$  time complexity, React made two realistic assumptions - element type match and keys provided by developers - to determine the minimum mutation path of DOM elements in linear time (Sommerville, 2016). Research in computer science supports the view that such heuristics greatly minimize the costs of CPU execution of the JavaScript code when interface state was being changed (Bierman et al., 2014).

Studies of state management approaches in React emphasize the fact that unidirectional data flow approach ensures predictable execution order of components (Hunt, 2014). Confining the transfer of data to one direction and passing immutable properties (props) downwards from parent components to children allows developers to avoid mutations of states caused by the two-way data binding paradigm (Mikowski & Powell, 2013). According to software design experts, unidirectional pipelines of data simplify debugging and tracking of the state in complex enterprise interface (Pressman, 2019).

The literature on declarative UI programming indicates that abstracting from lower-level DOM manipulations allows engineers to concentrate on the application logic and state description (Hunt,

2014). In place of adding/removing DOM elements manually, developers describe the desired interface state, delegating DOM manipulations to component reconciliation engines (Sommerville, 2016). Information systems specialists emphasize declarative programming as one of the main factors contributing to software bug density reduction and fast engineering (Pressman, 2019).

The literature on the architecture of Hooks-based React framework notes that functional components provide more elegant and testable alternative to class-based inheritance structures (Hunt, 2014). Functional components allow stateful logic to be encapsulated in reusable modular functions avoiding 'this' binding and class boilerplate code (Mikowski & Powell, 2013). Software engineering research supports the view that using hooks helps to maintain modular design and allows developers to share business logic among different UI modules without coupling them (Sommerville, 2016). Concurrent rendering as a key characteristic of modern React engines was discussed in the research dedicated to the characteristics of concurrent execution in React (Hunt, 2014). Interruption of the rendering of the low-priority off-screen content by immediate processing of user inputs was how React made sure that the interface did not freeze in case of heavy hydration of components (Gudla et al., 2016). As per the HCI literature, prioritizing input responsiveness ensures preservation of the user cognitive flow, thus creating a seamless digital experience irrespective of the high client CPU load (Shneiderman & Plaisant, 2010).

Component composition libraries as a part of enterprise application design provide the frontend developers with tools to create design systems (Curtis, 2016). Encapsulation of the design tokens, accessibility features and interaction patterns inside the core component libraries guarantees consistency of the operational

interfaces in software solutions that consist of several modules (Marcotte, 2011). Software engineering research indicates that the modular approach to building UI component libraries simplifies the long-term maintenance of the design solution and speeds up the onboarding process for new software engineers (Pressman, 2019).

Memory analysis of client-side overhead in React environments reveals efficient event delegation as a means of preserving the browser memory resources (Mesbah & van Deursen, 2007). Instead of creating individual event listeners for all dynamic DOM nodes, React attaches one single event listener to the root-level container (Hunt, 2014). Computer science literature proves that the Synthetic Events system created by React requires less memory heap allocation and lowers the risk of memory leaks caused by uncollected event listeners (Sommerville, 2016).

Information security research into client-side rendering engines reveals that automatic Cross-Site Scripting (XSS) sanitization was one of the fundamental characteristics of React (Saltzer & Schroeder, 1975). Since React automatically escapes all string variables, the common attack vectors of reflected and stored XSS attacks were protected by default (Pressman, 2019). Security scholars note, however, that developers had to remain vigilant when explicitly bypassing default escaping mechanisms, ensuring that untrusted user inputs were sanitized before rendering (Saltzer & Schroeder, 1975).

Analysis of SDUI and RSC focuses on the importance of executing certain components at the edge environment in order to reduce the size of client-side JavaScript bundles (Iskandar et al., 2020). With generation of the static component markup on edge servers and interactive components locally, React allows to combine the advantages of SPAs with fast initial loading of traditional server-rendered applications (Bundschuh et al., 2019). Systems



literature considers the hybrid execution of components on the server and client sides as an ideal paradigm for high-performance web platforms (Satyanarayanan, 2017).

However, technical literature analyzing state hydration and memoization strategies like `useMemo` and `useCallback` indicates that controlling re-rendering frequency plays an important role in optimizing performance in big-data web applications (Hunt, 2014). Through caching calculations results and maintaining object references throughout the render process, memoization helps prevent unnecessary re-renders of the child component trees (Sommerville, 2016). Empirical performance studies of web interfaces prove that using memoization effectively reduces CPU time spent on processing frames during real-time data streaming (Bierman et al., 2014).

Comparative studies assessing the quality of modern frontend UI frameworks and libraries point out that React's component model ranks quite highly due to its vast developer community and versatility in architectural integration (Iskandar et al., 2020). Unopinionated view layer enables engineering teams to incorporate custom state management, routing and data fetching solutions to meet specific business requirements (Pressman, 2019). Therefore, software engineering literature views React as an ideal basis for building enterprise-grade scalable web applications (Sommerville, 2016).

Studies exploring long-term software maintainability confirm that component-based React applications exhibit lower maintenance defect rates compared to legacy monolithic scripts (Pressman, 2019). Encapsulating CSS, HTML structures, and state management logic into single-file components simplifies code refactoring and isolated unit testing routines (Beck, 2003). Software engineering scholars note that isolated component

structures minimize unintended side effects during system upgrades, ensuring software stability over extended operational lifecycles (Sommerville, 2016).

Finally, modern academic studies of frontend engineering frameworks reveal that React's component architecture was an appropriate approach to developing mission-critical web applications (Hunt, 2014). Software engineering experts argue that declarative user interface and virtual reconciliation engine enable achieving balance between performance of UI rendering and productivity of web developers (Pressman, 2019). Thus, information systems literature strongly supports the use of the React engine as the base of modern data-intense web interfaces (Sommerville, 2016).

## 5. Node.js & Express Server Runtime Environment

However, server-side execution paradigms show that event-driven, non-blocking I/O runtime was required to ensure the concurrency needed for data-intensive web services. In his paper, Dahl (2009) showed that switching from multi-threaded server architectures to single-threaded event loops enables getting rid of expensive thread context switching. Backend engineering studies show that using the capabilities of the Google V8 engine through the Node.js environment allows backend services to efficiently perform thousands of concurrent I/O operations without wasting additional system memory resources.

However, the research of asynchronous backend architectures proves that the Reactor pattern was an essential principle behind efficient API gateway design (Tilkov & Vinoski, 2010). Through offloading all the blocking I/O operations such as disk writing and database queries to the thread pool via Libuv library, the main event loop thread becomes constantly available to receive HTTP client requests. Server performance studies show that non-blocking event loops keep the stable throughput even under

heavy load without causing crash-loops in application servers.

In web application framework engineering, Express.js provides an unopinionated middleware execution pipeline that simplifies HTTP request routing and payload transformations. In line with what Holowaychuk (2010) posited, middleware pipelines function as a series of processing gates wherein incoming requests move through authentication, input validation, and body parsing layers prior to invoking core business controllers. Software engineering literature attests to the fact that modular middleware design patterns segregate cross-cutting concerns in order to provide significant improvement in terms of legibility and unit testability of backend code.

Studies analyzing RESTful API design guidelines had stressed that the use of resource-based URLs along with uniform HTTP methods ensures loose coupling between backend endpoints and client runtimes (Fielding, 2000). According to Newman (2015), RESTful HTTP interfaces allow the independent evolution of the backend execution logic in service-oriented and microservice architectures without breaking the client integration layers. Literature on systems engineering confirms that REST-compliant Node.js backends facilitate third-party service integration and made long-term API versioning easy.

Scholarly studies related to layered backend architecture show that the segregation of route controllers, business logic services, and data access layers (DAL) ensures the absence of code degradation during system scaling (Fowler, 2002). Strict adherence to dependency inversion between backend domain layers ensures the independence of HTTP routing interfaces from the database query engines (Pressman, 2019). As per the software design literature, multi-tiered and clean backend architecture provides isolation of

unit testing as well as replacement of database engine without affecting business rules.

Literature analyzing server-side error handling methods emphasizes the severe risks of unhandled asynchronous promise rejections in relation to operational stability in single-threaded runtimes (Tilkov & Vinoski, 2010). According to the software reliability literature, the failure of a single thread in an event loop might lead to the abrupt termination of a whole server application and disrupt the service of all the users. Consequently, systems developers advocate using centralized error-handling middleware, structured promise chains (async/await), and global process watchers to catch exceptions gracefully and maintain platform uptime.

Studies concentrating on API rate limiting and dynamic request throttling mechanisms identify such approaches as key defensive measures against Denial-of-Service (DoS) attack vectors and computational exhaustion (Holmes, 2012). The use of rate limiting middleware based on token-bucket or sliding-window approaches limits the burst of incoming requests from individual client IP addresses (Saltzer & Schroeder, 1975). Information security literature validates that request throttling prevents backend computational resources from exhaustion and protects the database connections.

In contemporary web development practices, language convergence through the whole technology stack by using JavaScript or TypeScript across frontend elements and server controllers lowers context switching overheads (Dahl, 2009). Design system analysis indicates that unified language stacks enable engineering teams to reuse data models, validation schemes, and utility helper libraries between client and server parts of the architecture. Management software literature proves that unified stack systems increase engineering productivity, onboard developers faster, and decrease

human errors when mapping API payloads.

Empirical analysis evaluating database connection pool mechanisms in Node.js applications shows that proper configuration of dynamic connection limits allows avoiding stallings of backend worker threads (Silberschatz et al., 2020). The reason behind that was that asynchronous I/O operations create new database queries too fast, resulting in the overflow of socket connection limits on the database side. Computer science literature demonstrates that proper configuration of connection pooling strategy helps save database memory resources and ensures consistent response latency under load peaks.

Information security literature investigating server-side runtime environments notes that securing the HTTP header configuration provides an indispensable base to defend against client-side attack vectors (Saltzer & Schroeder, 1975). Integration of security headers by means of libraries like Helmet.js hides raw stack disclosures, enforces strict HTTP Strict Transport Security (HSTS) policy, and protects applications against clickjacking attacks (Pressman, 2019). Security researchers stress that hardening HTTP request-response boundaries at the Express middleware layer was a fundamental prerequisite for enterprise platform compliance.

Research into distributed log aggregation and telemetry instrumentation emphasizes structured logging as one of the critical requirements to debug serverless and microservices-based backends (Newman, 2015). Formatting application logs into machine-readable JSON structures via logging frameworks such as Winston or Morgan allows streaming log entries to central monitoring systems. Systems research demonstrates that backend telemetry in structured format speeds up Root Cause Analysis (RCA) and decreases Mean Time to Repair (MTTR).

Analysis of server-side caching patterns proves that positioning of in-memory storage engines (like Redis) before Express controller layers reduces API response times significantly (Fowler, 2002). In-memory storage of frequently requested data structures saves costly database round trips and serves responses from memory within sub-milliseconds. Web performance analysis proves that backend caching reduces database CPU load and increases capacity of backend services.

Comparative research works studying web server frameworks conclude that conventional multi-threaded web servers (such as Apache HTTP) were very efficient in conducting heavy CPU-bounded operations while asynchronous runtimes such as Node.js were faster than them in terms of I/O-based web socket and API operations (Tilkov & Vinoski, 2010). The single-threaded, event-driven model of Node.js was capable of allocating fewer threads and therefore provides better vertical scalability under concurrent connections. Hence, software architecture scholars recommend using Node.js as a runtime for real-time document routing and API integration layer.

The research of the problem of API documentation standards (e.g. OpenAPI / Swagger) reveals that auto-generation of live documentation from Express router schemas ensures accurate integration between backend and frontend developers (Newman, 2015). Keeping consistent API contracts eliminates any communication barriers and prevents payload type mismatch errors in the cross-team development process. Computer science scholars note that rigorous API contract enforcement improves overall software reliability and accelerates client-side application integration.

Finally, scholarly researches of server-side software engineering indicate that the combination of Node.js with Express represents a flexible and highly-efficient platform for current web architecture.

Software engineering scholars point to the fact that combining non-blocking asynchronous I/O with middleware pipeline execution could effectively achieve the balance between computational efficiency and efficient coding practices. Therefore, information systems researchers provide strong arguments about the use of the Node.js and Express as a key architectural component for modern enterprise backends.

## 6. TypeScript Static Type Systems

Type system research in computer science states that static type-checking at compile time eliminates many classes of defects that occur during program execution in dynamically typed environments (Pierce, 2002). Modern software engineering scholars showed that adding TypeScript structural type system to JavaScript allows obtaining advantages of compile-time verification while maintaining dynamic operational capability (Bierman et al., 2014). Software engineering evaluations confirm that explicit type annotations reduce number of defects in codebase and made maintenance process easier (Pressman, 2019).

Mathematical analysis of static type systems shows that the subtyping approach of the structural type systems allows introducing gradual typing to existing dynamic JavaScript repositories (Bierman et al., 2014). Applying compile-time type inference, generics, and union types, TypeScript was capable of verifying payload structures at compile time without performing type-casts (Pierce, 2002). Computer science researchers proved that compile-time structural type verification reduces number of `TypeError` exceptions during complex state transitions (Sommerville, 2016).

Systematic investigations into the TypeScript Compiler (tsc) architecture reveal that abstract syntax tree (AST)

transformations enforce strict structural constraints before code emits as ECMAScript targets (Bierman et al., 2014). The compiler had several phases including scanner, parser, binder, checker, and emitter to parse source code into immutable ASTs and to perform type-safety checking against symbol tables (Pierce, 2002). The systems literature was clear about how separate compilation phases lead to efficient incremental build capabilities, where code modules could be re-evaluated quickly to save build CPU time (Aho et al., 2006).

The academic research on IDEs shows that statically defined types help the development productivity by providing code-completion, symbol navigation and refactoring support (Bierman et al., 2014). In-time feedback about mismatches in property values, missing required parameters and invalid component signatures helps in avoiding human errors before the code reaches the source control (Pressman, 2019). Software engineering literature points out that the developer-oriented feedback loops improve the engineering velocity and help in reducing the code review effort (Forsgren et al., 2018).

In distributed enterprise application architecture, the combination of TypeScript interfaces with the backend database schemas creates type-safe contract between client and server-side (Bierman et al., 2014). The use of shared interface definitions helps in ensuring that any change in backend data structures leads to build errors in the client components before the deployment process (Newman, 2015). Systems engineering literature shows that type checking of interfaces at boundary levels removes bugs and reduces the integration maintenance costs over long software lifecycles (Pressman, 2019).

In the literature related to maintainability of enterprise applications' UI, sound typing patterns help in designing reusable



abstracted components (Pierce, 2002). Using TypeScript generics, developers parameterize component properties and state models to create reusable UI abstractions while preserving internal type checking (Hunt, 2014). Maintainability evaluations of software reveal that generic type abstractions avoid unsafe type casting and help in reducing the duplicate boilerplate coding for large engineering teams (Sommerville, 2016).

The literature discussing type erasure at compile time reveals that TypeScript compiler removes all type annotations completely from the emitted code and leaves behind the code that runs zero overhead at runtime (Bierman et al., 2014). As the emitted JavaScript had no additional runtime overhead, it was executed directly by the JavaScript engine in the browser (Dahl, 2009). Computer science evaluations indicate that type erasure found the right balance between static program analysis and dynamic execution.

Information security publications studying software compile stages state that strict type compilation eliminates untrusted script injection problems while developing code (Saltzer & Schroeder, 1975). Limiting dynamic code evaluation by using strict primitive types or string literal types in input properties eliminates common script injection attack channels during compile stage (Pressman, 2019). Modern software engineers point out that compile-time type definitions play a crucial role as a secondary security defense mechanism.

Team delivery metrics measured in empirical evaluations confirm that static type checking significantly decreases the number of bug reports after release in production environment (Bierman et al., 2014). Though static typing increases initial type definition costs when first setting up the system, maintenance costs and production debugging time become much smaller later (Sommerville, 2016). Therefore, software engineering publications identify TypeScript as one of the key technologies for enterprise software systems.

Experiments with build orchestration tools reveal that combining TypeScript's project references with incremental compilation flags increases the speed of build process in monorepo structure (Bierman et al., 2014). Incremental builds cache unchanged compilation results on disk to avoid unnecessary module checks when developers actively work with code (Aho et al., 2006). Computer scientists prove that incremental compilation was scalable for repository codebase performance as the number of project files grow into thousands of modules.

Technical literature evaluating control flow analysis in modern compilers highlights TypeScript's type-narrowing capabilities as a key factor in writing safe asynchronous code (Bierman et al., 2014). By tracking variable types through execution paths, type guards, and nullability checks, the compiler forces developers to handle edge cases—such as undefined or null API responses—explicitly (Pierce, 2002). Software reliability studies prove that explicit nullability checking prevents unexpected runtime crashes in complex client-side applications (Sommerville, 2016).

Comparative analyses of typed and untyped JavaScript projects prove that static type systems increase code readability and decrease technical debt accumulation (Bierman et al., 2014). Type interfaces become a form of self-documenting information that stays consistent with actual implementation logic (Pressman, 2019). Software engineering studies found that type contracts help in reducing maintenance costs and facilitating onboarding processes for new developers.

Research on continuous integration pipelines shows that automated TypeScript compilation with `tsc --noEmit` option was a must-have quality gate (Fitzgerald & Stol, 2017). Pull request builds automatically fail when there were type errors, and it was impossible

for type regressions to appear in staging or production branches (Humble & Farley, 2010). Software engineering studies found that type compilation gates improve deployment safety and preserve codebase consistency.

The academic analysis of the API client code generation shows that auto-generated TypeScript interface files from database schemas provide end-to-end typing between different software layers (Newman, 2015). Manual elimination of type declarations excludes transcription errors of backend schema changes (Silberschatz et al., 2020). Computer scientists state that auto-generation of types ensures compatibility of clients with database entities.

Finally, academic reviews of modern compiler architectures found that TypeScript had a structure necessary for mission-critical web applications (Bierman et al., 2014). The software engineering literature highlights that combining type checking at the compilation stage and dynamic execution enhances flexibility while maintaining stability (Pressman, 2019). Thus, information systems literature strongly endorses the TypeScript compiler as a core technical pillar for scalable enterprise software.

## 7. Tailwind Utility-First CSS Framework

Research of client-side UI style architectures states that the utility-first approach was useful for limiting CSS accumulation and preventing class name conflicts (Marcotte, 2011). In classical cascading stylesheets architecture, global CSS rules lead to cascade effects and unnecessary CSS due to their unpredictable behavior in the course of software lifetime (Wathan, 2017). Performance experiments prove that utility-first CSS generators such as Tailwind CSS reduce the size of CSS bundles by reusing utility classes (Gudla et al.,

2016).

Investigations of JIT CSS generation engines state that the dynamic creation of styles conserves the bandwidth of client browsers in the course of page hydration (Wathan, 2017). The scanning of source code for active utility classes and compiling only necessary and purged CSS bundles prevents the distribution of unnecessary CSS classes (Bundschuh et al., 2019). The performance of web pages proved that smaller CSS bundles reduce latency of render process and improve FCP metrics (St. Amant & McClain, 2018).

Academic analysis of component style co-location found that style definitions inside components were more maintainable and localizable (Curtis, 2016). Developers did not need to switch between isolated style files and component code by using utility classes (Wathan, 2017). Software maintainability literature found that style co-location within component bounds reduces technical debt growth (Sommerville, 2016).

Research focusing on cross-platform layout standardization shows that utility-first grid and flexbox abstractions simplify responsive design across heterogeneous device viewports (Marcotte, 2011). Defining layout rules through predictable responsive utility prefixes ensures consistent visual rendering across varying device screen resolutions (Shneiderman & Plaisant, 2010). Human-Computer Interaction (HCI) literature confirms that visual layout responsiveness directly preserves user engagement and reduces cognitive friction during complex administrative workflows.

In the case of software architecture design, setting up centralized design tokens in Tailwind configuration files ensures

visual consistency at the institution level among the component libraries (Curtis, 2016). Limitation of design choices to pre-defined spacing scales, color palettes, and typography rules prevents arbitrary styling and assures brand consistency (Marcotte, 2011). Researches in software engineering proved that centralized design token management simplifies UI maintenance when updating a platform (Pressman, 2019).

Measurement of CSS parser performance in browsers demonstrates that low specificity utility classes decrease the overhead of style recalculating in browser engines (Mesbah & van Deursen, 2007). As utility classes use single-class selectors, browser engines resolved visual properties in linear time and did not had to went through the selector tree of the DOM (Bierman et al., 2014). Research in computer science shows that flat specificity hierarchy of CSS saves CPU cycles on the client side during frequent UI state transitions.

DevEx research measures the context-switching efficiency of utility-first approach during feature implementation and shows positive results (Forsgren et al., 2018). Elimination of the cognitive burden of defining custom CSS class names enables programmers to focus on managing UI state and business logic (Wathan, 2017). The studies in software engineering industry demonstrate that decreased cognitive overhead leads to acceleration of feature release.

The literature in information security regarding visual presentation layer on the client side states that utility-first approach reduces vulnerabilities related to CSS injection (Saltzer & Schroeder, 1975). Limitation of styling options to pre-defined compiler utilities did not allow malicious scripts to modify the DOM styling (Pressman, 2019). Information security researchers point out that utility generation on the compilation stage serves as a barrier for UI assets.

Researches on the topic of component-driven design systems prove that usage of Tailwind CSS with React components creates modular UI component pipeline (Hunt, 2014). Wrapping of the utility-styled elements inside the reusable React components forms isolated UI primitives (Curtis, 2016). Literature in software engineering proves that modular libraries of UI components speed up onboarding and reduce unnecessary UI development.

Analysis of comparative studies between traditional CSS methods and utility-based ones reveals that the latter had much smaller CSS file increases (Wathan, 2017). In contrast to conventional stylesheets, which grow proportionally with each new user interface feature, the growth of utility-first CSS files was rapid until it reached a peak value when the reuse of utility classes became maximal (Bundschuh et al., 2019). According to computer science literature, there was predictability of network loading performance because of bounded CSS file sizes.

Research on the architecture of dark mode and theme switching indicates that the utility-first design made it easy to switch themes using media queries or classes (Marcotte, 2011). It was enough to write down the dark and light themes of any atomic element in order to be able to change them without full page reloads (Shneiderman & Plaisant, 2010). HCI literature proves that dynamic theme changes made platform more accessible.

The analysis of post-processing tools such as CSS purging shows that they remove unused CSS rules automatically (Bundschuh et al., 2019). Comparing the production files with generated utility class definitions, compilers detect unnecessary selectors and produce only essential CSS files (Wathan, 2017). The web engineering literature demonstrates that automated CSS purging decreases mobile network transfer times.

Accessibility (ally) requirements analysis shows that utility-first framework gives out-of-the-box support to accessible focus state and screen reader utilities (Shneiderman & Plaisant, 2010). Writing down accessibility utilities made interactive elements to conform to WCAG requirements without writing specific CSS hacks (Pressman, 2019). Software studies confirm that integrated accessibility utilities help developers maintain inclusive application interfaces.

Continuous integration builds analysis reveals that automated CSS linters and validators guarantee the hygiene of styles before deploying (Fitzgerald & Stol, 2017). Automated detection of misused utility classes guarantees absence of visual artifacts on the deployed website (Humble & Farley, 2010). Software engineering literature proves that automated verification helps to increase delivery reliability.

Finally, academic reviews of modern styling toolchains indicate that Tailwind CSS represents an ideal and efficient platform for enterprise application interfaces (Wathan, 2017). According to software engineering literature, integration of atomic utility styling with JIT build pass compilation combines developer efficiency and client execution speed (Pressman, 2019). Hence, academic research in information systems field strongly recommends utility-first CSS frameworks as one of the crucial pillars of modern web engineering.

## 8. Supabase & Managed PostgreSQL Database Architecture

Object-relational database management literature indicates that object-relational database management systems guarantee enterprise-level data integrity, ACID compliance, and structured persistence of enterprise applications (Codd, 1970; Silberschatz et al., 2020). In modern cloud database architecture design, PostgreSQL represents the extensible database engine able to work with both complex relations and JSON data primitives (Elmasri & Navathe,

2015). According to software engineering evaluations, encapsulation of PostgreSQL databases in managed BaaS services like Supabase simplifies backend infrastructure while preserving enterprise-level relational guarantees (Pressman, 2019).

Scientific reviews and systematic investigations into Row-Level Security (RLS) paradigms prove that moving access control boundaries into database layer guarantees elimination of authorization vulnerabilities on application level (Saltzer & Schroeder, 1975). Implementation of declarative SQL policy rules on database tables allows evaluating native user identity tokens (auth.uid()) by PostgreSQL in each query execution (Silberschatz et al., 2020). Security scholars confirm that database-level authorization prevents unauthorized data access bugs caused by faulty backend API controller logic.

Literature analyses of JSONB data primitives in relational engines prove that storage of semi-structured documents alongside with relational objects ensures the combination of SQL data integrity with NoSQL data model flexibility (Elmasri & Navathe, 2015). JSONB binary format of PostgreSQL supports Generalized Inverted Index (GIN) indexing, which provides sub-millisecond query execution against nested document attributes (Silberschatz et al., 2020). According to computer science research, hybrid relational-document storage was possible with guaranteeing relational foreign-key integrity.

Research on Write-Ahead Logging (WAL) and transaction management proves that ACID compliance provides data integrity in the case of any concurrent transactions in relational database systems (Reuter & Härder, 1983). With writing transactions into immutable WAL stream before applying modifications to storage pages, PostgreSQL guarantees full system recovery in the case of any hardware crash (Silberschatz et al., 2020). Database engineering literature



confirms that WAL enforcement was crucial for auditing document transactions.

In terms of database architecture design, the PostgREST engine reflects PostgreSQL database schemas automatically into instantly available RESTful API endpoints (Fielding, 2000). Creation of secure API endpoints based on database tables removes the need to manually create CRUD controllers and routing functions (Newman, 2015). From the perspective of systems engineering literature, automatic creation of data access layers increases development velocity and reduces controller bugs density.

Literature related to database indexing technique discusses that B-Tree, Hash, and GIN indexes increase performance of queries in large (multi-million row) tables (Elmasri & Navathe, 2015). Indexing foreign key columns and columns often used in filtering operations avoids expensive table scans and saves disk I/O and RAM usage (Silberschatz et al., 2020). Research into database performance demonstrates that intelligent index tuning guarantees query performance under concurrent data workload.

Research focusing on connection pooling utilities (such as PgBouncer) highlights connection management as a critical requirement for serverless cloud environments (Tanenbaum & Van Steen, 2016). Serverless applications generate many short-lived database connections and might face database socket exhaustion issues under high concurrency (Armbrust et al., 2010). Systems literature confirms that using lightweight connection proxy services before connecting to PostgreSQL distributes database connections efficiently and guarantees server stability.

In terms of data management in an enterprise, Foreign Key constraints, Unique indexes, and CHECK triggers guarantee domain integrity at the persistence layer (Codd, 1970). Exclusion of corrupted or incomplete rows from persistence saves data quality in

various application modules (Silberschatz et al., 2020). Database scientists state that schema-based constraints guarantee system consistency during software lifecycle which could last for decades.

Empirical evaluation of database backup and point-in-time recovery (PITR) mechanisms demonstrates that continuous WAL archiving guarantees minimum Recovery Point Objectives (RPO) (Reuter & Härder, 1983). Archiving WAL records into secure cloud storage allows system administrators to recover database state to precise timestamps after data corruption events (Silberschatz et al., 2020). Disaster recovery literature states that PITR was an indispensable feature for enterprise document platforms.

Data security literature examining database encryption notes that using SSL/TLS transport security along with AES-256 storage encryption meets stringent data protection criteria (Saltzer & Schroeder, 1975). Encrypting data at rest ensures the integrity of persistent storage volumes, whereas TLS protects database requests from eavesdropping on the network. Researchers in the field highlight that data encryption was a had to for the sake of administrative compliance.

Explorations into stored procedures and PL/pgSQL functions implemented in PostgreSQL databases indicate that running business logic in the database server minimizes the number of round trips (Elmasri & Navathe, 2015). Running calculations and table update processes in the database function decreases the latency between the client and the server (Fowler, 2002). Computer science literature confirms that database-side execution optimizes throughput for heavy analytical tasks.

The literature assessing tools for managing the schema of the database (e.g., Supabase CLI migrations) points out that managing schema through version control made the database consistent across

different environments (Humble & Farley, 2010). Running declarative SQL migration scripts automatically within CI/CD pipelines avoids the need for database management in terms of manually created schemas (Fitzgerald & Stol, 2017). The research on maintainable software states that versioned database migrations simplify platform maintenance.

Comparative studies of managed BaaS solutions and custom hosted database servers reveal that managed PostgreSQL databases help decrease operational overhead considerably (Armbrust et al., 2010). Managed solutions automate OS patching, backups, failovers, and scaling of storage capacity. Therefore, the literature on software architecture highlights that managed PostgreSQL database platforms were a convenient persistence layer for web engineering.

Research on database telemetry and query optimization shows that monitoring the results of EXPLAIN ANALYZE query planning reveals computational bottlenecks in complex SQL joins (Silberschatz et al., 2020). Monitoring the execution costs allows adjusting queries and indexes proactively (Elmasri & Navathe, 2015). Systems performance research proves that monitoring the queries helps to maintain low API latencies even when the volume of the data increases.

Lastly, evaluations of academic literature of database persistence engines prove Supabase-managed PostgreSQL to be a reliable and scalable data storage system for enterprise applications (Silberschatz et al., 2020). Integration of row-level security, relational database ACID compliance, JSONB schema flexibility, and automatic generation of API made up for the gap between data safety administration and engineering convenience (Codd, 1970; Fielding, 2000). Therefore, literature on information systems provides strong support for PostgreSQL database engines as one of the pillars of digital operations.

## 9. Real-Time CDC & Distributed Event Streaming

Distributed event-driven architecture literature proves real-time data replication to be an essential feature of contemporary collaborative software systems (Tanenbaum & Van Steen, 2016). In client-server synchronization architecture, broadcasting data changes instantly by means of persistent WebSocket connection was an efficient alternative to client polling loop (Mesbah & van Deursen, 2007). Evaluations of software engineering literature confirm Supabase Realtime, built upon PostgreSQL logical decoding Write-Ahead Log (WAL) replication, to ensure timely updates of distributed interfaces (Silberschatz et al., 2020).

Systematic investigation of Change Data Capture (CDC) architectures proves direct parsing of database transaction logs as the way to get table mutation changes without application-level triggers (Silberschatz et al., 2020). Supabase Realtime utilizes PostgreSQL wal2json or logical replication slots to convert binary WAL streams into continuous JSON format of changes (Elmasri & Navathe, 2015). Computer science literature proves that CDC architectures had lower overhead of CPU utilization by databases compared to polling queries, which was necessary for change propagation.

Analysis of academic literature on Pub/Sub mechanics of WebSockets proves that maintaining persistent TCP connection allows real-time servers to push state changes to subscribed client channels instantly (Tanenbaum & Van Steen, 2016). By routing broadcast events through distributed Elixir/Phoenix Erlang VM nodes, real-time engines manage to process millions of WebSocket connections while keeping the memory overhead low (Armstrong, 2010). Research in distributed systems proves that Erlang actor model ensures high fault tolerance of live data stream.

Research of client-side state synchronization proves that getting instant row-level updates removes data staleness in administrative dashboard interfaces (Mesbah & van Deursen, 2007). When administrative users approve or update any document record, real-time CDC engines propagate these changes to all active supervisor interfaces within milliseconds (Shneiderman & Plaisant, 2010). Literature on Human-Computer Interaction proves that visual state synchronization provides higher operational awareness and no duplicate efforts.

Design of a real-time system that uses database Row-Level Security (RLS) with event broadcast channels ensures that change events for rows of authorized data were broadcasted only (Saltzer & Schroeder, 1975). Real-time filter layers check subscriber JWT credentials against RLS rules of table before broadcasting any WAL mutations through WebSocket channels (Silberschatz et al., 2020). Information security research confirms that integration of RLS prevents data leakage in multi-tenant real-time channels.

Literature analyzing presence and broadcast tracking algorithms indicates that tracking active user states (for instance, active document editing and position of cursors) improves collaborative processes (Tanenbaum & Van Steen, 2016). By using in-memory CRDTs (Conflict-Free Replicated Data Types) or distributed presence pools, client applications broadcast transitory state metadata without writing transitory update records into the database (Shneiderman & Plaisant, 2010). Systems engineering research shows that transitory state broadcasting decreases load on persistent databases while improving collaborative processes.

Research analyzing mobile network reconnection strategy highlights exponential backoff and replay message queue as key components of strategy used when dealing with mobile network instability (Tanenbaum & Van Steen, 2016). In cases where WebSocket connection

with client was temporarily lost, messages remain in queue until network connectivity was restored (Mesbah & van Deursen, 2007). Software reliability research proves that resiliency in the reconnection process helps to avoid data loss when connection with the network was lost temporarily.

In case of enterprise system monitoring, tracking of real-time event distribution metrics enables system engineers to detect network delivery bottlenecks proactively (Newman, 2015). Telemetry tools monitor latencies of WebSocket messages, channel subscriptions, and WAL replication lag times in real time. Computer science researchers emphasize that operational observability made sure of high service availability on distributed application tiers.

Empirical evaluations measuring network overhead in CDC real-time systems confirm that streaming sparse change deltas reduces bandwidth consumption compared to full page updates (Tanenbaum & Van Steen, 2016). Real-time broadcast of only modified columns of JSON payload ensures low volume of packets sent over cellular networks (Nurseitov et al., 2009). Performance web research proves that broadcasting delta changes optimizes mobile client's battery and data consumption.

Literature in information security regarding the boundaries of real-time messaging asserts that real-time messaging channels should encrypt all packets with TLS (wss:// protocol) in order to mitigate packet sniffing attacks (Saltzer & Schroeder, 1975). Real-time event stream encryption protects the document state transitions from any possible packet sniffing when traversing public networks (Pressman, 2019). Information security experts highlight the importance of enforcing transport security for enterprise real-time messaging communications.

Research work on real-time database publication filters reveals that subscribing only for table events (INSERT, UPDATE, DELETE) or

particular primary key filters reduces the amount of client events (Silberschatz et al., 2020). Filtering out channel subscription to relevant application domains helps client browsers to avoid receiving unnecessary database events noise (Mesbah & van Deursen, 2007). Research in system performance confirms that subscription filtering improves client CPU utilization.

Technical literature evaluating the advantages of event-driven architecture reveals that integration of real-time database CDC with edge serverless computing enables automated workflows (Newman, 2015). Immediate execution of edge functions on particular database mutation triggers the automation of secondary tasks like sending notification emails or OCRing the documents without database blocking (Davenport & Ronanki, 2018).

Comparative analysis of polling architecture and CDC real-time streaming shows that log-based event propagation reduces the server CPU utilization of database by 80% during high concurrency user loads (Silberschatz et al., 2020). Getting rid of polling with repetitive SQL queries allows the database to process user transactions instead of polling requests (Elmasri & Navathe, 2015). Consequently, software architecture literature identifies CDC real-time streaming as the standard pattern for high-concurrency platforms.

Research work in multi-region synchronization reveals that placing edge WebSocket nodes nearby the users' location helps reduce connection latencies (Satyanarayanan, 2017). Geographically distributed edge nodes propagate database replication events through optimized cloud infrastructure backbone, ensuring worldwide low latency updates. Distributed computing research proves that edge acceleration ensures responsiveness in real-time events distribution.

Lastly, evaluation of real-time engines by the academia confirms that Supabase Realtime represents an optimal and secure architecture for modern event-driven applications (Tanenbaum & Van Steen, 2016). According to software engineering specialists, integration of PostgreSQL Write-Ahead Log CDC with WebSocket communication ensures unification of relational persistence with operational observability (Silberschatz et al., 2020). Hence, the literature in the area of information systems is supportive of real-time CDC engines as an integral pillar for digital administrative systems.

#### 10. Vercel & Serverless Edge Infrastructure

In the context of cloud computing, serverless PaaS architectures ensure the absence of server provision operations, which enables fast deployment and dynamic scalability of web applications (Armbrust et al., 2010; Buyya et al., 2018). The current practice of deployment of cloud frontends includes integration of Vercel Global Edge Network that combines static assets delivery with serverless architectures (Satyanarayanan, 2017). Evaluation of software engineering confirms that deployment of frontend single-page applications to edge platforms guarantees low latency, high availability and automatic deployment (Pressman, 2019).

Systematic investigations into Edge Network Caching demonstrate that distributing static assets across geographically dispersed Point of Presence (PoP) edge nodes minimizes network latency for global users (Satyanarayanan, 2017). By serving pre-rendered HTML shells, compiled JavaScript bundles, and CSS assets directly from edge caches, client applications achieve sub-100ms Time to First Byte (TTFB) loading times worldwide (St. Amant & McClain, 2018). Web performance research proves that edge asset caching drastically reduces server origin load while accelerating client application rendering.



The literature analysis of Serverless Functions execution (like Vercel Serverless/Edge Functions) demonstrates that ephemeral event-driven execution scales dynamically according to demands of incoming traffic (Buyya et al., 2018). Rather than running servers constantly, serverless runtimes spawn ephemeral micro execution contexts to process the API calls and shut down the execution environments right after it (Armbrust et al., 2010). Studies of systems engineering confirm optimization of infrastructure utilization and conversion of server costs into usage costs for serverless computing architectures.

Literature on CI/CD automation of cloud infrastructures emphasizes Git-based deployment pipelines as a great approach for improving engineering velocity (Fitzgerald & Stol, 2017). Every push to the branches of repositories triggers automated builds, preview environments, and automated testing frameworks (Humble & Farley, 2010). Software management research demonstrates that preview deployments provide better collaboration within teams, simplify code review processes and minimize risks during the pre-production stage.

The design of cloud enterprise security involves automated certificate provisioning for SSL/TLS, WAFs and DDoS protection mechanisms for application endpoints (Saltzer & Schroeder, 1975). Edge networks dynamically absorb large volume attacks at the network level before any malicious requests would hit application servers or database nodes (Buyya et al., 2018). Cybersecurity studies demonstrate that edge network security measures represent a crucial first line of defense for enterprise platforms.

The literature on Incremental Static Regeneration (ISR) and Partial Prerender (PPR) paradigms indicates that static edge caching with dynamic server-side execution maximizes page loading metrics (Iskandar et al., 2020). ISR allows

static edge pages to re-render asynchronously in the background when underlying data mutates, serving fast static content without delivering stale data to users (Bundschuh et al., 2019). Computer science scholars confirm that hybrid static-dynamic rendering balances low edge latency with fresh database content.

The literature on global domain routing and zero-downtime Blue-Green deployments points out that dynamic DNS routing algorithms enable instant traffic switchover to new software builds without terminating established connections (Tanenbaum & Van Steen, 2016). Instantly switching the production traffic back to older build artifacts in case of operational defects enables engineers to minimize the number of downtime incidents (Humble & Farley, 2010). Research on software reliability emphasizes that instant atomic release rollbacks ensure continuous system availability for ongoing updates.

Regarding cloud observability and performance telemetry, the literature highlights that real-time analytics monitoring metrics such as Core Web Vitals, serverless function duration, and error rate provides engineers with platform observability (Forsgren et al., 2018). Real-user monitoring (RUM) metric collection over global client sessions enables identification of regional latency problems and proactive code optimization (St. Amant & McClain, 2018). Literature on systems performance confirms that continuous telemetry instrumentwasrequired for maintaining SLAs.

Serverless cold start latency empirical evaluations indicate that lightweight JavaScript/WebAssembly edge runtimes significantly decrease cold start latencies in comparison with heavy containers (Armbrust et al., 2010). The elimination of unnecessary Node.js APIs and code execution inside isolated V8 isolates result in < 10 ms edge functions cold start latencies (Bierman et al., 2014). Computer science literature proves that low latency edge runtime ensures

interface responsiveness during server-side API processing.

Information security literature analyzing cloud environment configuration stresses that managing sensitive API keys and database credentials via secure Environment Variable management systems prevents credential leaks (Saltzer & Schroeder, 1975). Restricting environment secret access to encrypted production build pipelines ensures secrets were never exposed in client bundle code (Pressman, 2019). Security scholars emphasize that encrypted environment configuration management was mandatory for administrative cloud security.

Research on edge middleware execution indicates that edge-level authentication, geolocation, and request rewrites filter out unauthorized requests from reaching the origin (Satyanarayanan, 2017). Edge middleware performs instantaneous evaluation of incoming user session tokens and redirects unauthenticated traffic avoiding consumption of expensive backend processing resources (Newman, 2015). Literature on systems performance highlights that edge-level request filtering helps preserve the compute capacity and optimize the platform efficiency.

Technical sources discussing the process of image optimization service note that serverless image conversion and image compression decrease bandwidth usage of media assets by as much as 70% (St. Amant & McClain, 2018). Converting media files such as administrative documents and images on the fly in WebP and AVIF format helps to decrease client side loading time considerably (Bundschuh et al., 2019). It was proven by Web engineering literature that automated media optimization increases application hydration speed.

Studies comparing monolithic cloud hosting, for example, EC2 virtual machines, to serverless edge platforms demonstrate that edge platforms provide higher deployment frequency, less

maintenance costs, and better global availability metrics (Armbrust et al., 2010; Forsgren et al., 2018). As far as engineers did not had to took care of server maintenance, OS updates and scaling, they could concentrate on developing business logic (Kim et al., 2016). Thus, software architecture literature states that serverless edge infrastructure was the standard hosting platform for web applications.

Studies examining disaster recovery and multi-region resilience of platforms proved that redundant edge node routing provides platform reliability during a failure of a particular cloud data center (Tanenbaum & Van Steen, 2016). In case the cloud data center fails, edge DNS routers automatically redirect clients' requests to alternative working data centers (Buyya et al., 2018). Distributed systems research proved that automatic regional failover routing preserves platform operational continuity.

## **2.4 Synthesis of the Reviewed Literature**

The reviewed literature and studies collectively demonstrate that the digital transformation of administrative and human resource workflows significantly improved operational efficiency, data accuracy, and organizational transparency. Foreign research focused that while digital onboarding and document management systems simplify administrative procedures, their success depended heavily on structured communication, user-friendly interface designs, and real-time interaction to prevent newcomer isolation and feedback delays. Furthermore, studies on technical architectures revealed that modern web development frameworks specifically Single Page

Application (SPA) architectures using ReactJS, persistent bidirectional communication protocols via WebSockets, and modular cloud architectures like Supabase with Row-Level Security, provided the necessary responsiveness, low latency, and data integrity needed for complex, high-performance digital workflows. In addition, modern studies stated that adopting DevOps practices, including continuous integration, automated testing, and continuous deployment, improved software quality by enabling rapid feature delivery, early defect detection, and reliable system maintenance throughout the software development lifecycle.

Local literature reinforced these findings within the Philippine context, proving that changes from paper-based and manual file tracking to web-based management information systems drastically reduced processed times, limit document loss, and increased operational productivity across local government units, academic institutions, and enterprise HR environments. Additionally, local studies highlighted that digital document platforms handling sensitive personal identification and employment contracts had to strictly comply to statutory requirements such as Republic Act No. 10173 (Data Privacy Act of 2012) through multi-layered security controls, including encrypted storage, secure authentication, and Role-Based Access Control (RBAC). These findings also supported the implementation of secure cloud-based platforms that combined centralized document management with role-based authorization and continuous system monitoring to ensure both regulatory compliance and operational reliability.

Despite of the documented benefits, several critical gaps remain in the existing literature and current systems:

- Many standard digital onboarding platforms lack continuous, real-time synchronization between applicants, HR personnel and administration. Traditional request to response (HTTP polling) architectures result in slow feedback loops, leaving users

unaware of sudden changes in document status changes or approval requirements.

- Prior research indicated a disconnect between fully automated, AI-driven automation models which lack necessary human evaluation and traditional paper-based processes which were prone to administrative mistakes and most of that, it needed human decision making. Existing platforms often fail to achieve a balanced workflow that provides automated routing while keeping HR decision makers actively in the loop.
- Many local web-based repositories focused primarily on static document archiving and basic retrieval rather than incorporating dynamic, real-time access control policies such as Row-Level Security coupled with WebSocket transmission, to protect sensitive employee records while active transactions were taking place.

Furthermore, only a limited number of existing systems integrate DevOps practices throughout the software development lifecycle. Many studies focused primarily on the completed application rather than emphasizing continuous testing, automated deployment, and ongoing monitoring, which were essential for maintaining the reliability, security, and scalability of modern cloud-based web applications.

The proposed system directly bridges these identified research and technical gaps through an Automated End-to-End Onboarding System with Real-Time Document Handling:

- To resolve feedback delays, the project integrated WebSocket protocol and Change Data Capture (CDC) mechanisms (via Supabase Realtime). This ensured that document submissions, reviews, modifications, and approvals reflect instantly across Admin,

applicant, and HR personnel interfaces without requiring page reloading.

- The project implemented automated document routing, real-time status tracking, and centralized repository capabilities while directly preserving the human decision-making role. HR personnel maintained full supervision over critical verification and contract approval steps through an intuitive, state-aware portal built on Single Page Application (SPA) principles using ReactJS and Tailwind CSS.
- The proposed platform addressed security and legal compliance by implementing Role-Based Access Control (RBAC) paired with database Row-Level Security (RLS) and encrypted cloud storage. Aligned directly with the NIST Cybersecurity Framework and Republic Act No. 10173 (Data Privacy Act of 2012), the system ensured that sensitive employment records were only accessible to authorized roles during both storage and real-time transmission.
- Furthermore, the proposed system adopted DevOps practices throughout its development lifecycle by integrating continuous testing, version control through Git and GitHub, and automated deployment using Vercel. This approach supported rapid feature integration, early detection of defects, and reliable system maintenance, ensuring that updates could be delivered efficiently while maintaining the stability, security, and availability of the application.

## CHAPTER 3

### METHODOLOGY

#### 3.1 Research Design

The study utilized a Developmental Research Design, a systematic approach focused on the design, development, and evaluation of instructional programs, processes, or products. This specific design was selected because it allows for an iterative process of creation and validation, which was essential for solving the identified human resource (HR) inefficiencies in the local industry. Unlike descriptive research, which merely observes a problem, or experimental research, which tests a single hypothesis, developmental research aims to produce a functional solution. In this case, a software system that addresses the real-world gap between manual document handling and secure digital automation.

This design was categorized into two main types: Type I (Product and Tool Development) and Type II (Model Development). SecureFlow follows the Type I approach, as the primary objective was the creation of a tangible software tool that automates the recruitment workflow while enhancing data security and real-time document handling. By using this design, the research moves through a structured lifecycle: it begins with an intensive Analysis of user requirements to identify system needs, proceeds to the Design and Implementation of a role-based architecture, and concludes with a rigorous Evaluation based on accuracy, usability, and performance. To support this iterative development process, the researchers adopted the DevOps methodology, which emphasizes continuous development, testing, deployment, operations, and monitoring. This approach enabled the team to rapidly integrate new features, detect issues early, and maintain a stable and reliable application throughout the development



lifecycle.

Furthermore, the choice of a developmental design justifies the project's focus on software development principles and database management as practical applications of Information Technology knowledge. It provides a scientific framework for building the React frontend and Supabase backend, ensuring that the resulting platform was not just a coding exercise but a validated technological solution for modern organizational needs. This design ensures that every feature from the real-time status updates to the implementation of encrypted cloud storage and secure role-based access mechanisms, was grounded in a systematic methodology that addresses the specific security vulnerabilities found in current remote and hybrid work environments. By combining Developmental Research Design with the DevOps lifecycle, the study ensures that the software was continuously refined through iterative development, testing, deployment, and monitoring until it satisfies both the technical requirements and the operational needs of its intended users.

### **3.2 System Development Methodology**

The development of SecureFlow follows DevOps with the combination of Agile, an integrated approach that combines the iterative nature of Agile with the continuous integration and delivery (CI/CD) principles of DevOps. This methodology was selected to ensure that the system remains adaptable to changing user requirements while maintaining a stable, secure, and live deployed environment throughout the development lifecycle. By utilizing this approach, the project was broken down into small, manageable increments called **Sprints**, allowing the team to focus on specific features such as real-time updates or secure authentication in short cycles.

The methodology was structured into seven interconnected phases that

facilitate a continuous feedback loop between the developers and the stakeholders:

## **1. Plan**

This foundational phase involves conducting interviews and directed observations of manual preboarding workflow with the client to define the system requirements. The requirements gathered during this initial discovery were translated into a structured Product Backlog composed of user stories categorized by role (Applicant, HR Employee, and Administrator). To manage development risk, this backlog was divided into six two-week Agile sprints, prioritizing core infrastructure such as authentication and database schemas in early iterations. At the same time, the DevOps environment was established by initializing a GitHub version control repository and configuring Continuous Integration and Continuous Deployment (CI/CD) pipelines with Vercel and Supabase, ensuring that code combined at the end of each sprint was automatically built, tested, and deployed to a staging environment.

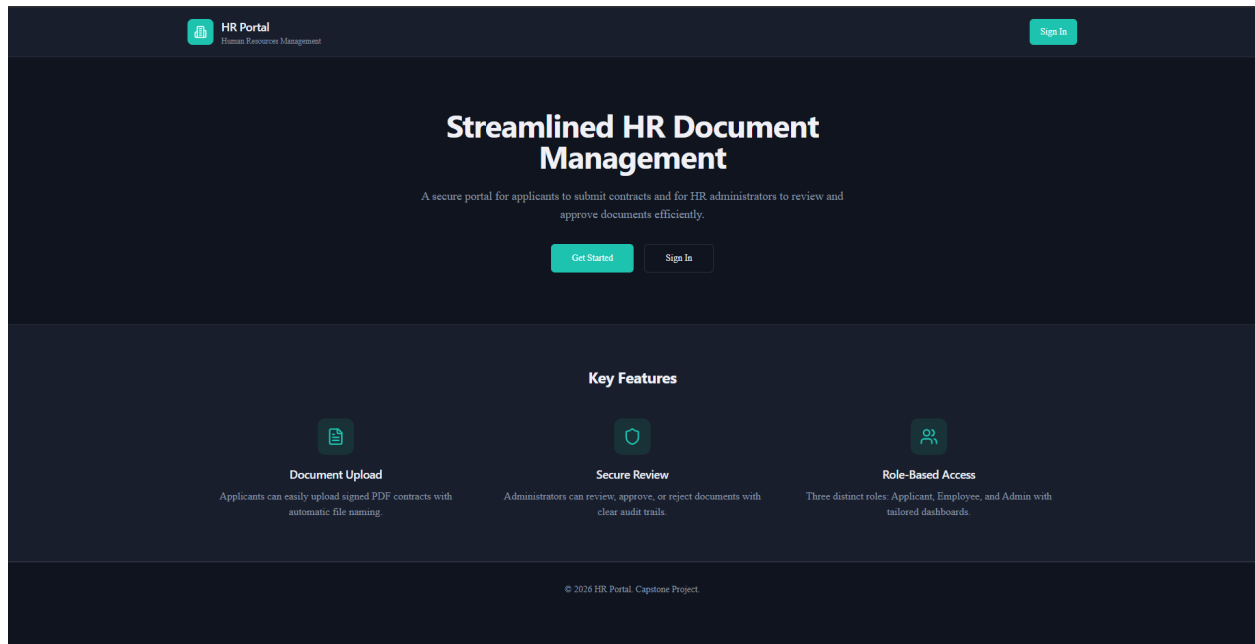
## **2. Design**

The design phase transformed the finalized functional requirements into detailed technical specifications, establishing both the data infrastructure and user interface components before front-end construction began. Relational entities including profiles, user\_roles and applicants were mapped using Entity Relationship Diagrams (ERD) to define foreign key constraints, primary keys, and automated database triggers. To safeguard sensitive candidate information, Row Level Security (RLS) policies were explicitly designed at the database level to enforce strict Role-Based Access Control (RBAC). Additionally, system interactions were modeled using Use Case and Sequence Diagrams to define the flow of data between the React

front-end, Supabase Storage, and background Edge Functions. Finally, high-fidelity wireframes were prototyped using a modern dark-mode aesthetic, ensuring accessible, responsive dashboard navigation across both desktop and mobile viewports. This ensures that the system architecture supports role-based access before a single line of code was written.

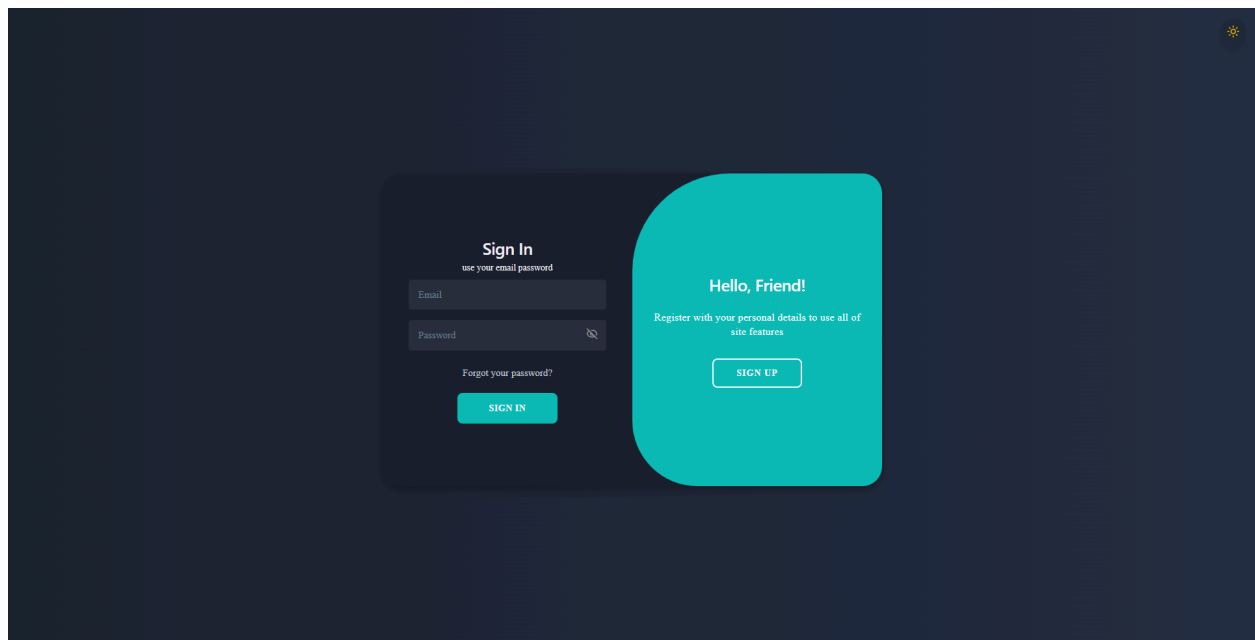
Following the frontend implementation using React.js and integration with Supabase cloud services, the platform's user interfaces were fully realized across its three primary user roles: **Applicant**, **HR**, and **Administrator**. This illustrates the live operational environment, demonstrating functional execution for user authentication, submission processing, administrative evaluation, and role-based data access.

**Home Page**



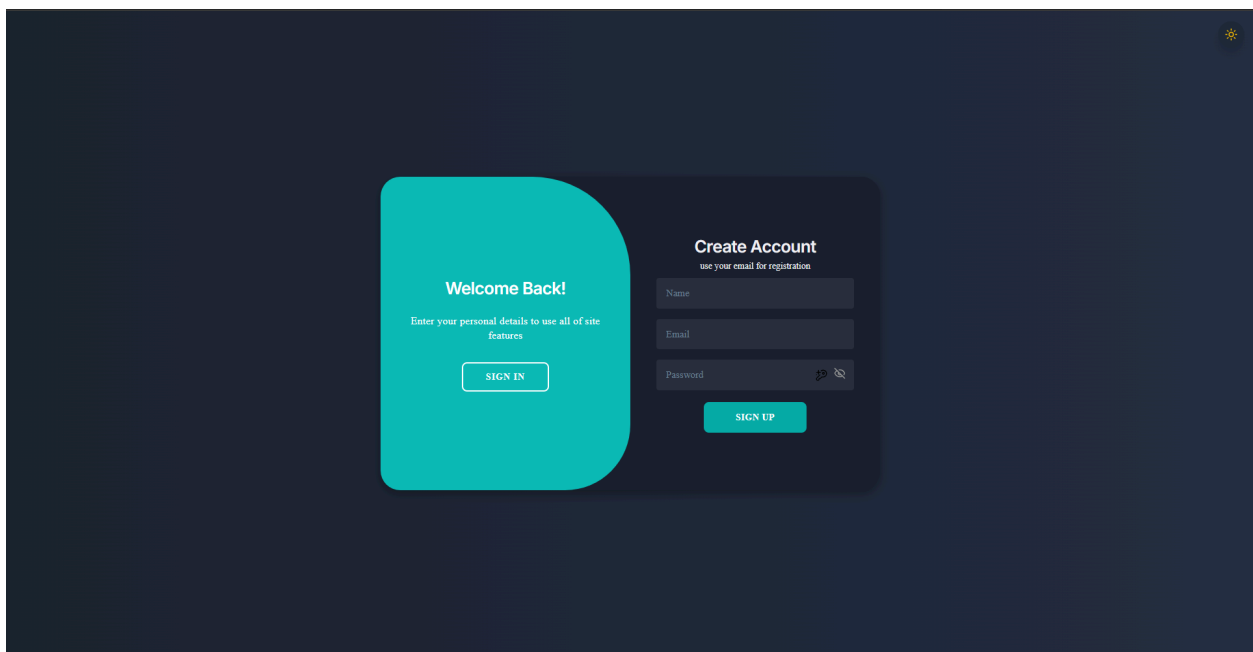
**Figure 3.1 Home Page**

The design phase transformed the finalized functional requirements into detailed technical specifications, establishing both the data infrastructure and user interface



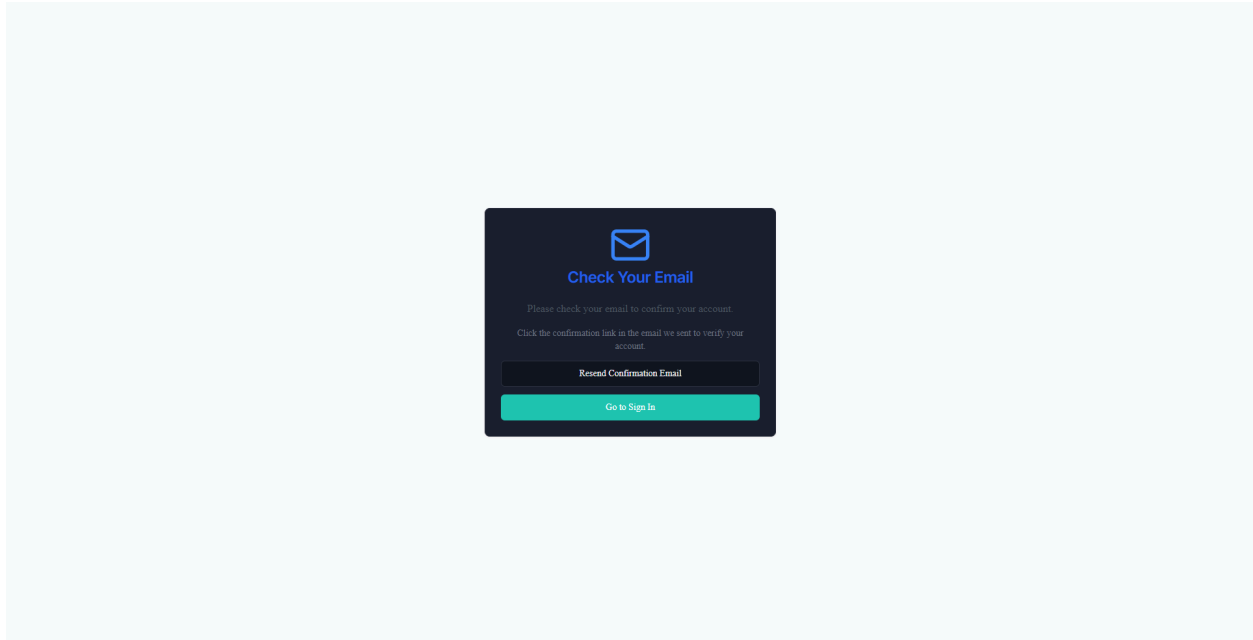
**Figure 3.2 Sign in**

A split-card modal UI for existing users to sign in using their registered email address and password. It includes password visibility toggles, a "Forgot your password?" recovery link, and an interactive side-panel ("SIGN UP") allowing users to switch dynamically to the registration form.



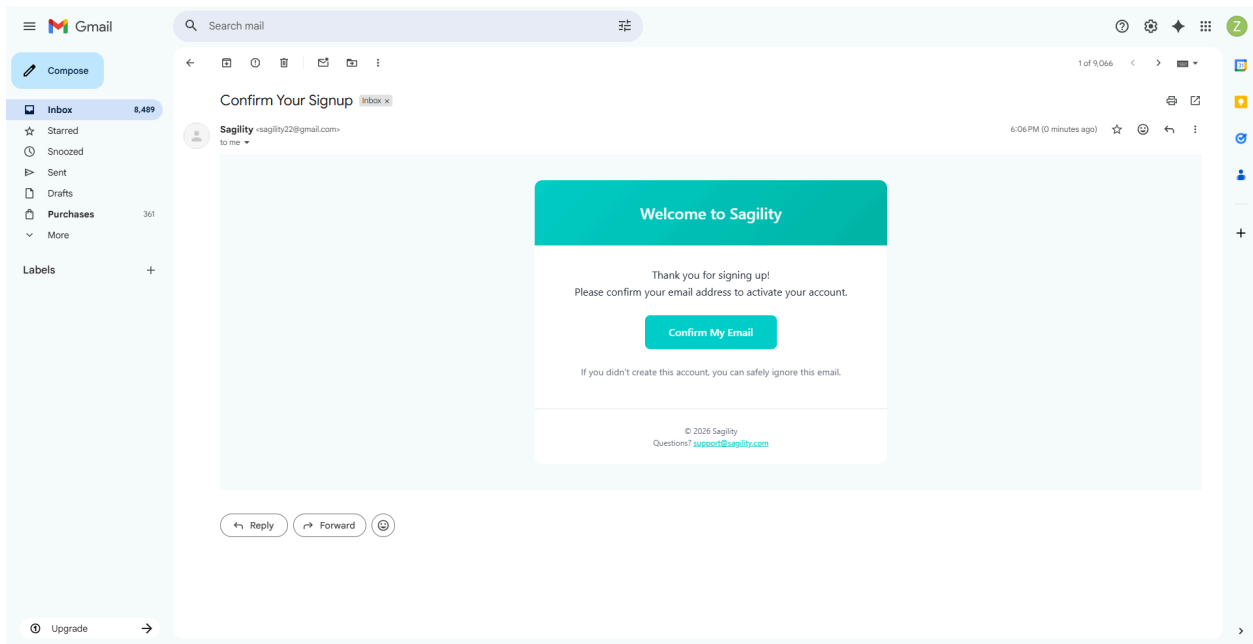
**Figure 3.3 Sign up**

A post-registration notification screen instructing the newly registered applicant to check their email inbox to confirm account creation. It includes buttons to resend the confirmation email or return directly to the Sign-In interface.



**Figure 3.4 Email confirmation**

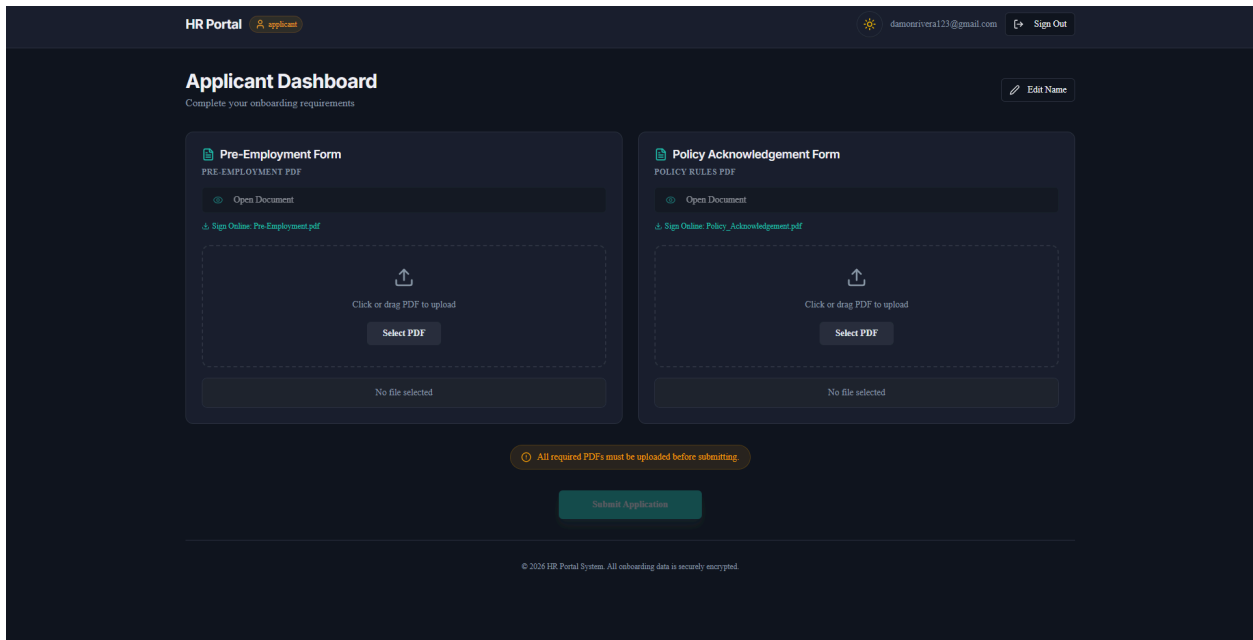
An automated email notification sent via SMTP integration. It contains a "Confirm My Email" verification button that authenticates the user's email address and grants them access to the Applicant Portal.



**Figure 3.5 Email confirmation**

An automated email notification sent via SMTP integration. It contains a "Confirm My Email" verification button that authenticates the user's email address and grants them access to the Applicant Portal.

## 1.Applicant Page



**Figure 3.6 Applicant Dashboard**

The main workspace for authenticated applicants. It displays assigned preboarding requirement cards (e.g., Pre-Employment Form and Policy Acknowledgement Form) featuring drag-and-drop file dropzones, direct PDF links, real-time validation warnings, and a submission lock that enforces uploading all required documents prior to final submission.

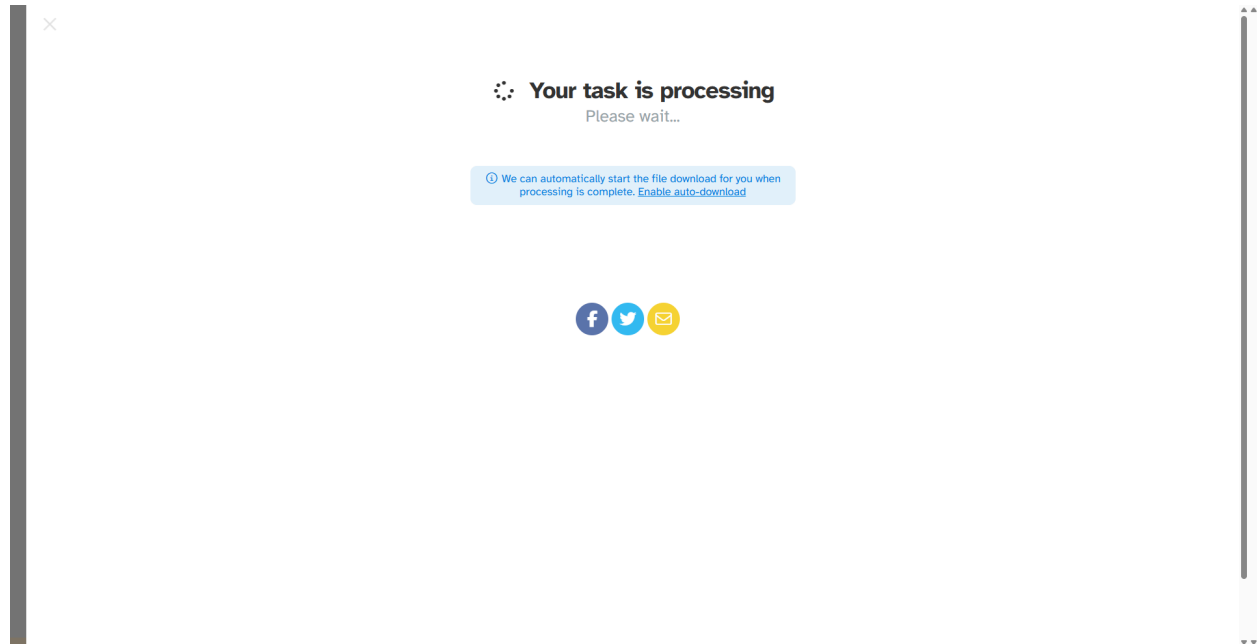


The screenshot shows a web-based PDF editor interface for Sagility. The header includes the Sagility logo and navigation links like 'All Tools', 'Compress', 'Edit', 'Fill & Sign', 'Merge', 'Delete Pages', 'Crop', 'Pricing', 'Desktop', and 'Log in'. The main title is 'Fill & sign PDF BETA' with the subtitle 'Add signature to PDF. Fill out PDF forms'. Below this is a toolbar with 'Text', 'Forms', 'Sign', and 'Undo' buttons. The document content is a 'NEW HIRE PAY SCHEME ACKNOWLEDGEMENT FORM' with a paragraph of text and a table. The table has three columns: 'Date of Joining', 'Covering', and 'First Payout'. The table contains five rows of data. A green 'Apply changes' button is visible at the bottom of the table.

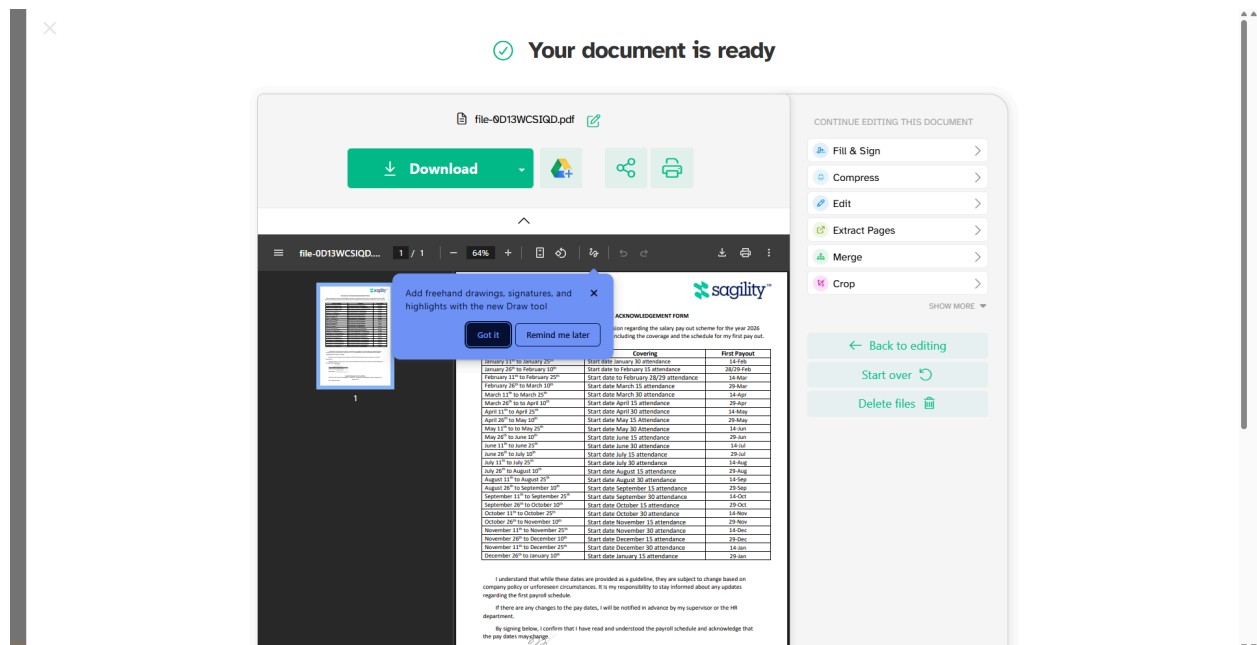
| Date of Joining                                        | Covering                                | First Payout |
|--------------------------------------------------------|-----------------------------------------|--------------|
| January 11 <sup>th</sup> to January 25 <sup>th</sup>   | Start date January 30 attendance        | 14-Feb       |
| January 26 <sup>th</sup> to February 10 <sup>th</sup>  | Start date to February 15 attendance    | 28/29-Feb    |
| February 11 <sup>th</sup> to February 25 <sup>th</sup> | Start date to February 28/29 attendance | 14-Mar       |
| February 26 <sup>th</sup> to March 10 <sup>th</sup>    | Start date March 15 attendance          | 29-Mar       |
| March 11 <sup>th</sup> to March 25 <sup>th</sup>       | Start date March 28 attendance          | 14-Apr       |
| March 26 <sup>th</sup> to April 10 <sup>th</sup>       | Start date April 12 attendance          | 29-Apr       |

**Figure 3.7 Applicant PDF Acknowledgement**

An integrated web-based PDF editor interface that allows applicants to review onboarding policies, complete mandatory fields, and attach digital signatures directly to requirement documents (such as the New Hire Pay Scheme Acknowledgement Form).



A modal loading state indicating active server-side processing while the system processes user inputs, generates completed PDF signatures, and packages the document for download or database upload.



A post-processing interface displaying the finished PDF document with

applicant signature verification. It allows applicants to preview, download, or edit their completed files before uploading them into the SecureFlow applicant dashboard.

The screenshot displays the Sejda 'Fill & sign PDF' web application. The interface includes a top navigation bar with the Sejda logo, a list of tools (All Tools, Compress, Edit, Fill & Sign, Merge, Delete Pages, Crop), and links for Pricing, Desktop, and Log in. The main heading is 'Fill & sign PDF BETA' with the subtitle 'Add signature to PDF. Fill out PDF forms'. Below this is a toolbar with buttons for Text, Forms, Sign, and Undo. A search bar is also present. The central area shows a PDF form from Sagility. The form is titled 'I. PERSONAL INFORMATION' and has a 'MANDATORY FIELDS' label. The form contains various input fields for personal data, with some fields marked as mandatory with a red slash icon. A green 'Apply changes' button is visible at the bottom of the form.

| I. PERSONAL INFORMATION |                      |                   |                      |
|-------------------------|----------------------|-------------------|----------------------|
| FAMILY NAME:            | <input type="text"/> | HOME ADDRESS:     | <input type="text"/> |
| GIVEN NAME:             | <input type="text"/> |                   |                      |
| MIDDLE NAME:            | <input type="text"/> | POSTAL CODE:      | <input type="text"/> |
| NICKNAME:               | <input type="text"/> | TEL. NO.:         | <input type="text"/> |
| GENDER:                 | <input type="text"/> | MOBILE NO.:       | <input type="text"/> |
| CIVIL STATUS:           | <input type="text"/> | OFFICE NO./LOCAL: | <input type="text"/> |
| BLOOD TYPE:             | <input type="text"/> | NATIONALITY:      | <input type="text"/> |
| BIRTH DATE:             | <input type="text"/> |                   |                      |

An online form editor displaying mandatory personal information fields (Family Name, Given Name, Home Address, Tax/SSS IDs). It highlights required inputs to ensure data completeness prior to document submission.

I A TextFormsSignUndo



AUTHORIZATION FORM

I hereby authorize release to **Sagility Philippines B.V.**, the academic information required below for my pending job application with said company.

FULL NAME

:

DEGREE/COURSE

:

COLLEGE/UNIVERSITY

:

ADDRESS/BRANCH

:

STUDENT NO.

:

DATES ATTENDED

:

GRADUATION DATE

:

SCHOOL EMAIL ADDRESS

:

SCHOOL CONTACT NUMBER

:

Apply changes >

An interactive digital PDF form allowing applicants to input academic history—including full name, degree/course, institution, student number, and contact details—authorizing background verification checks for Sagility Philippines B.V. as part of the onboarding workflow.

Applicant Dashboard

Complete your onboarding requirements

Edit Name

Pre-Employment Form

PRE-EMPLOYMENT PDF

Open Document

Sign Online: Pre-Employment.pdf

Click or drag PDF to upload

Select PDF

file-4D13WCSIQD.pdf

Policy Acknowledgement Form

POLICY RULES PDF

Open Document

Sign Online: Policy\_Acknowledgement.pdf

Click or drag PDF to upload

Select PDF

file-3VXL1E4UYYY.pdf

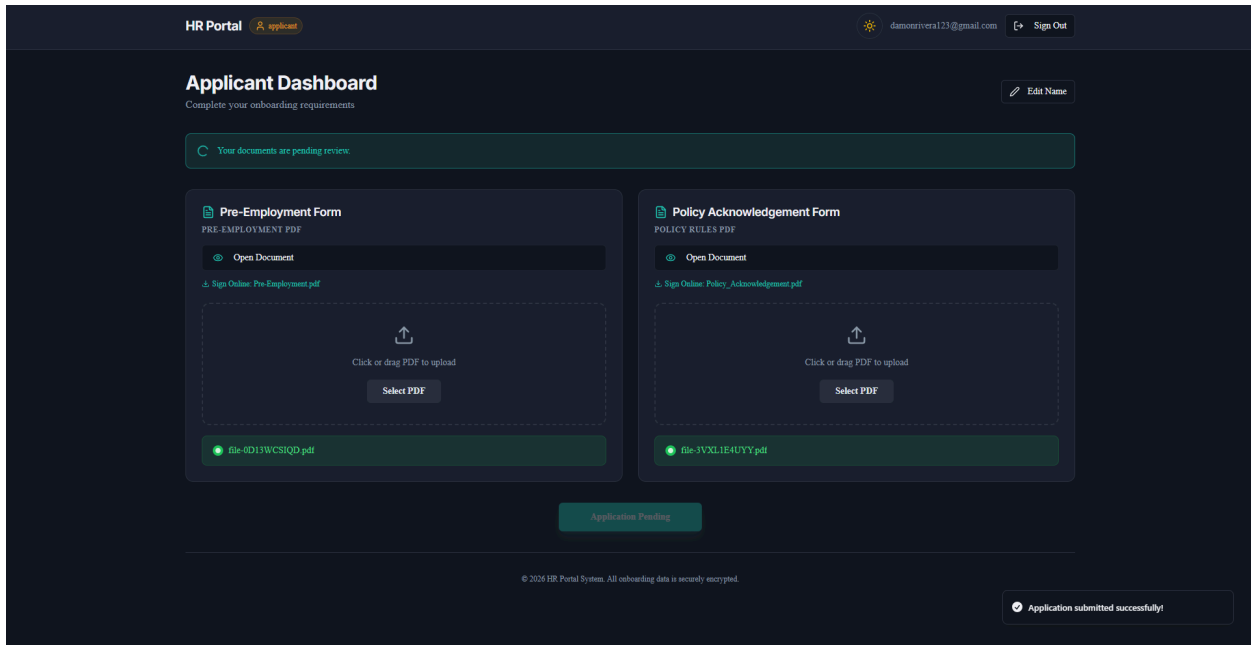
All required PDFs must be uploaded before submitting.

Submit Application

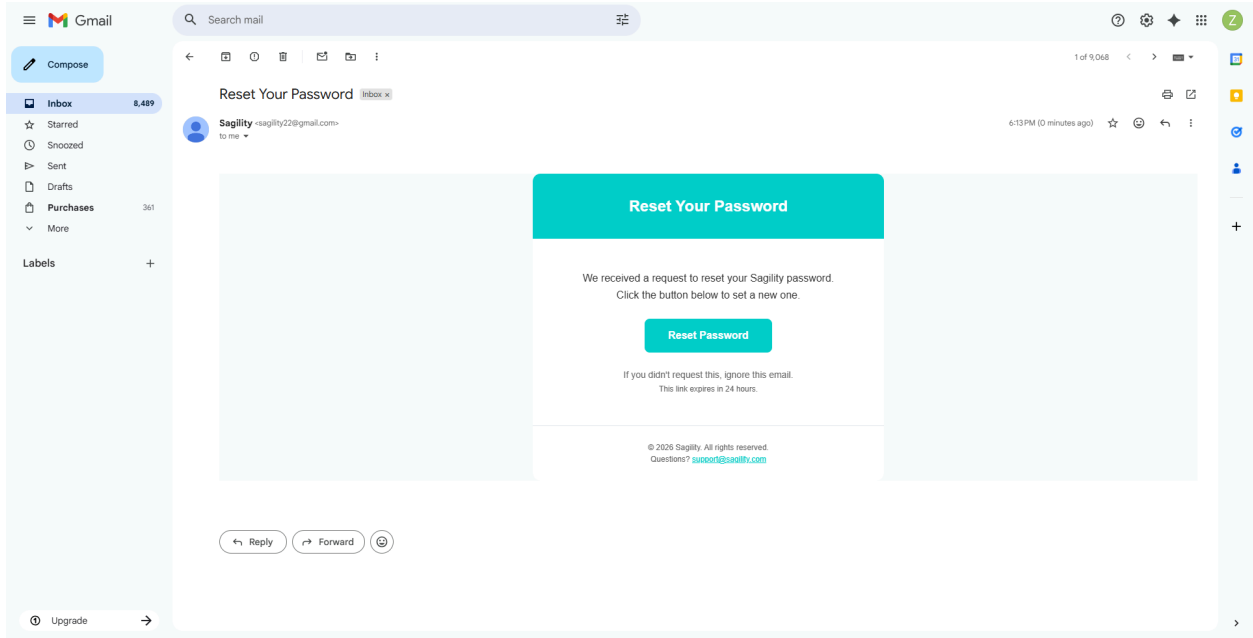
© 2025 HR Portal System. All onboarding data is securely encrypted.

✓ Name updated successfully!

The applicant dashboard displaying both required PDF files successfully staged in green upload slots. A bottom toast alert ("Name updated successfully!") confirms real-time profile data sync, enabling the "Submit Application" button.



The dashboard state following successful application submission. A top status banner confirms "Your documents were pending review," the action button transitions to "Application Pending" (disabled), and a toast notification confirms submission success, locking further document modifications.



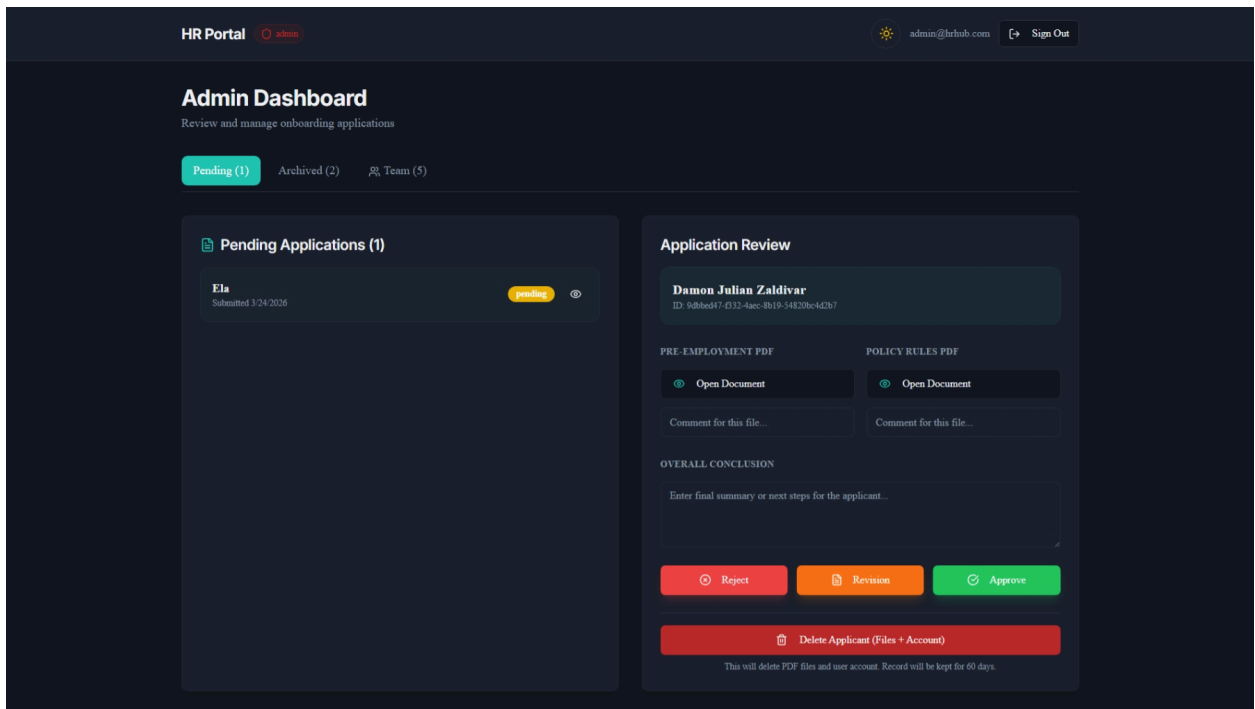
An automated account recovery email triggered via Supabase Auth and SMTP. It provides a secure, time-sensitive "Reset Password" button allowing the user to recover lost account credentials securely.

A screenshot of a "Reset Password" form. The form has a dark background with a light-colored card in the center. The card has a title "Reset Password" and a subtitle "Enter your new password". There are two input fields: "New Password" and "Confirm Password". Both fields have a placeholder text "Enter new password" and a toggle icon. Below the input fields is a teal button labeled "Update Password". At the bottom of the card, there is a section titled "Password must include:" with a list of requirements: "At least 8 characters", "Uppercase letter", "Lowercase letter", "Number", and "Special character (@, #, \$, etc.)". Each requirement is preceded by a checkmark.

The password reset entry form prompting users for a "New Password"

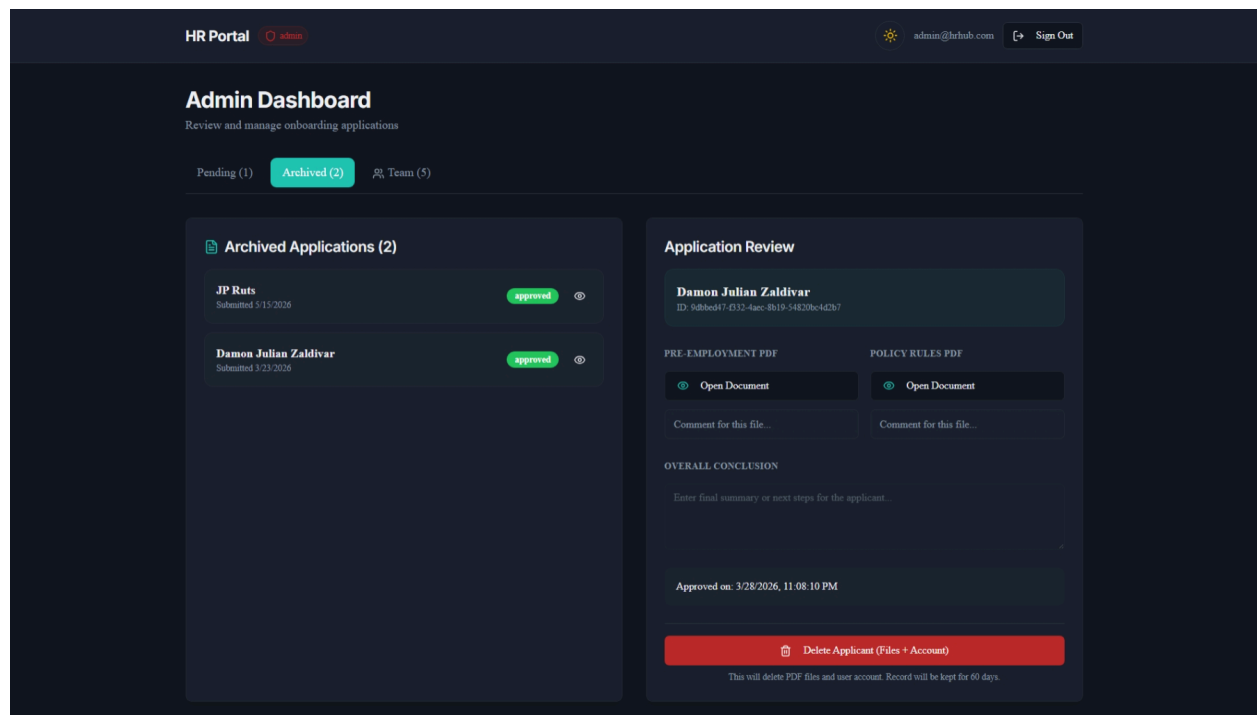
and "Confirm Password". It features dynamic real-time password security criteria validation (minimum 8 characters, uppercase, lowercase, numbers, and special characters) before enabling account updates.

## Admin Page



The screenshot displays the Admin Dashboard of an HR Portal. The top navigation bar includes the "HR Portal" logo, a user profile icon, the email "admin@hrhub.com", and a "Sign Out" button. The main heading is "Admin Dashboard" with the subtitle "Review and manage onboarding applications". Below this, there are three tabs: "Pending (1)", "Archived (2)", and "Team (5)". The "Pending Applications (1)" section shows a single application for "Ela" submitted on 3/24/2026, with a "pending" status. The "Application Review" section for "Damon Julian Zaldivar" (ID: 9dbbed47-0332-4aec-8b19-54820bc42b7) includes two PDF documents: "PRE-EMPLOYMENT PDF" and "POLICY RULES PDF", each with an "Open Document" button and a "Comment for this file..." field. Below these is an "OVERALL CONCLUSION" section with a text area for a final summary or next steps. At the bottom of the review section are three buttons: "Reject", "Revision", and "Approve". A red button labeled "Delete Applicant (Files + Account)" is at the very bottom, with a note: "This will delete PDF files and user account. Record will be kept for 60 days."

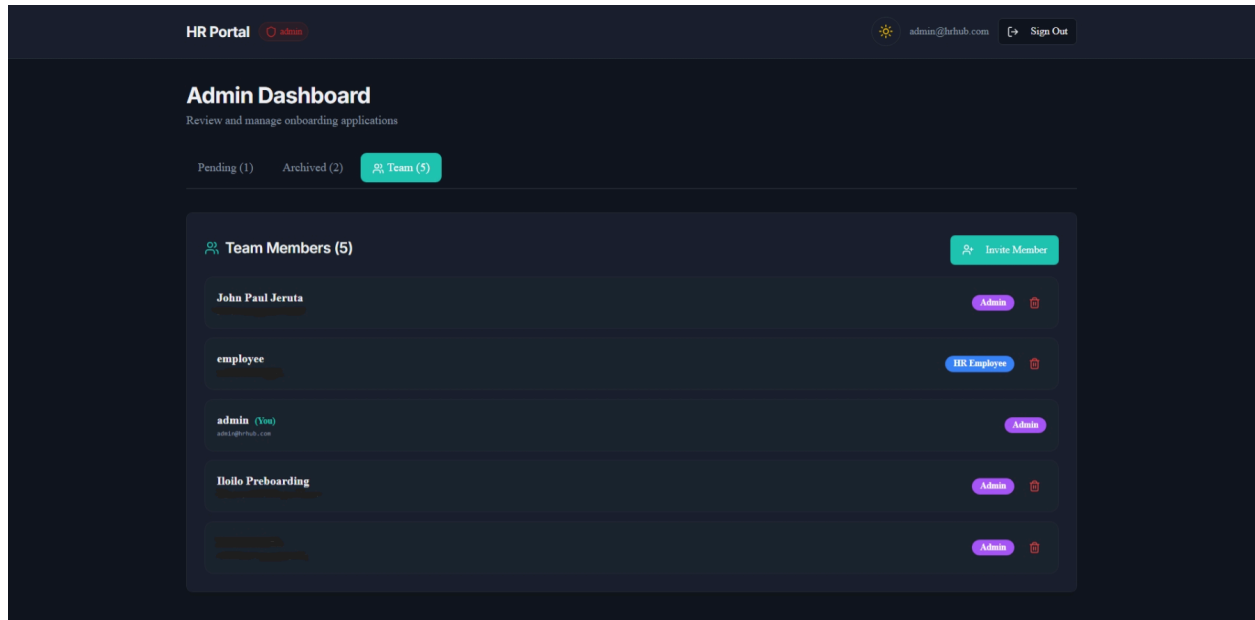
The primary evaluation workspace for administrators. The left panel lists pending applicant submissions (e.g., *Ela*), while the right panel allows evaluators to inspect submitted PDF documents, enter field-level feedback, input an overall evaluation summary, delete application and issue an official status decision ("Reject", "Revision" "Approve").



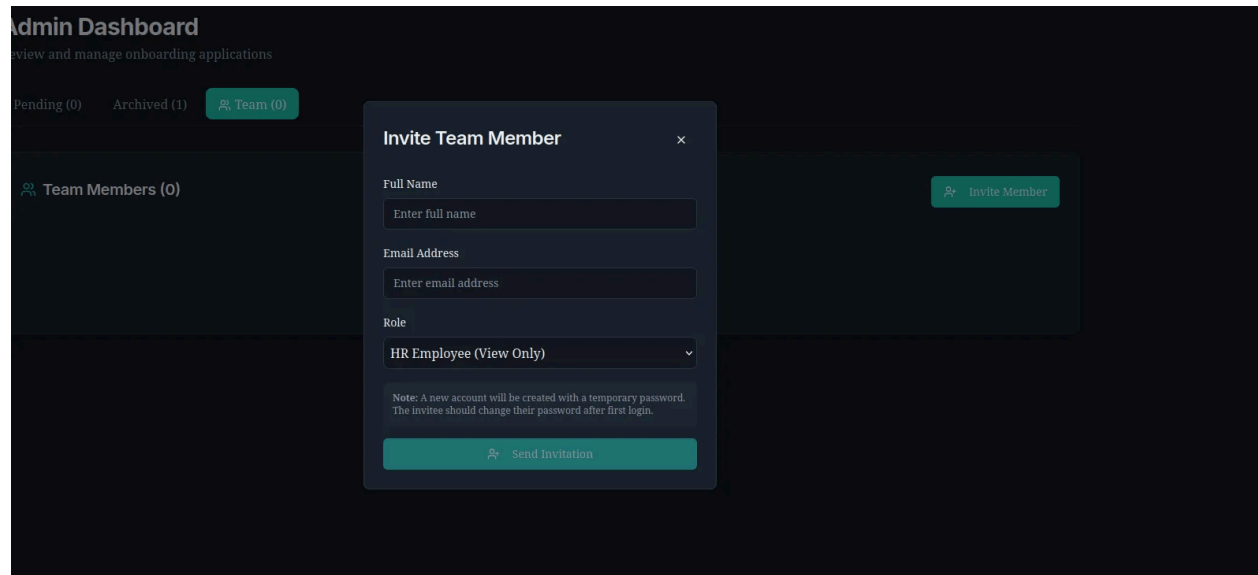
the administrator views all archived applicant records. The admin



could select an applicant to review submitted documents, read previous comments, view the approval date, and permanently delete the applicant's account and uploaded files when necessary.

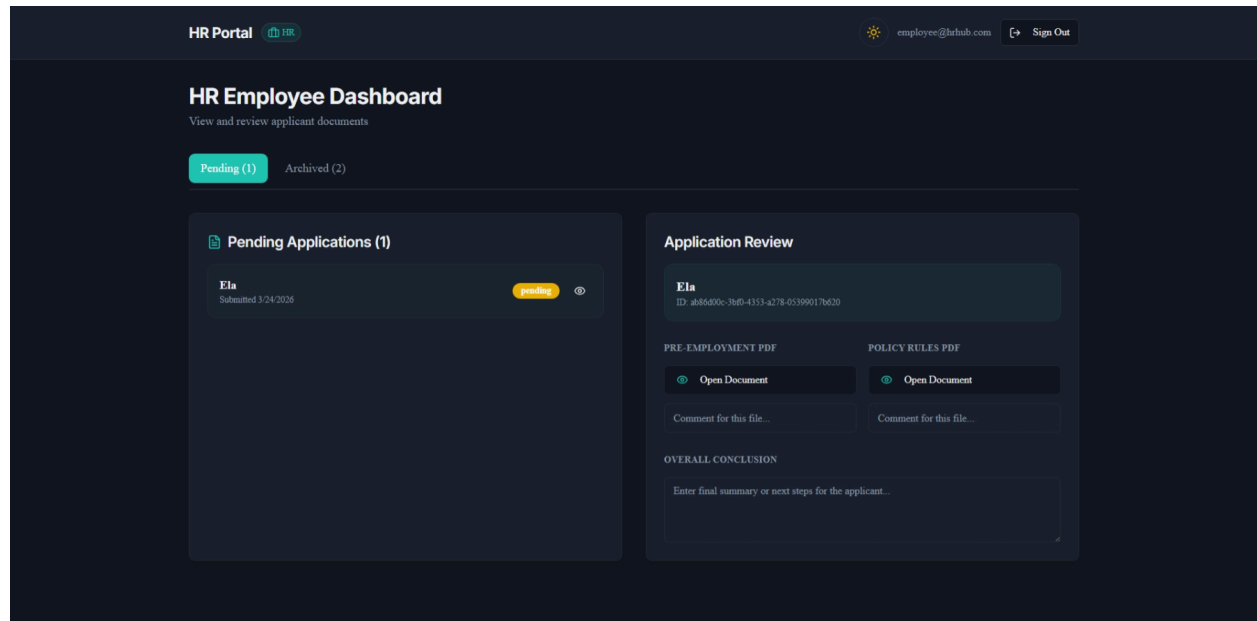


The administrative management view showing the "Team" management tab within the Admin Dashboard. It provides administrators with a dedicated interface to oversee HR personnel accounts and launch user invitation workflows via an "Invite Member" action button.

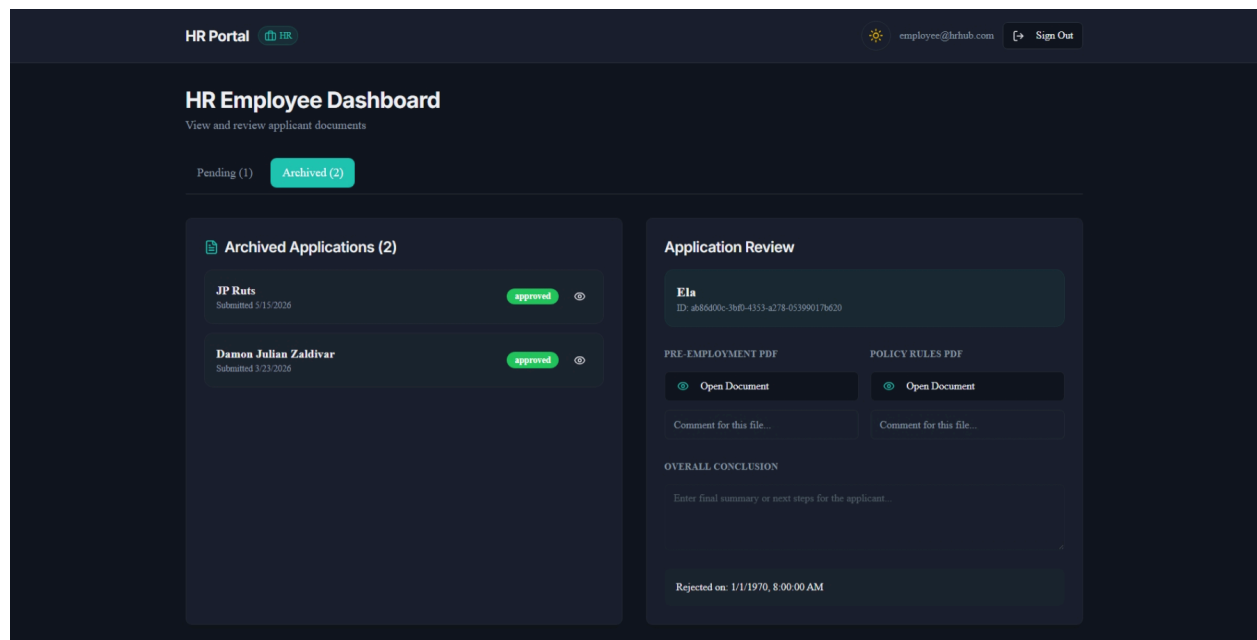


A modal dialog for onboarding internal staff. It collects full name and email credentials while providing granular Role-Based Access Control (RBAC) assignment options (e.g., *HR Employee (View Only)*), automatically dispatching temporary login credentials upon submission.

## HR Employee Page



This page allows HR employees to review pending applicant submissions. Users could open uploaded documents, provide comments for each file, and enter an overall assessment before forwarding the application for further review or approval.



This displays archived applications that had already been processed. HR employees could review applicant information, access submitted documents, and view the final application status and decision history for reference purposes.

### **3.Code (Development)**

Once the designs were locked in, this phase moved into writing the actual code by building the platform as a **Single Page Application (SPA)** using React.js, allowing applicants and admins to navigate smoothly between dashboards and forms without full page reloads. The frontend was broken down into reusable components, like dropzone file uploaders, real-time status badges, and interactive admin tables. For the backend, the system integrated Supabase to manage user authentication, PostgreSQL database tables, and secure storage buckets for uploaded PDFs. To handle server-side tasks such as sending approval emails and purging expired files, serverless Edge Functions were implemented using TypeScript. Throughout the coding phase, the team used Git and GitHub for version control, using feature branches to build separate components simultaneously like SPA routing and backend security rules, without conflicting before merging into the main branch.

### **4.Testing (Integration)**

Testing was integrated directly into our development workflow rather than left until the very end. Every time a new feature branch was merged into GitHub, we conducted testing to ensure new changes didn't break existing features. We broke down our QA process into three main categories:

| Testing Category                 | Tests Performed                               |
|----------------------------------|-----------------------------------------------|
| Authentication Testing           | User registration with all required fields    |
|                                  | Password strength validation                  |
|                                  | Password visibility toggle                    |
|                                  | Form field clearing on tab switch             |
|                                  | Email/password authentication                 |
|                                  | Role-based redirection                        |
| Database and Security Testing    | User role assignment                          |
|                                  | Row Level Security (RLS) policy enforcement   |
|                                  | Application creation and retrieval            |
|                                  | Status workflow (Pending → Approved/Rejected) |
| UI/UX and Responsiveness Testing | Dark mode toggle                              |
|                                  | Responsive layout                             |
|                                  | Form validation                               |
|                                  | Error message display                         |
|                                  | Loading state verification                    |

## **5. Deployment**

Once a feature successfully passed all testing checks, the code was merged and automatically pushed to a live production environment using **Vercel**. By linking the GitHub repository directly to Vercel, any approved changes immediately triggered a live build without requiring manual server uploads. This automated deployment pipeline ensured that HR personnel, Admins, and testing users always had real-time access to the most updated, functional version of the application at all times.

## **6. Operations**

In this phase where the system went live, the focus shifted to real-time monitoring and maintaining platform reliability. This phase involved managing backend performance, enforcing ongoing database security through Row Level Security (RLS) policies, and monitoring Supabase Edge Functions to ensure that applicant approval notifications and emails reach HR and Admin personnel without delay. Regular database logs and performance metrics were reviewed to address any runtime errors and kept the live environment secure and stable.

## **7. Monitoring**

The team monitored the system performance and checked logs for any technical issues. Client feedback was gathered during this stage to identify areas for improvement, which then fed back into the Planning phase to begin the next cycle of the project.

### 3.3 System Architecture

The SecureFlow application was an automated, web-based onboarding system engineered to streamline pre-employment document submission and evaluation while enforcing robust data security, role-based access control (RBAC), and real-time status tracking. The system addresses the operational bottlenecks and security vulnerabilities inherent in traditional paper-heavy and unencrypted digital onboarding workflows. By replacing fragmented communication channels such as physical paper submissions and standard email attachments with a centralized Single Page Application (SPA), SecureFlow provides dedicated, role-tailored interfaces for Applicants, HR Personnel, and System Administrators.

Unlike conventional Human Resource Information Systems (HRIS) that encompass broad, enterprise-wide employee management features, SecureFlow focuses specifically on optimizing the automated preboarding lifecycle. The platform enables job applicants to securely register, upload required pre-employment PDF credentials, and monitor their verification status in real time. For HR Personnel and System Administrators, the application provides an interactive verification dashboard equipped with embedded document previewing, per document feedback logging, workflow status updating (*Pending, Approved, Rejected*), and automated email notifications executed via serverless Edge Functions.

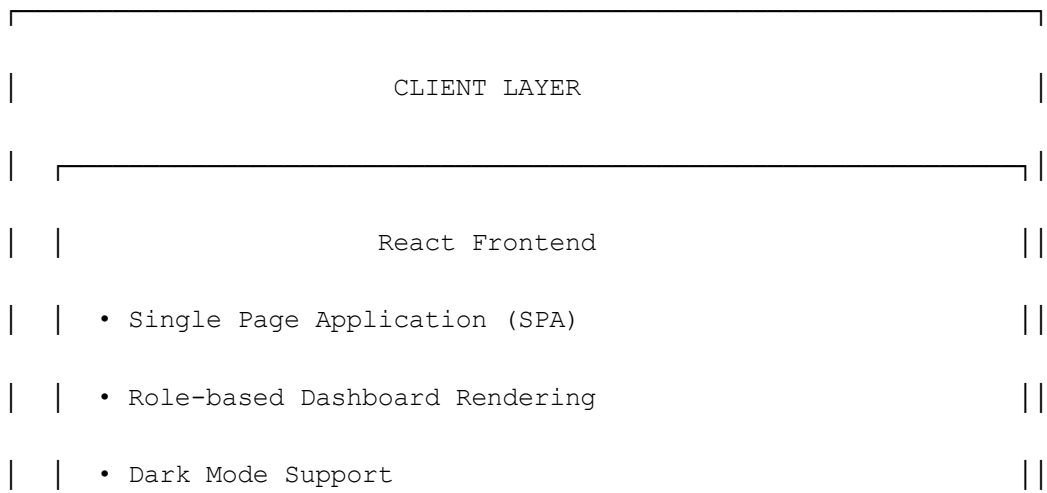
Built upon a modern, cloud-native architecture, the system leverages a React.js frontend deployed on Vercel for high-performance user interaction, paired with Supabase for backend services. Data

integrity and fine-grained privacy were sustained at the database level using PostgreSQL Row Level Security (RLS) policies, while unstructured document assets were securely stored and accessed through managed Supabase Storage buckets.

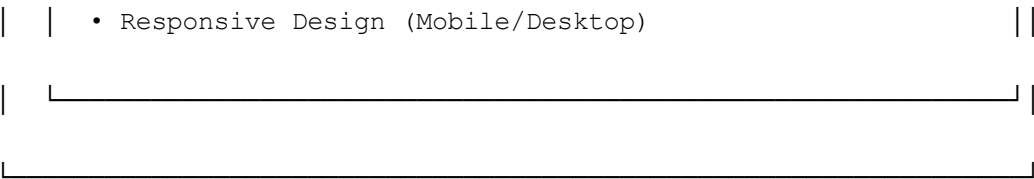
3.3.1 Client-server setup

The **SecureFlow** platform utilized a **three-tier cloud-based architecture** designed to handle real-time status tracking, document management, and strict role-based access control. This setup established a clear separation between the presentation frontend, serverless service layer, and database storage. By leveraging modern cloud infrastructure, the system ensured scalable document processing, low-latency updates, and database-level security across all user interactions.

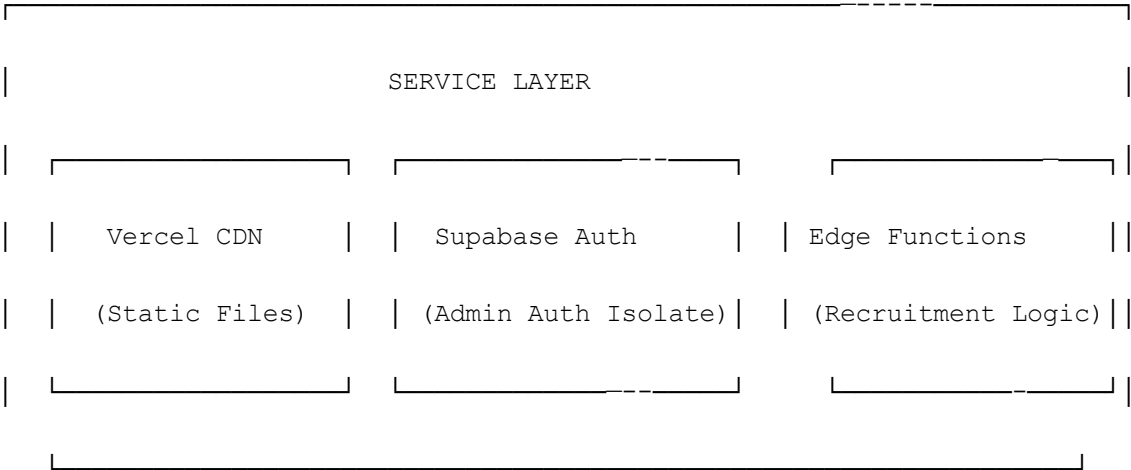
Three-tier cloud-based Architecture



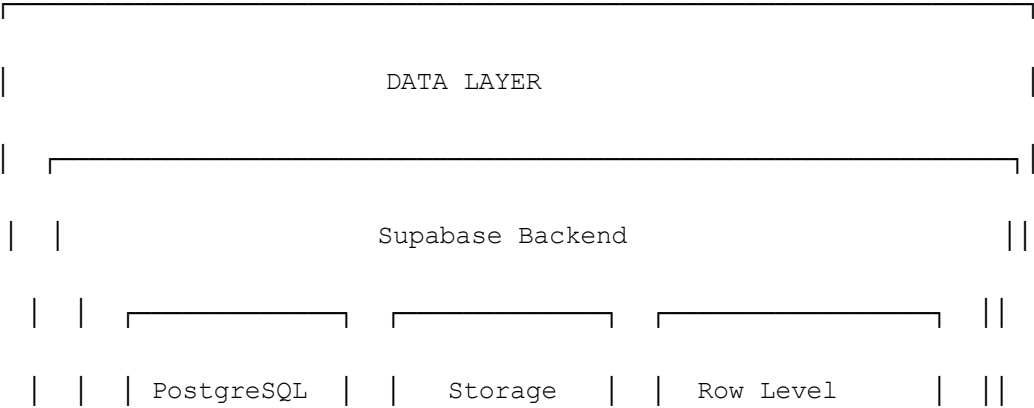


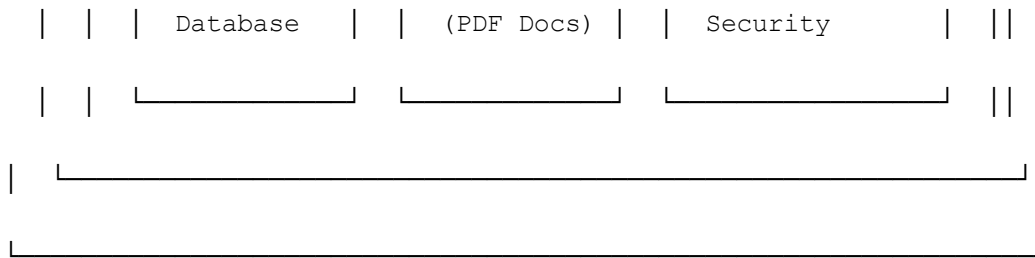


HTTPS (SSL/TLS)



REST API / WebSocket





The architecture was structured into three distinct operational tiers: the **Client Layer**, the **Service Layer**, and the **Data Layer**. Each layer performed specialized roles to guarantee low latency, robust data isolation, and real-time responsiveness throughout the onboarding workflow.

## 1. Client Layers

The Client Layer serves as the primary interface through which users interact with the SecureFlow system. The frontend operates as a Single Page Application (SPA) and built using React to provide a dynamic user experience without requiring full page reloads during navigation. The interface dynamically adapted its layout and features based on the logged in user's role, whether an Applicant, HR Employee, or Administrator, ensuring that users only saw the components relevant based on their authorization level. Additionally, the presentation layer incorporates responsive design principles to ensure seamless functionality across both desktop and mobile web browsers, alongside integrated Dark Mode support for enhanced accessibility.

## 2. Service Layer

The Service Layer acts as the intermediary execution engine, processing requests from the presentation layer and executing core

business logic over encrypted HTTPS (SSL/TLS) connections. Static frontend assets were hosted and distributed globally via Vercel's Content Delivery Network (CDN), minimizing loading latency for end users. User identity management and access control were handled through Supabase Auth, which manages authentication tokens and isolates administrative credentials to prevent unauthorized privilege escalation. Serverless Edge Functions handle system logic, including recruitment processes, payload validation for incoming files, and triggering real-time email notifications.

### **3. Data Layer**

The Data Layer oversees all persistent storage, file hosting, and data-level access enforcement, communicating with the Service Layer through RESTful APIs and WebSocket protocols for real-time status updates. Structured system data such as user credentials, application histories, review comments, and system logs, was securely stored within a PostgreSQL relational database. Unstructured binary files, specifically pre-employment forms and signed policy agreements in PDF format, were saved in dedicated, encrypted buckets within Supabase Storage. To enforce strict data governance, PostgreSQL Row Level Security (RLS) policies were applied directly at the database engine level, ensuring that users could only query or modify records specifically granted to their access role.

#### **3.3.2 User Interaction Flow**

The user interaction flow defines how each user engages with the system according to their assigned roles and system boundaries. The platform categorizes interactions across three distinct user roles: Applicants, Employees, and HR/Administrators.

The Applicant pathway guides candidates through account

creation, onboarding document submission, and real-time status monitoring. Applicants interact directly with the front-facing portal to fulfill pre-employment requirements and remain informed about their progress.

- Candidates set up their account credentials and complete their primary personal details.
- Applicants upload required onboarding credentials strictly formatted as PDF files.
- Candidates review and upload their signed policy agreement in PDF file.
- Applicants could update their demographic, contact details or any personal information at any point prior to final processing.
- Candidates could monitor real-time updates regarding their application progress directly onto their portal dashboard.
- Automated email updates were dispatched to the applicant upon approval or key application status changed by the higher up.

The HR Employee pathway provides administrative staff with visibility over incoming candidate files for monitoring purposes. Employees operate under a restricted permission model designed to preserve data integrity across candidate records.

- HR Employees could view all pending application submissions within the pending onboarding queue.
- Staff members had access to historical application records in a strictly read only format.
- HR Employees were restricted from approving, rejecting, modifying, or deleting any candidate records. The only task was to monitor all of the candidates files.

The Administrator pathway controls the governance, evaluation, and user lifecycle operations of the platform. HR/Admin possess elevated

access to made official approval decisions, manage system users, and oversee data retention.

- Administrators could access a comprehensive overview of all pending, active, and completed candidate files.
- Administrators evaluate submitted documents from the candidates and execute final decision actions.
- Reviewers attach explanatory notes, comments or feedback directly to individual candidate records.
- Administrators decide to issue platform invitations and provision access roles for new staff members.
- Administrators hold exclusive authority to permanently remove candidate entries when necessary.
- Administrators manage and inspect historical, archived candidate records.

### **3.4 Tools and Technologies Used**

#### **3.4.1 Frontend Technologies**

- **React.js:** Chosen for its component-based architecture, React allows the team to build a highly interactive UI that supports the real-time document handling requirements of the project.
- **TypeScript:** By using TypeScript (an extension of JavaScript), the development team ensures type safety, which reduces runtime errors and improves the overall reliability of the system.
- **Tailwind CSS:** This utility-first CSS framework was used to rapidly design a user-friendly and responsive interface for both the admin and applicant portals.

| Technology   | Version | Purpose             |
|--------------|---------|---------------------|
| React        | 18.3.1  | UI Framework        |
| TypeScript   | 5.8.3   | Type Safety         |
| Vite         | 5.4.19  | Build Tool          |
| Tailwind CSS | 3.4.17  | Styling             |
| Shadcn/UI    | Latest  | Component Library   |
| React Router | 6.30.1  | Client-side Routing |
| Lucide React | 0.462.0 | Icons               |

#### 3.4.2 Backend and Database Technologies

- Supabase (PostgreSQL): Instead of a standard SQL setup, SecureFlow utilizes Supabase to leverage PostgreSQL, a robust relational database known for its stability and advanced security features like Row-Level Security (RLS).
- Real-time Engine: Supabase provides built-in real-time data synchronization, which was essential for the system's "instant visibility" feature for document status changes.

| Technology         | Version | Purpose              |
|--------------------|---------|----------------------|
| Supabase           | Latest  | Backend-as-a-Service |
| PostgreSQL         | 15+     | Relational Database  |
| Edge Functions     | Deno    | Serverless Functions |
| Row Level Security | -       | Database Security    |

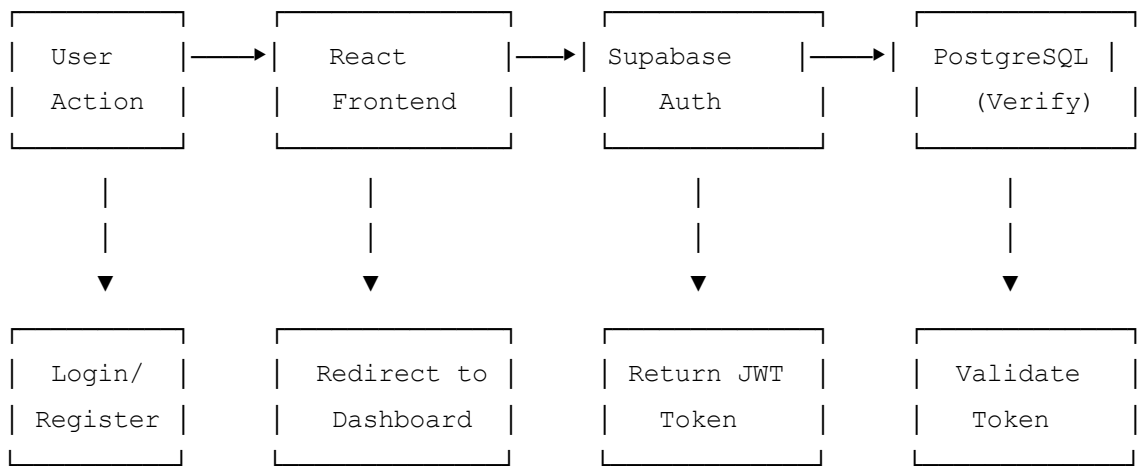
### 3.4.3 Deployment and Development Tools

- Vercel: This platform was used for deployment, ensuring that the frontend remains highly available and that updates were pushed automatically through the Agile DevOps cycle.
- Visual Studio Code & Git: These served as the primary development environment and version control system, facilitating seamless collaboration among the six developers on the team.

| Technology | Purpose            |
|------------|--------------------|
| Git        | Version Control    |
| npm        | Package Management |

|          |                 |
|----------|-----------------|
| ESLint   | Code Linting    |
| Prettier | Code Formatting |

3.3.3 Security architecture



The security architecture illustrates how user authentication and authorization were handled within the system. When a user logs in or registers through the React frontend, the request was processed by Supabase Authentication. Supabase verifies the user's credentials and returns a JWT (JSON Web Token) upon successful authentication. The token was then validated by PostgreSQL to ensure that only authenticated and authorized users could access protected system resources. After successful verification, the user was redirected to the appropriate dashboard.

3.3.2 Security Measures Implemented

The system implements several security measures to protect user



accounts, data, and system resources:

#### 1. Password Security

- Minimum 8 characters
- Requires uppercase, lowercase, number, special character
- Secure hashing via Supabase Auth

#### 2. Multi-Factor Authentication (MFA)

- Framework built with UI components (stubs ready)
- Optional for all user roles

#### 3. Role-Based Access Control (RBAC)

- Three distinct roles: Admin, HR, Applicant
- Server-side role verification
- Database-level RLS policies
- 

#### 4. Content Security Policy (CSP)

- Prevents XSS attacks
- Restricts script sources
- Controls external resource loading

#### 5. Security Headers

- X-Frame-Options: DENY
- X-Content-Type-Options: nosniff
- X-XSS-Protection enabled
- Referrer-Policy: strict-origin-when-cross-origin

#### 6. Secure Password Generation

- Cryptographically secure random values
- 16+ character temporary passwords
- Auto-generated for invited users
- 

### 3.3.3 API Architecture

#### 1. REST Endpoint

The API architecture defines how the system's frontend communicates with backend services through REST endpoints. Each endpoint was assigned a specific HTTP method, function, and access level to ensure that users could only perform authorized operations.

| Endpoint     | Method | Description         | Access        |
|--------------|--------|---------------------|---------------|
| /auth/signup | POST   | Register new user   | Public        |
| /auth/login  | POST   | User authentication | Public        |
| /auth/logout | POST   | End user session    | Authenticated |
| /profiles    | GET    | Get user profile    | Authenticated |
| /profiles    | PATCH  | Update profile      | Authenticated |

|                 |       |                    |               |
|-----------------|-------|--------------------|---------------|
| /applicants     | GET   | List applications  | HR/Admin      |
| /applicants     | POST  | Submit application | Applicant     |
| /applicants/:id | PATCH | Update status      | Admin         |
| /user_roles     | GET   | Get user roles     | Authenticated |

**2. Admin Operations**

Admin operations provided secure tools for managing users, roles, notifications, and system records. The system allowed administrators to invite and delete users, send approval emails through Edge Functions, and automatically remove records older than 45 days through scheduled tasks.

| Operation   | Method             | Purpose                                            |
|-------------|--------------------|----------------------------------------------------|
| Invite User | Supabase Admin API | Create user with temp password, assign role        |
| Delete User | Supabase Admin API | Remove user from auth, profiles, roles, applicants |

|                     |                      |                                    |
|---------------------|----------------------|------------------------------------|
| send-approval-email | Edge Function        | Send professional onboarding email |
| delete-old-records  | Edge Function (Cron) | Auto-delete after 45 days          |

### 3.5 Data Gathering Techniques

- Interview

Conducted with the client to understand the process of manual onboarding and identify its concerns and requirements for improving.

Research questions:

1. Can you describe your current workflow from receiving applications to making the final hiring decision?
2. What tools or systems do you use to collect and manage applicant information?
3. What challenges do you commonly encounter during the recruitment and screening process?
4. Which part of reviewing and evaluating applicants takes the most time?

5. What tasks in your hiring process were still done manually?
6. Have you experienced issues with missing, incomplete, or duplicated applicant records? If yes, how does it affect your work?
7. How do you organize and track applicant documents such as resumes and interview notes?
8. What types of applicant information were most difficult to verify or manage?
9. What improvements would you like to see in your current recruitment and applicant tracking process?
10. What features would help you make faster and more accurate hiring decisions?

Based on the client of their current process, applicants submitted their materials through online job platforms or email which were then routed to a single shared inbox (*iloilopreboarding@...*) serving all sites. HR personnel manually reviewed each incoming email, extract candidate details, and log the information into shared spreadsheets. Throughout the hiring lifecycle, testing results and interview notes arrived in separate, unlinked email threads for the same candidate, leaving the team without a unified view of an applicant's progress.

The entire preboarding operation relied on this single email inbox paired with manual spreadsheets, using basic tags and folder structures to organize records. Document management remained bound to these individual email threads, forcing HR staff to rely on exact names or application dates to locate attachments or else, resulting in slow manual scrolling.

Because almost the entire pipeline was handled manually from initial resume screening and test link distribution to document collection and spreadsheet updates, the workflow faces severe operational delays. The most time consuming task was constantly searching the inbox to piece together basic details, such as whether a candidate applied, received a test link, or passed an evaluation. As a result , HR team members spend significantly more time searching for buried emails than evaluating talent.

Relying on manual email management introduced frequent errors and missing, incomplete, or duplicate records. Candidates often reached out from multiple email addresses, creating split threads for a single person. When an HR representative was absent, unmatched test results piled up because colleagues lacked context on the initial screening. Crucially, tracking candidate status was the most difficult information to manage; without a single dashboard displaying the pipeline, the team frequently loses track of candidates, faces delay inducing update requests from applicants, and risks duplicated efforts such as interviewing the same applicant twice.

The client's desired improvement was that it had to be one place where everything connects. Applicant emails in → status updates as they move through testing → interview → decision. No more inbox hunting, no more wondering which stage someone was in. The feature that would help him was a dashboard that shows the full pipeline: who just applied, who's waiting for the test, who passed, who needs documents. Automated status emails so applicants don't had to call for updates. Documents attached to the applicant's profile not buried in an email attachment.

### 3.6 Testing Procedures

To ensure that **SecureFlow** operates reliably and securely, testing was integrated continuously throughout the development process rather than left until the final stage. The system underwent three primary levels of testing to validate individual features, end-to-end data flows, and overall alignment with user expectations.

#### 1. Unit Testing

Unit testing focused on evaluating core system components independently to verify that their underlying logic and state management functioned correctly in isolation before integration.

- **Authentication and Identity System:** Tested user registration, email/password validation, password strength indicators (enforcing special characters and character length), password visibility toggling, and automatic form resets on tab switches.
- **Document Handling Module:** Tested the dropzone upload components, file format restrictions (PDF validation), and file size limits to ensure documents were overwritten correctly in Supabase Storage without creating duplicate entries.
- **Role-Based Access Control (RBAC):** Verified that route guards within the React Single Page Application (SPA) correctly restricted page navigation based on authenticated session roles (Applicant, HR, Admin).

#### 2. System Testing

System testing evaluated the complete, integrated application to verify that all frontend interfaces, database operations, and serverless Edge Functions worked together automatically.

- **Database and Security Policy Enforcement:** Verified that PostgreSQL Row Level Security (RLS) policies successfully restricted data access ensuring applicants could only view their own uploaded files while HR and Admin users could access

evaluation queues.

- **End-to-End Application Workflow:** Tested the full status lifecycle, tracking an applicant's profile creation from initial document submission(pending) through HR evaluation and final decision (approved or rejected).
- **Automated Notification Delivery:** Tested serverless Supabase Edge Functions (send approval email) connected to Gmail SMTP to ensure approval updates triggered real-time email notifications.
- **UI/UX and Responsiveness:** Validated dark-mode theme rendering, real-time loading states, error toast displays, and layout adaptation across mobile and desktop devices.

### 3. User Acceptance Testing (UAT)

User Acceptance Testing was conducted directly with stakeholders and the client to evaluate the live application deployed on the Vercel staging environment.

- **Client Evaluation:** Stakeholders reviewed the live administrative dashboard and applicant interfaces to confirm that document upload, verification, and automated email workflows accurately meet their actual preboarding operational needs.
- **Process Alignment:** The client verified that role permissions (such as spectator view only access for HR employees versus full action rights for Admins) met operational security requirements.
- **Sign Off:** Upon passing all evaluation scenarios, the client confirmed that the platform successfully automates manual onboarding tasks, meeting the project's core functional scope.



### **3.7 Evaluation Criteria**

To systematically measure the quality, performance, and overall effectiveness of SecureFlow, the platform was evaluated based on five key software quality criteria adapted from standard ISO software quality models:

#### **1. Functionality**

Evaluated the degree to which the system delivers all intended features and satisfies stated operational requirements.

- **Authentication and Role Management:** Verified that user registration, login, password recovery, and Role-Based Access Control (RBAC) function without logical gaps.
- **Document Processing Lifecycle:** Assessed the end-to-end flow of applicant credential submissions, file status tracking (Pending, Approved, Rejected), and inline feedback tools.
- **Automated Workflow Execution:** Measured the execution accuracy of serverless Supabase Edge Functions for sending real-time approval emails and executing scheduled data cleanup policies.

#### **2. Usability**

Assessed the clarity, accessibility, and ease of interaction across all user roles (Applicants, HR Personnel, and Administrators).

- **Interface Design and Navigation:** Evaluates the intuitiveness of the dark-mode Single Page Application (SPA) layout, dashboard component organization, and clear status indicators.
- **User Experience (UX) Feedback:** Measured how effectively the system communicates actions to users through loading spinners, interactive error displays, and clear confirmation prompts.

- **Cross-Device Responsiveness:** Evaluated interface adaptability and element alignment across both desktop monitors and mobile viewports.

### **3. Reliability**

Measured system stability, uptime, and error consistency during active operational use.

- **System Uptime and Availability:** Assessed the continuous live availability of the frontend hosted on Vercel and backend services on Supabase.
- **Error Handling and State Recovery:** Evaluated the system's capacity to handle network disruptions, invalid form inputs, or failed upload attempts gracefully without crashing or corrupting state.
- **Database Consistency:** Verified that database triggers (such as `handle_new_user`) executed reliably to prevent missing profile records or broken primary/foreign key relationships.

### **4. Security**

Evaluated the protection of sensitive candidate documents, personal information, and administrative capabilities against unauthorized access.

- **Data Isolation via Row Level Security (RLS):** Assessed the enforcement of PostgreSQL RLS policies to guarantee applicants could access only their own files, preventing unauthorized cross-user data exposure.
- **Storage and Access Controls:** Verified secure bucket permissions, zero-egress Edge Function security (utilizing Service Role Keys for administrative tasks), and safe file-handling mechanics.

- **Credential Protection:** Evaluated password complexity enforcement, secure session token handling via Supabase Auth, and protected route guards in the frontend.

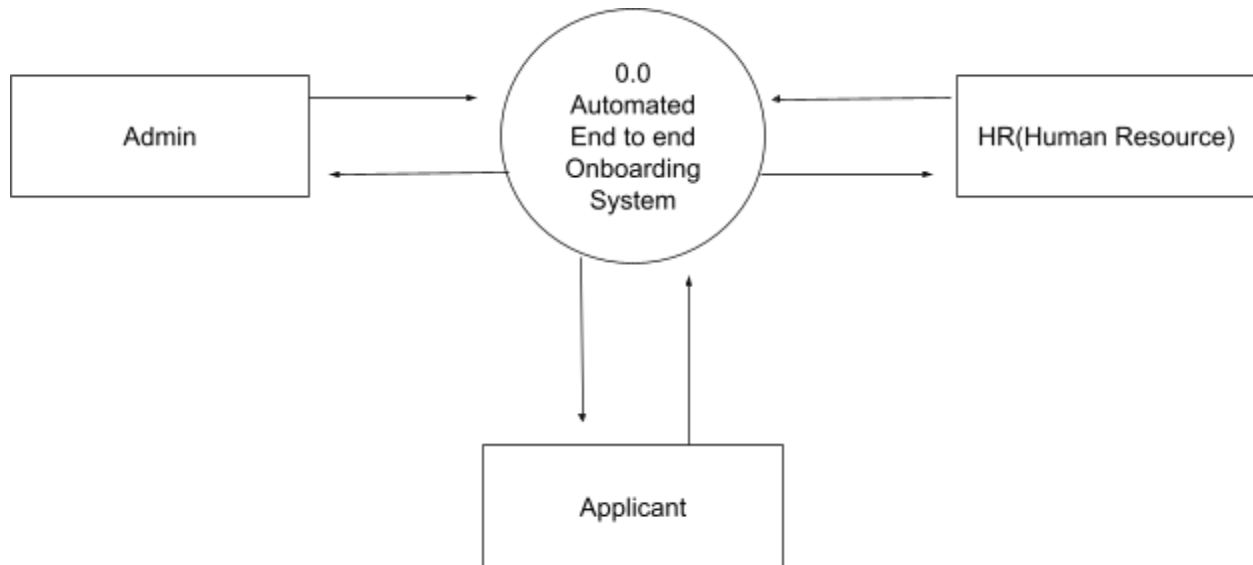
## **5. Efficiency**

Measured system response times, resource consumption, and processing speed during typical user workflows.

- **Page Load and Navigation Speed:** Evaluated the rendering speed of the React Single Page Application (SPA) without unnecessary full-page refreshes.
- **Upload and Storage Optimization:** Assessed file handling speeds and storage optimization via upsert (overwrite) logic to minimize storage footprint and network overhead.
- **Edge Function Execution:** Measured the execution latency of serverless backend functions when triggering approval emails or background processes.

## SUPPORTING DIAGRAMS (Included within Chapter 3)

### Data Flow Diagram (DFD - Level 0)

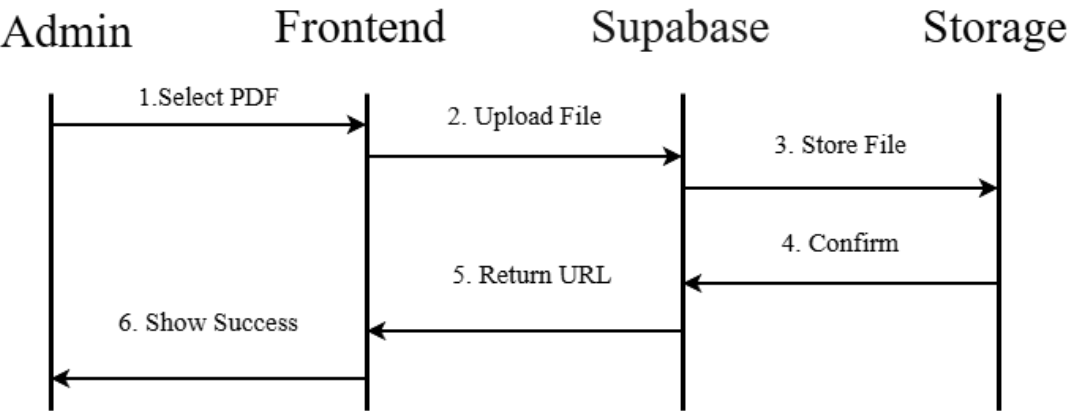


The operational workflow of SecureFlow began in the Applicant Interface where users initiate the onboarding process by submitting their comprehensive personal information and required digital credentials through a secure portal. Upon submission, the system utilized its backend integration to ensure that all data was stored securely while simultaneously triggering real-time updates across the administrative dashboard. This immediate data synchronization eliminated the loss typically associated with manual file handling. Once the data was live, Human Resource (HR) personnel were tasked with the critical phase of reviewing and verifying the submitted documentation for authenticity and compliance.

As HR interacts with the records, the system's automated logic dynamically updates the application status, providing instantaneous transparency; the system then executes an automated notification protocol to inform the user of their progress, ensuring a seamless

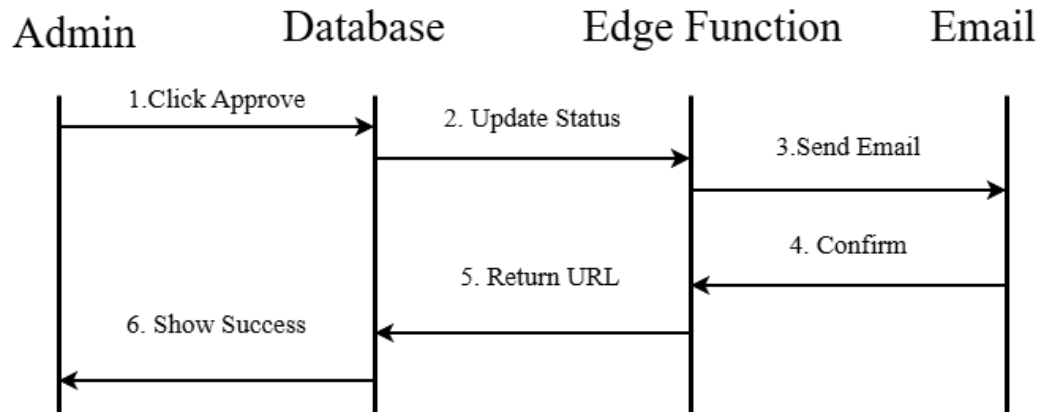
feedback loop. Overseeing the entire ecosystem was the Administrator, who possesses the high-level authority to manage and audit all organizational records, ensuring system integrity. Throughout this entire lifecycle, the architecture ensured that all data points were systematically stored in and retrieved from the centralized database, leveraging Row-Level Security to maintain strict confidentiality and data persistence.

**Document Upload Flow**



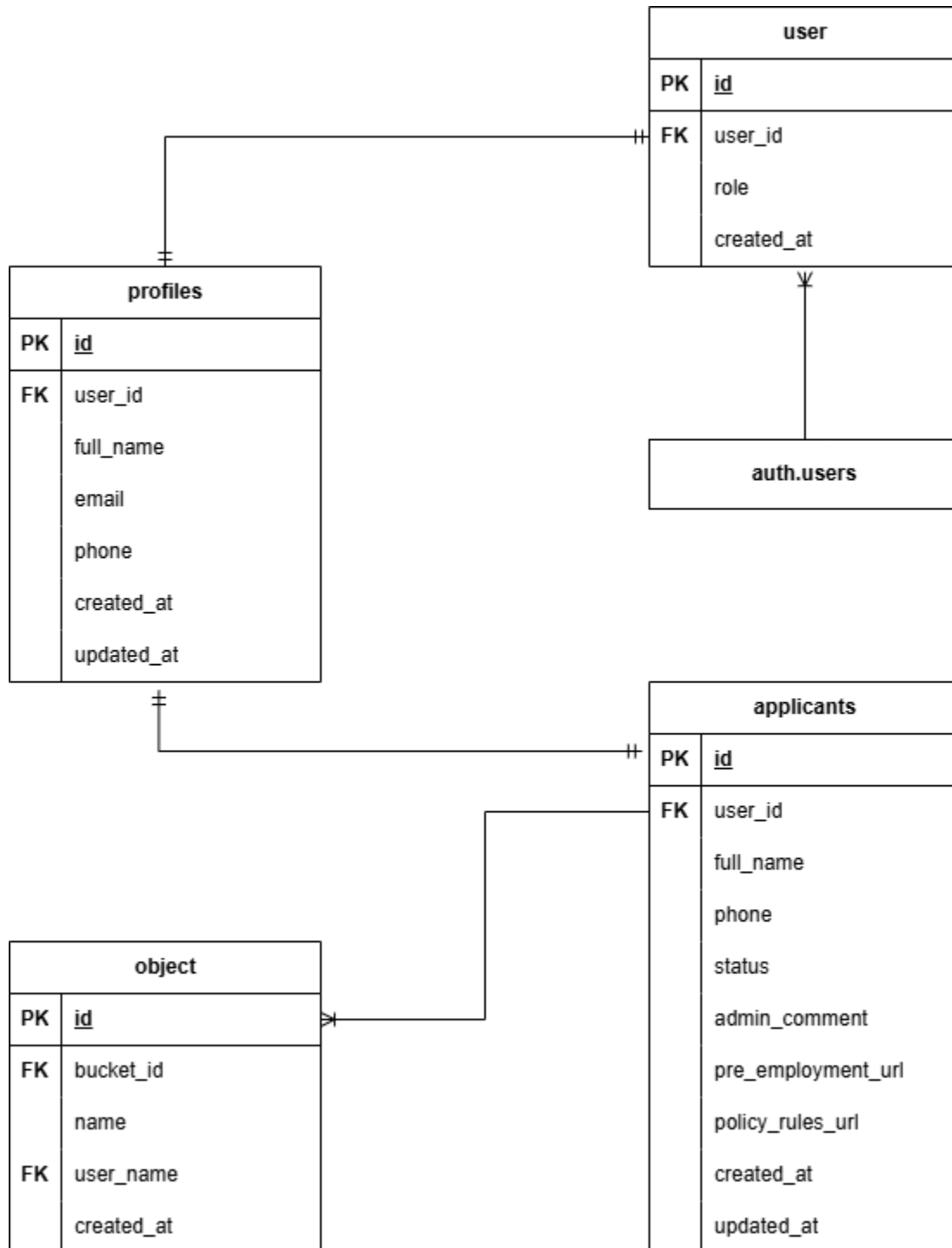
The flow illustrated the sequential execution of the pre-employment document submission feature. The process was initiated when the Applicant selects a PDF file via the React frontend. The application issued an authenticated upload request to Supabase Storage, enforcing Row Level Security (RLS) rules before writing the file to storage. Upon confirmation, the backend recorded the document file path in the applicants table, returning a success response to the client interface to confirm completion.

## Approval WorkFlow



The flow illustrated the sequence of events executed when an Admin or HR Personnel approved an applicant's preboarding status. The process was initiated when the evaluator updates the status to "Approved" on the React frontend, sending an update query to the Supabase database. Upon updating the row, a database trigger or webhook invoked a Supabase Edge Function, which securely dispatches an automated notification email to the candidate via an integrated email service. Once delivery was confirmed, the system returns a success response to update the frontend status in real time.

## Entity Relationship Diagram (ERD)



The operational lifecycle of **SecureFlow** was governed by a series of

hierarchical and collaborative relationships designed to maintain strict data integrity and workflow efficiency. The process initiates at the user level, where a newly registered **Applicant** triggers the creation of a record in `auth.user` which automatically populates a corresponding entry in `public.profiles` via a system trigger. Through an automated backend procedure, the user was assigned the default "Applicant" role in `public.user_roles`. Conversely, administrative control was managed top down by the **Admin**, who provisions accounts and manages elevated permissions for internal HR staff, enforcing Row Level Security (RLS) boundaries across all system modules.

Once an applicant account was active, an applicant's record was established to track the candidate's hiring status. The core preboarding workflow centers on direct document submission: the **Applicant** uploads digital credentials and required pre-employment forms directly to Supabase storage.objects. This action updated the applicant's record, enabled **HR personnel** to inspect and verify submitted files through the central onboarding dashboard. During this evaluation phase, HR staff reviewed the uploaded documents and executed real-time document verification. Following evaluation, **HR or Administrator** accounts update the applicant's status (e.g., from "Pending" to "Approved" or "Rejected"), which updates the database in real time. Finally, HR and Admins maintained collaborative oversight across all applicant records and stored files, ensuring pre-employment compliance and record accuracy without manual inbox searching.

**Database Scheme**

**Table:public.profiles**

| Column | Type | Constraints | Description |
|--------|------|-------------|-------------|
|--------|------|-------------|-------------|



|            |             |                     |                            |
|------------|-------------|---------------------|----------------------------|
| id         | UUID        | PRIMARY KEY         | Auto-generated             |
| user_id    | UUID        | NOT NULL,<br>UNIQUE | FK to<br>auth.users        |
| full_name  | TEXT        | NOT NULL            | User's full<br>name        |
| email      | TEXT        | NULLABLE            | Identification<br>fallback |
| phone      | TEXT        | NULLABLE            | Contact number             |
| created_at | TIMESTAMPTZ | DEFAULT NOW()       | Record<br>creation         |
| updated_at | TIMESTAMPTZ | DEFAULT NOW()       | Last update                |

The `public.profile` table stored extended user profile information synchronized with Supabase Authentication(`auth.users`). The table detailed its schema structure, maintaining non-sensitive identity metadata while linking relational references across the system's database ecosystem.

**Table:public.user**

| Column | Type | Constraints | Description |
|--------|------|-------------|-------------|
|--------|------|-------------|-------------|

|            |             |                     |                                        |
|------------|-------------|---------------------|----------------------------------------|
| id         | UUID        | PRIMARY KEY         | Auto-generated                         |
| user_id    | UUID        | NOT NULL,<br>UNIQUE | FK to<br>auth.users                    |
| role       | TEXT        | NOT NULL            | 'admin',<br>'employee',<br>'applicant' |
| created_at | TIMESTAMPTZ | DEFAULT NOW()       | Role<br>assignment<br>date             |

To implement Role-Based Access Control (RBAC) securely across the application, system access levels were governed by the public.user\_role table. The table detailed the schema structure that maps authenticated users to specific system roles (Applicant, HR, Admin), enforcing permission boundaries throughout the cloud-native environment.

**Table:public.applicants**

The core entity driving the onboarding submission workflow was the public.applicant table hosted within the Supabase PostgreSQL database.details its data dictionary, outlining attribute definitions, data types, relational constraints, and cloud storage pointers utilized for document management.

| Column             | Type        | Constraints       | Description               |
|--------------------|-------------|-------------------|---------------------------|
| id                 | UUID        | PRIMARY KEY       | Auto-generated            |
| user_id            | UUID        | NOT NULL, FK      | FK to auth.users          |
| full_name          | TEXT        | NOT NULL          | Applicant name            |
| phone              | TEXT        | NULLABLE          | Contact number            |
| status             | TEXT        | DEFAULT 'pending' | pending/approved/rejected |
| admin_comment      | TEXT        | NULLABLE          | Admin review notes        |
| pre_employment_url | TEXT        | NULLABLE          | PDF storage path          |
| policy_rules_url   | TEXT        | NULLABLE          | PDF storage path          |
| created_at         | TIMESTAMPTZ | DEFAULT NOW()     | Application date          |
| updated_at         | TIMESTAMPTZ | DEFAULT NOW()     | Last update               |

**Table:Storage.objects(PDF Files)**

In addition to structured relational tables, binary asset management was handled by the `storage.objects` table inside Supabase Cloud Storage. It detailed the metadata schema utilized to reference uploaded PDF documents, linking storage objects back to individual authenticated user accounts while enforcing access control.

| Column    | Type        | Constraints                    | Description                                                                       |
|-----------|-------------|--------------------------------|-----------------------------------------------------------------------------------|
| id        | UUID        | PRIMARY KEY,<br>Auto-generated | Unique identifier for the stored file object entity.                              |
| bucket_id | TEXT        | FOREIGN KEY                    | Key identifying the target storage bucket (e.g., 'pre-employment-docs').          |
| name      | TEXT        | NOT NULL                       | File system path and original filename of the uploaded PDF asset.                 |
| user_id   | UUID        | FOREIGN KEY                    | Relational key linking object ownership directly to <code>auth.users(id)</code> . |
| created_a | TIMESTAMPTZ | DEFAULT                        | Timestamp recording the exact time of document                                    |

|   |  |       |         |
|---|--|-------|---------|
| t |  | NOW() | upload. |
|---|--|-------|---------|

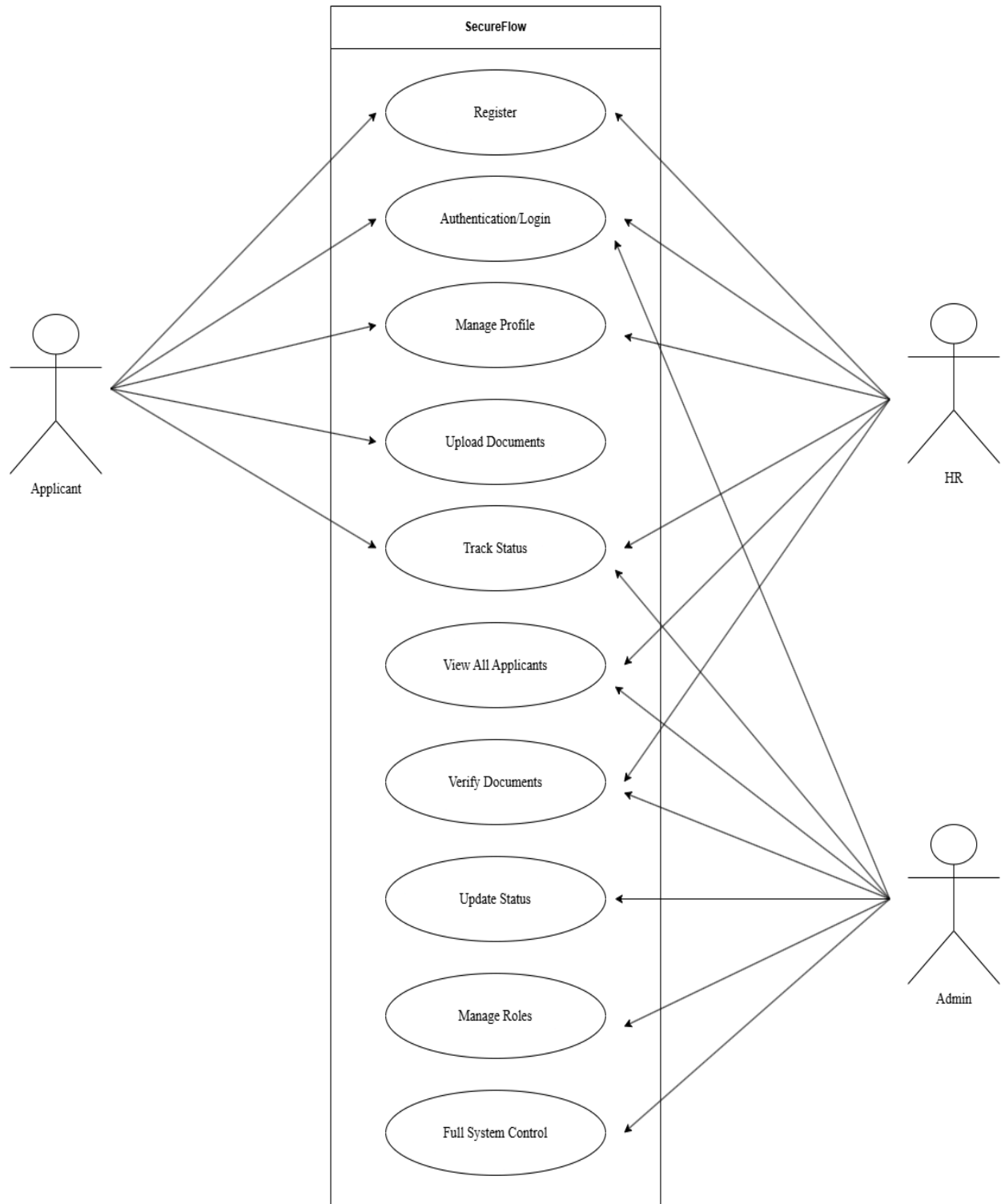
**Row Level Security(RLS) Policies**

To enforce strict Role-Based Access Control (RBAC) at the database layer and prevent cross-role data leakage, Row Level Security (RLS) policies were configured directly on PostgreSQL tables in Supabase Cloud. It outlines the specific security rules applied to the core system entities.

| Table      | Policy Name              | Description                        |
|------------|--------------------------|------------------------------------|
| profiles   | Users view own profile   | Users could read their own profile |
| profiles   | Users update own profile | Users could update their own data  |
| profiles   | Admins view all          | Admins could view all profiles     |
| user_roles | Role assignment          | Automatic role                     |

|            |                     |                                       |
|------------|---------------------|---------------------------------------|
|            |                     | assignment on signup                  |
| applicants | Applicants view own | Applicants saw only their application |
| applicants | HR view all         | HR could view all applications        |
| applicants | Admins full access  | Admins could perform all operations   |

## Use Case Diagram



The functional capabilities of SecureFlow were defined through the interactions of three primary actors : the Applicant, the HR Personnel, and the Administrator operating across ten core use cases within the system boundary. The onboarding process initiated with the Register Account use case, which served as the intake point for candidates, automatically requiring the Authenticate / Login use case through an included relationship to establish a secure session. Once logged in, candidates engaged with the Manage Profile use case to input their personal contact information, establishing their digital identity within the platform.

The core operational value for candidates was realized through the Upload Pre-Employment Documents use case, which was exclusive to the Applicant actor. Uploading requirements automatically connects to the Track Application Status use case, providing candidates with real-time visibility into their pipeline progress (*Pending, Approved, or Rejected*) and directly resolving the lack of status transparency in the current manual process.

As submissions enter the system, the View All Applicants, and Verify Submitted Documents use cases became the primary responsibilities of HR Personnel and Administrators. Evaluators access a centralized dashboard to review incoming applications, inspect uploaded pre-employment files, and log evaluation decisions without searching through scattered email threads. Finally, system governance was maintained through the Manage User Roles and Delete / Archive Record use cases, and Update Application Status reserved exclusively for the Administrator actor. These high-level use cases granted admins exclusive control to provision internal accounts, adjust user access levels, and purge outdated records to maintain data privacy compliance.



## CHAPTER 4

### SYSTEM DEVELOPMENT, IMPLEMENTATION, TESTING, EVALUATION, RESULTS, AND DISCUSSION

#### 4.2 Overview of the Developed System

The **SecureFlow** application was an automated web-based onboarding system developed to streamline pre-employment document submission and evaluation while ensuring robust data security, role-based access control, and real-time status tracking. The system addressed the operational bottlenecks and security vulnerabilities inherent in traditional paper-heavy and unencrypted digital onboarding workflows. By replacing fragmented communication channels (such as physical paper submissions and standard email attachments) with a centralized Single Page Application (SPA), SecureFlow provided dedicated, role-tailored portals for Applicants, HR Personnel, and System Administrators.

Unlike conventional Human Resource Information Systems (HRIS) that combined broad employee management features, SecureFlow focused specifically on the automated preboarding lifecycle. The platform enabled job applicants to securely register, upload required pre-employment PDF credentials, and monitor their verification status in real time. For HR Personnel and Administrators, the application provided an intuitive verification dashboard equipped with embedded document viewing, per-document feedback logging, status workflow updating ( Pending ,Approved, Rejected) and automated email notifications triggered via serverless Edge Functions.

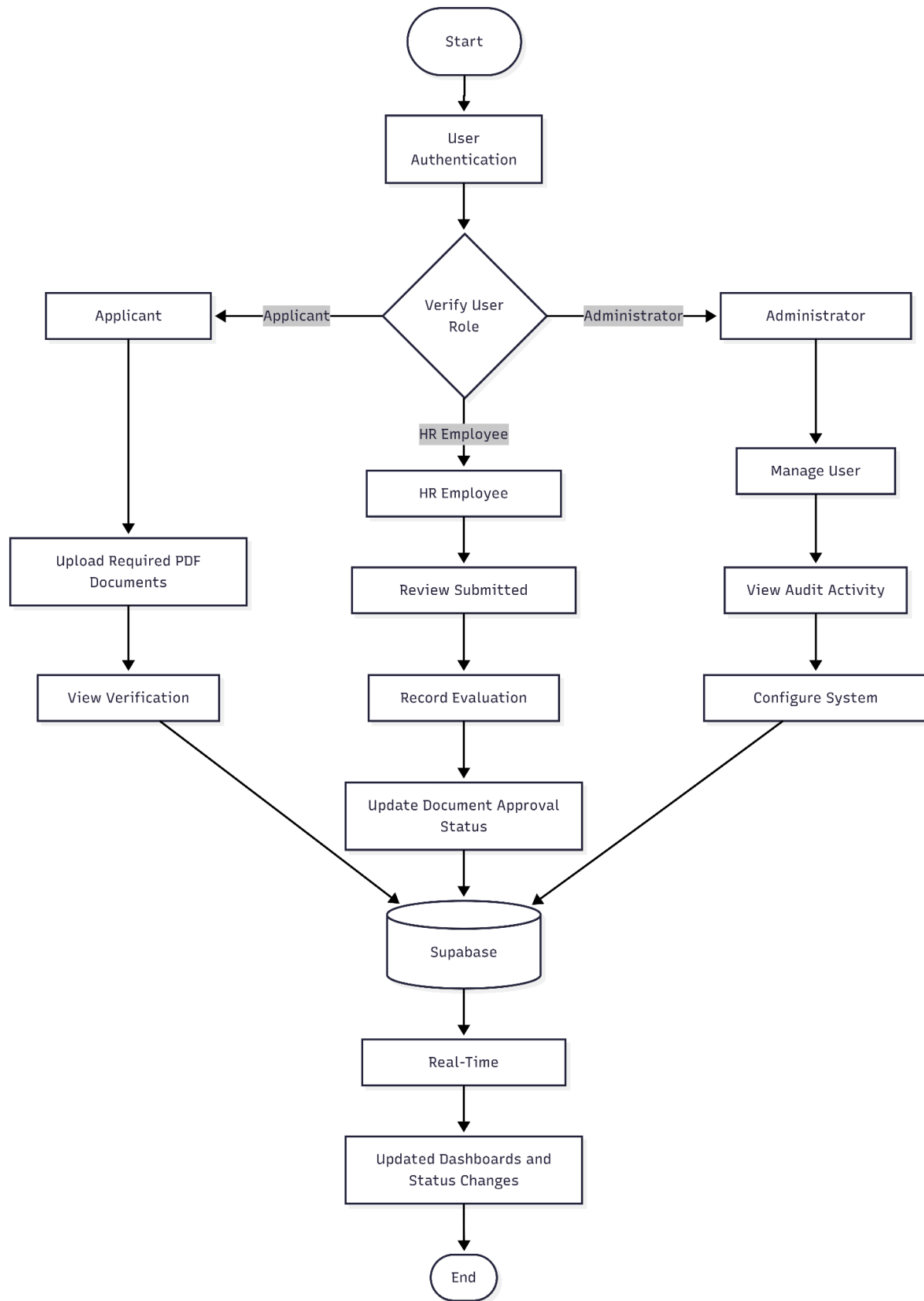
The platform provided small and medium enterprises (SMEs) and corporate HR departments with an efficient, auditable, and secure onboarding solution that accelerates time-to-hire, enforces data privacy compliance, and minimizes administrative overhead.

The major objectives of the developed system were as follows:

- Automated the registration, document upload, and tracking of pre-employment onboarding workflows.
- Secured candidate credentials during transmission and storage using cloud-native encryption and Row Level Security (RLS).
- Provided dedicated, role user interfaces for Applicants, HR Personnel, and Administrators.
- Enabled real-time document inspection, per document feedback logging, and approval status updates.
- Eliminated unencrypted file distribution by centralizing credential management within secure object storage buckets.
- Supported administrative oversight through automated audit logging and automated email notifications via SMTP integration.

The developed SecureFlow follows modern software engineering and cybersecurity principles by incorporating responsive frontend design, serverless backend architecture, granular access control policies, and automated event triggers. The completed system consists of several interconnected modules that work together to capture, store, verify, and notify users throughout the preboarding lifecycle.

**Figure 4.1: Overall System Workflow Diagram of SecureFlow**



The overall workflow began with user authentication. Once authenticated through Supabase Auth, the user's assigned role (Applicants, HR employee, Administration) determines their access path. Applicants were directed to their personal onboarding dashboard to upload required PDF documents and view verification updates. HR Personnel accessed the evaluation workspace to inspect candidate files, record granular notes. System Administrators utilized the administrative panel to manage user roles, audit activity logs, update document approval statuses and configure system parameters. Real-time updates and status changes were dynamically synchronized across all active user sessions without requiring page reloads.

#### **4.3 System Development Methodology**

The developed SecureFlow adopted **DevOps with Agile Methodology**, an integrated approach that combines the iterative feature delivery of Agile with the Continuous Integration and Continuous Deployment (CI/CD) principles of DevOps. This combined methodology was selected to ensure that SecureFlow remains highly adaptable to evolving user requirements while maintaining a secure, highly reliable, and continuously updated live cloud environment throughout the software development life cycle (SDLC).

By leveraging this hybrid framework, project plans were structured into iterative sprints, while technical execution relied on an automated delivery pipeline using GitHub, Vercel, and Supabase. In this cloud-native pipeline, traditional intermediate software releases were fully automated and merged directly into the Deployment phase, allowing approved feature updates to be tested and shipped to production seamlessly.

DEVOPS METHODOLOGY LIFECYCLE



**1. Plan**

The foundational phase involved collecting functional and non-functional requirements through direct interviews, advisory

consultations, and workflow analysis of manual preboarding practices. The gathered requirements were translated into a structured Product Backlog composed of role-based user stories (Applicant, HR Employee, and Administrator). To mitigate technical risk, this backlog was divided into structured two week Agile sprints, prioritized core infrastructure, such as user authentication and database schemas in early iterations. Simultaneously the DevOps foundation was established by initializing a GitHub version-control repository and configuring automated pipeline triggers for continuous delivery.

## **2. Design**

The design phase transformed functional requirements into detailed technical and security specifications prior to frontend construction. Database structures were mapped using Entity-Relationship Diagrams (ERD) to define primary keys, foreign key constraints, and database triggers. To safeguard sensitive applicant documentation, Row Level Security (RLS) policies were explicitly modeled at the database level to enforce strict Role-Based Access Control (RBAC). System interactions were further defined using Use Case and Sequence Diagrams to model data flows between the React client, Supabase Storage, and serverless Edge Functions, accompanied by high-fidelity wireframes drafted for mobile and desktop viewports.

## **3. Code**

Once architecture designs were finalized, the platform was developed as a Single Page Application (SPA) using React.js and Tailwind CSS to ensure smooth dashboard transitions without full-page reloads. The frontend was broken down into modular, reusable components, including file dropzones, real-time status badges, and administrative evaluation tables. The backend utilized Supabase Cloud to manage user authentication, PostgreSQL database tables, and secure PDF storage buckets. Serverless Edge Functions were developed in TypeScript to handle automated email notifications via SMTP and background maintenance tasks. All codebase changes were managed using Git

feature-branching workflows to build separate system capabilities concurrently without version conflicts.

#### **4. Testing**

Testing was embedded continuously into the development pipeline rather than deferred to the end of the SDLC. Every time a feature branch was merged into the main repository, automated build checks and manual regression tests were executed across three key domains:

- **Authentication Testing:** Validated user registration, role mapping, password strength enforcement, visibility toggles, and secure redirection.
- **Database & Security Testing:** Verified automatic role assignments, tested Row Level Security (RLS) policies to prevent cross-role data leakage, and validated end-to-end applicant submission workflows.
- **UI/UX & Responsiveness Testing:** Verified layout responsiveness, loading indicators, error feedback displays, and dark-mode styling across desktop and mobile devices.

#### **5. Deployment**

Once a feature branch successfully passed all testing gates, it was merged into the main GitHub branch, immediately triggering an automated Continuous Deployment (CD) build on Vercel. In this continuous pipeline, software releasing and hosting were unified into a single automated step. Linking the GitHub repository directly to Vercel and Supabase Cloud eliminated manual file transfers and scheduled downtime, ensuring that stakeholders, HR personnel, and administrators always had immediate access to the latest live version of the application.

#### **6. Operations**

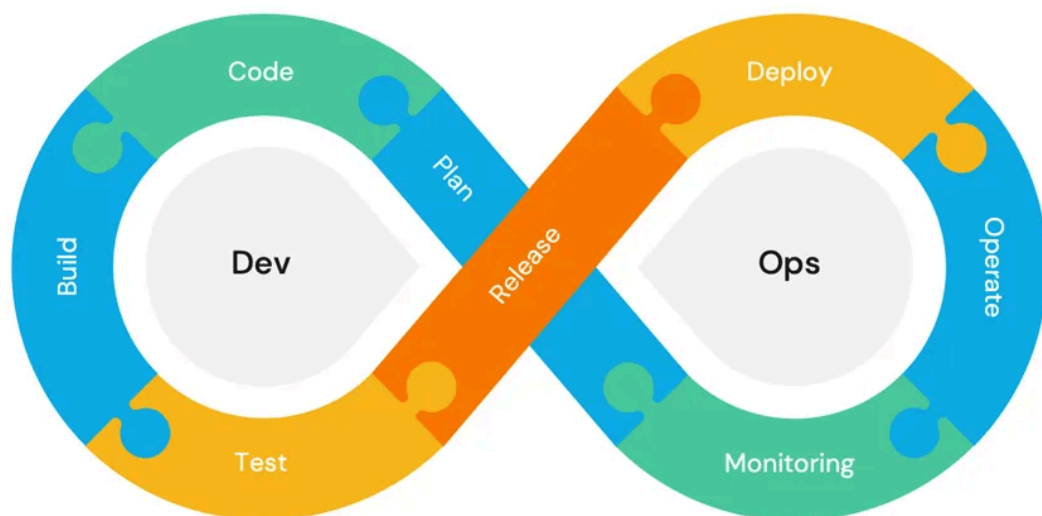
Once live, operational focus shifted to platform reliability, runtime performance, and backend security. This phase involves managing cloud

serverless execution, ensuring uninterrupted Supabase Edge Function processing for automated HR notifications, and maintaining active Row Level Security (RLS) database policies. Ongoing database configuration adjustments and security patch deployments were applied directly to the live environment to ensure continuous stability.

## 7. Monitoring

The final phase focused on proactive system observation and continuous feedback. Backend logs, runtime error reports, and database performance metrics were regularly reviewed in Supabase to identify latency or security anomalies. Qualitative feedback gathered from live administrative and client interactions during this phase was analyzed and fed directly back into **Phase 1 (Plan)**, initiating the next iterative development loop for continuous system optimization.

**Figure 4.2: DevOps Software Development Life Cycle Model**

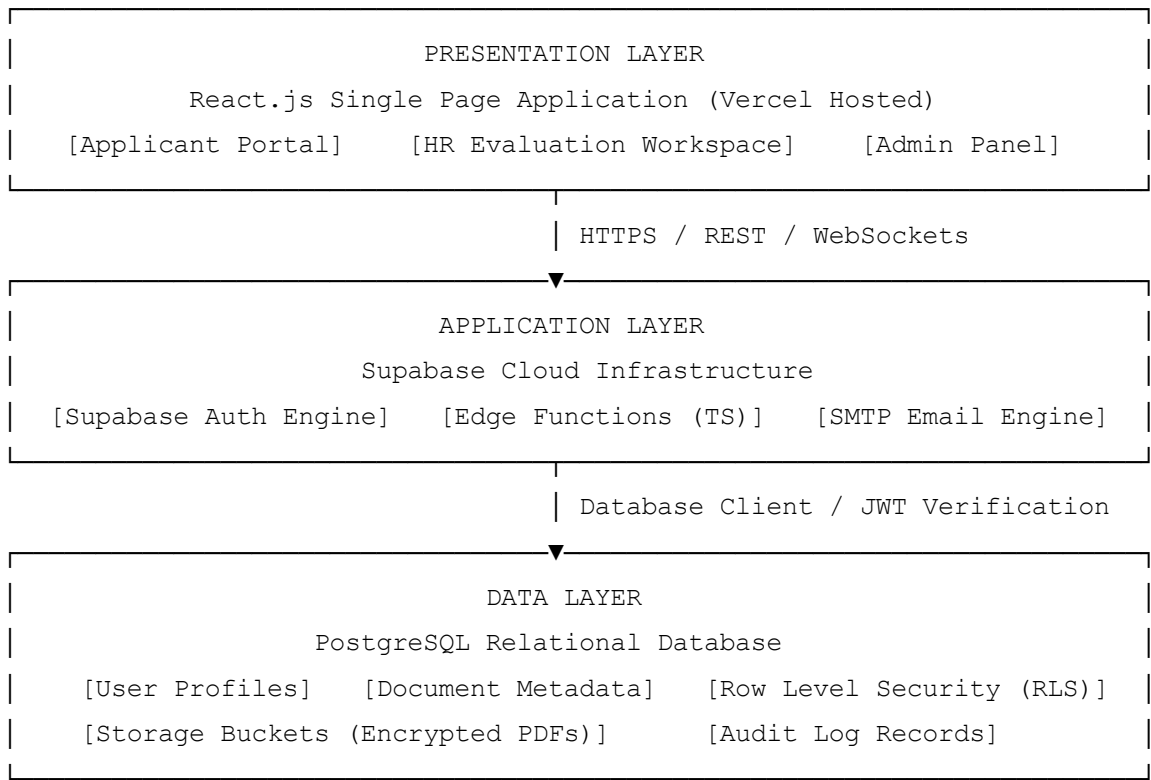




4.4 System Architecture

SecureFlow utilized a modern **Three-Tier Cloud-Native Architecture**, strictly separating the presentation layer, application layer, and data layer. This architectural pattern ensured modularity, high performance, maintainability, and enterprise-grade data isolation.

Figure 4.3: Three-Tier System Architecture of SecureFlow



1. Presentation Layer (Frontend Client)

The presentation layer consists of a responsive Single Page Application (SPA) built with React.js and styled using Tailwind CSS. It handles user interactions, renders dynamic dashboards, provides drag-and-drop file upload interfaces, and displays real-time status updates.

- **Key Modules:** Login/Register Pages, Applicant Document Upload Dropzone, HR Verification Table & PDF Viewer, Admin User Management Panel, System Settings.

## 2. Application Layer (Backend Business Logic)

The application layer runs on Supabase serverless infrastructure. It manages user authentication sessions via JSON Web Tokens (JWT), processes business logic, executes database functions, and orchestrates serverless background workers.

- **Key Components:**
  - **Supabase Auth Engine:** Handled encrypted user registration, password hashing, JWT session signing, and role assignment.
  - **Supabase Edge Functions:** Serverless TypeScript functions that execute backend logic (such as sending automated verification emails via Gmail SMTP) without exposing API keys.
  - **Realtime Subscription Engine:** Listens to database mutation events and pushes status updates instantaneously to connected clients via WebSockets.

## 3. Data Layer (Persistence & Object Storage)

The data layer relied on a managed PostgreSQL relational database and cloud object storage. It stored structured application metadata and unencrypted/encrypted document objects while strictly enforcing data isolation rules.

- **Key Components:**
  - **PostgreSQL Relational Database:** Stored user profiles, application tracking records, document metadata, and system audit logs.
  - **Row Level Security (RLS):** Database-level security policies that restrict row visibility based on the requesting user's JWT identity (e.g., Applicants could only view

their own uploaded records, whereas HR could access assigned candidate files).

- **Supabase Storage Buckets:** Secure object storage configured with private access controls and file-type restrictions (PDF only) to host uploaded candidate files securely.

4.5 Hardware and Software Specifications

4.5.1 Hardware Requirements

Table 4.1 outlined the hardware specifications required for developing, hosting, and interacting with the SecureFlow platform efficiently.

Table 4.1: Hardware Specifications

| Component          | Minimum Specification       | Recommended Specification                  |
|--------------------|-----------------------------|--------------------------------------------|
| Processor          | Dual-Core 2.0 GHz Processor | Quad-Core Intel i5 / AMD Ryzen 5 or higher |
| Memory (RAM)       | 4 GB                        | 8 GB or higher                             |
| Storage            | 128 GB SSD                  | 256 GB SSD or higher                       |
| Display Resolution | 1366 × 768 pixels           | 1920 × 1080 Full HD Display                |

|                           |                           |                                    |
|---------------------------|---------------------------|------------------------------------|
| <b>Network Connection</b> | 5 Mbps Broadband          | 25 Mbps Stable Internet Connection |
| <b>Client Devices</b>     | Desktop / Laptop / Tablet | Workstation / Desktop Computer     |

**4.5.2 Software Requirements**

Table 4.2 details the software tools, frameworks, and environments utilized to construct, test, and deploy the application.

**Table 4.2: Software Environment Specifications**

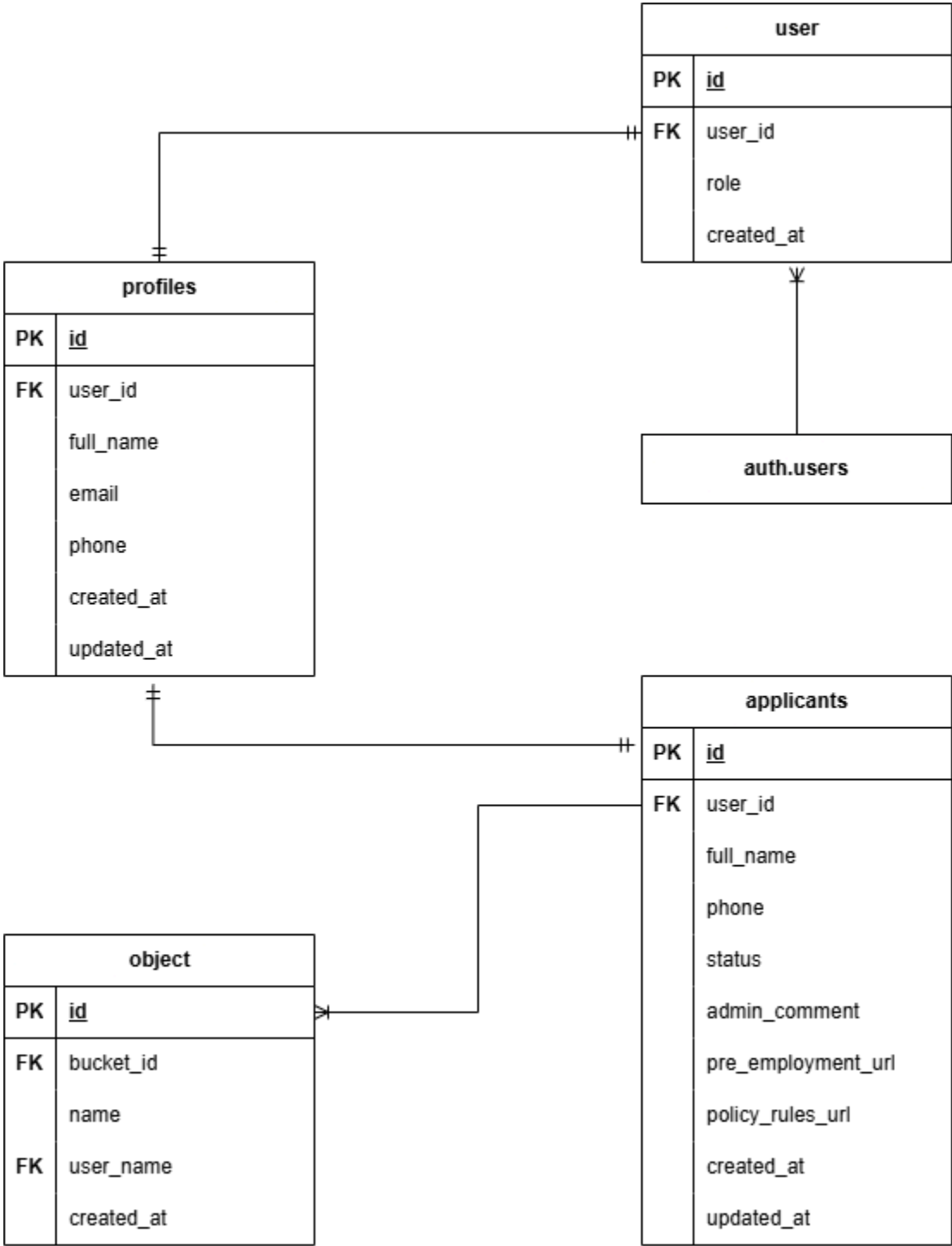
| <b>Software Tool / Technology</b>  | <b>Category / Purpose</b>       |
|------------------------------------|---------------------------------|
| <b>Windows 11 / macOS</b>          | Development Operating System    |
| <b>Visual Studio Code (VSCode)</b> | Source Code Editor & IDE        |
| <b>React.js (v18+)</b>             | Frontend SPA Web Framework      |
| <b>Tailwind CSS</b>                | Utility-First Styling Framework |

|                                |                                                 |
|--------------------------------|-------------------------------------------------|
| <b>Node.js &amp; npm</b>       | JavaScript Runtime & Package Manager            |
| <b>Supabase Cloud</b>          | Backend-as-a-Service (BaaS)<br>Infrastructure   |
| <b>PostgreSQL</b>              | Relational Database Management System           |
| <b>Supabase Auth &amp; RLS</b> | Session Handling & Row-Level Security Policies  |
| <b>Supabase Edge Functions</b> | Serverless TypeScript Logic & Email Integration |
| <b>Git &amp; GitHub</b>        | Version Control System & Source Code Repository |
| <b>Vercel</b>                  | Production Cloud Hosting & CI/CD Deployment     |
| <b>Google Chrome / Edge</b>    | Target Web Browsers for Execution & Testing     |

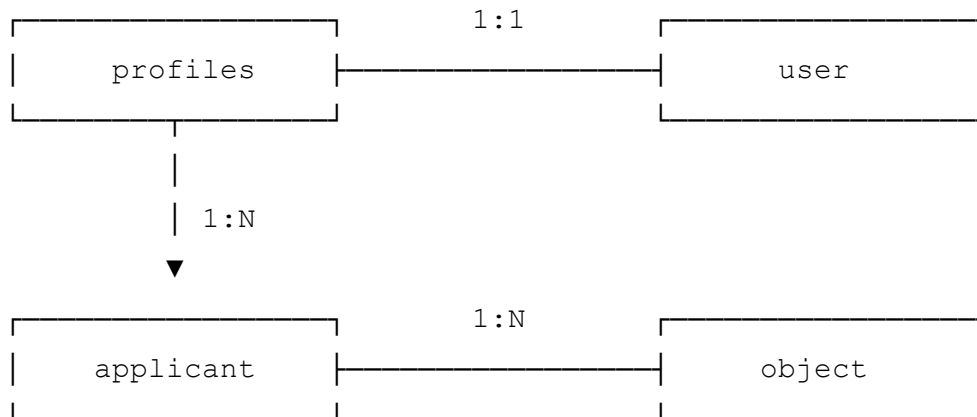
#### **4.6 Database Design**

The database architecture served as the foundation of SecureFlow, storing user identities, profile attributes, document upload metadata, approval statuses, and system audit records. A relational PostgreSQL model was engineered to maintain data integrity, eliminate redundancy through normalization, and enforce access control directly at the database level.

Figure 4.4: Entity Relationship Diagram (ERD) of SecureFlow



The relational schema comprised the following primary entities:



### 1. User Table

Stores systemic user permissions and access designations.

| Column  | Type | Constraints      | Description                            |
|---------|------|------------------|----------------------------------------|
| id      | UUID | PRIMARY KEY      | Auto-generated                         |
| user_id | UUID | NOT NULL, UNIQUE | FK to auth.users                       |
| role    | TEXT | NOT NULL         | 'admin',<br>'employee',<br>'applicant' |

### 2. User Profiles Table (profiles)



Extends Supabase Auth users with custom application metadata.

| Column     | Type        | Constraints         | Description                |
|------------|-------------|---------------------|----------------------------|
| id         | UUID        | PRIMARY KEY         | Auto-generated             |
| user_id    | UUID        | NOT NULL,<br>UNIQUE | FK to<br>auth.users        |
| full_name  | TEXT        | NOT NULL            | User's full<br>name        |
| email      | TEXT        | NULLABLE            | Identification<br>fallback |
| phone      | TEXT        | NULLABLE            | Contact<br>number          |
| created_at | TIMESTAMPTZ | DEFAULT NOW()       | Record<br>creation         |

**3. Applicant Table (documents)**

Tracks candidate file submissions, storage paths, and approval statuses.

| Column | Type | Constraints | Description    |
|--------|------|-------------|----------------|
| id     | UUID | PRIMARY KEY | Auto-generated |

|                       |      |                      |                                   |
|-----------------------|------|----------------------|-----------------------------------|
| user_id               | UUID | NOT NULL, FK         | FK to<br>auth.users               |
| full_name             | TEXT | NOT NULL             | Applicant<br>name                 |
| phone                 | TEXT | NULLABLE             | Contact<br>number                 |
| status                | TEXT | DEFAULT<br>'pending' | pending/app<br>roved/rejec<br>ted |
| admin_commen<br>t     | TEXT | NULLABLE             | Admin<br>review<br>notes          |
| pre_employe<br>nt_url | TEXT | NULLABLE             | PDF storage<br>path               |
| policy_rules<br>_url  | TEXT | NULLABLE             | PDF storage<br>path               |

#### 4. Document Audit Logs Table (object)

Maintained an immutable historical record of administrative status changes for auditability.

| Column    | Type | Constraints                    | Description                                                              |
|-----------|------|--------------------------------|--------------------------------------------------------------------------|
| id        | UUID | PRIMARY KEY,<br>Auto-generated | Unique identifier for the stored file object entity.                     |
| bucket_id | TEXT | FOREIGN KEY                    | Key identifying the target storage bucket (e.g., 'pre-employment-docs'). |
| name      | TEXT | NOT NULL                       | File system path and original filename of the uploaded PDF               |

|            |             |               |                                                                     |
|------------|-------------|---------------|---------------------------------------------------------------------|
|            |             |               | asset.                                                              |
| user_id    | UUID        | FOREIGN KEY   | Relational key linking object ownership directly to auth.users(id). |
| created_at | TIMESTAMPTZ | DEFAULT NOW() | Timestamp recording the exact time of document upload.              |
| Column     | Type        | Constraints   | Description                                                         |

## **CHAPTER 5**

*(5 to 10 pages..)*

### **CONCLUSION AND RECOMMENDATIONS**

#### **5.1 Introduction**

This chapter provided a comprehensive conclusion to the development of the SecureFlow system. It synthesized the findings of the study, evaluated the extent to which the research objectives were achieved, and presented recommendations for future system enhancements and implementation strategies to further improve the system's effectiveness and usability for HR departments and organizations. Additionally, this chapter highlighted the overall contributions of the study to the field of Information Technology by demonstrating the successful implementation of a secure, cloud-based onboarding platform that streamlined document management, strengthened data security, and enhanced the efficiency of the employee onboarding process. It also discussed the limitations encountered during the development of the system and identified opportunities for future research and continuous system improvement.

#### **5.2 Summary of Findings**

The development and evaluation of the Secureflow yielded significant results aligned with the research objectives:

- The system successfully automated the digital submission, storage, and management of pre-employment documents, eliminating the need for manual paperwork and reducing the risks associated with physical document handling.
- The role-based approval workflow effectively streamlined the review process by enabling HR personnel and administrators to review applications, provide structured feedback, and approve or reject submissions efficiently, resulting in reduced processing delays.
- Real-time status tracking through Supabase Realtime and automated email notifications ensured that applicants remained informed throughout the onboarding process, minimizing follow-up inquiries and improving communication transparency.
- The implementation of Role-Based Access Control (RBAC) and Row-Level Security (RLS) strengthened data security by ensuring that users could only access information according to their assigned roles, helping protect sensitive applicant and employment records.
- The cloud-based architecture, utilizing React, Supabase, and Vercel, provided a scalable, reliable, and centralized platform that supported secure document storage, real-time synchronization, and efficient system accessibility.
- The adoption of the DevOps development approach enabled continuous development, testing, deployment, and monitoring throughout the project lifecycle, contributing to improved software quality, faster issue resolution, and a more stable application.
- User Acceptance Testing (UAT) confirmed that the system was intuitive and easy to use, reducing the learning curve for both HR personnel and applicants while improving the overall user experience.
-

### **5.3 Conclusion**

The system had proven to be a viable and effective solution for organizations seeking to modernize their employee onboarding process. By transforming manual paperwork into a structured digital workflow, the system empowers HR teams to manage document submissions, approvals, and applicant status with greater efficiency. The project successfully addressed the need for an affordable, responsive, and secure tool, demonstrating that structured document management could be made accessible to organizations of all sizes. The research questions were answered affirmatively, confirming that the system enhances operational efficiency and improves the overall onboarding experience.

### **5.4 Recommendations**

To ensure the continued relevance and efficacy of the TOPBuy App, the following recommendations were proposed:

#### **System Improvements**

- Integrate automated notification alerts for pending application reviews and documents requiring re-submission, to further improve the processing workflow.
- Enhance security protocols by completing and enforcing two-factor authentication, which the portal already supports through email-based OTP verification, to better protect sensitive applicant and employment data.
- Develop advanced export features allowing approval and applicant reports to be generated in multiple formats (e.g., PDF, CSV, Excel) for easier sharing and integration with external HR and payroll systems.

- Implement a comprehensive audit trail that records all user activities, including document uploads, status updates, approvals, and administrative actions, to improve accountability and support compliance with organizational policies.
- Provide customizable dashboard analytics that display key performance indicators (KPIs), such as application processing time, approval rates, and pending document statistics, to assist HR personnel in monitoring onboarding performance.
- Introduce automated document expiration and archival management to securely archive completed onboarding records while maintaining efficient storage utilization.

#### **Future Research Directions**

- Explore the integration of machine learning algorithms to enable predictive analytics, such as forecasting application volume and identifying documents likely to be returned for revision based on historical patterns.
- Conduct a longitudinal study to analyze the long-term impact of the system on onboarding efficiency and employee retention.
- Investigate the feasibility of a mobile-native application to provide offline functionality for applicants with intermittent internet connectivity.
- Evaluate the integration of electronic signature (e-signature) functionality to enable legally compliant digital signing of employment contracts and onboarding documents.
- Investigate the application of artificial intelligence for automated document verification, including detecting incomplete or inconsistent submissions before HR evaluation.
- Assess the scalability of the system when deployed in larger organizations with multiple departments, branches, or geographically distributed offices.



## **Implementation Strategies**

- HR and admin staff should conduct training sessions to ensure consistent document upload and review habits, which were critical for accurate records.
- Organizations were encouraged to perform regular data backups and system
- Organizations should establish a regular security review schedule, including periodic vulnerability assessments, user access audits, and updates to Role-Based Access Control (RBAC) policies to maintain compliance with the Data Privacy Act of 2012.
- The system should follow a continuous maintenance and update process using DevOps practices, including version control, continuous testing, and automated deployment, to ensure that future enhancements could be delivered efficiently without disrupting system availability.
- System administrators should continuously monitor application logs, database performance, and real-time synchronization services to promptly detect and resolve operational issues.

## **5.5 Impact of the Study**

SecureFlow contributed to the field of Information Technology by demonstrating the practical implementation of a cloud-based human resource onboarding system that integrated digital document management, real-time synchronization, and role-based access control

within a secure and scalable environment. By leveraging modern web technologies such as React, Supabase, and DevOps practices, the study illustrated how contemporary software development approaches improved organizational efficiency, streamlined recruitment workflows, and strengthened the protection of sensitive applicant information. The platform provided HR personnel with an accessible and centralized onboarding solution that bridged the gap between traditional paper-based processes and secure digital automation. Although the project had limitations, including its reliance on a stable internet connection and email-based verification, the findings demonstrated the significant potential of cloud-based technologies to enhance operational efficiency, support regulatory compliance, and promote digital transformation in modern organizations.

## **5.6 Summary**

In summary, SecureFlow represented a successful integration of software engineering principles and human resource management. It served as a testament to the effectiveness of tailored technology solutions in addressing real-world business challenges. This research provided a solid foundation for future developers to build upon, contributing to the broader goal of digital transformation in organizational human resource processes. Furthermore, the study demonstrated that integrating cloud computing, real-time synchronization, and role-based security mechanisms into a centralized onboarding platform significantly improved the efficiency, transparency, and security of the recruitment process. The successful implementation and evaluation of the system confirmed that modern web technologies could effectively address the limitations of traditional onboarding methods while supporting organizations in their transition toward secure, reliable, and technology-driven human resource management.

